PROJECT REPORT

AI-Based Synthetic Telecom Customer Data Generation and Churn Prediction Using GANs and Deep Learning

Submitted by

Simon V

Batch: TN-DS-AN B02

Project Category: Deep Learning / Machine Learning / Generative AI / GAN / Data Science

Institution Name - Entri Elevate

Department - Advanced Program in Data Science - TN

Supervisor: Sree Varshaa S, Data Science Trainer

Academic Year: 2026–2027

Submission Date: 28-10-2026

AI-Based Synthetic Telecom Customer Data Generation and Churn Prediction Using GANs and Deep Learning

A project report submitted in partial fulfilment of the requirements for
[Degree / Programme name - Data Science with Generative AI**]**

by

Simon V
Register / Roll Number: N/A

Under the guidance of
Supervisor name, designation, department - Sree Varshaa S, Data Science Mentor

[Entri Elevate]
[Chennai, State]
[October 2026]

Declaration

I, **Simon V**, hereby declare that the project report entitled "AI-Based Synthetic Telecom Customer Data Generation and Churn Prediction Using GANs and Deep Learning" is a record of original work carried out by me under the guidance of [Supervisor name - Sree Varshaa S] at [Institution Name - Entri Elevate]. To the best of my knowledge, this work has not been submitted, in part or in full, for the award of any other degree, diploma or certificate. Sources of data, methods and text taken from other works have been acknowledged in the references.

Place: Chennai

Date: 28-10-2026

_____

Signature of the student - Simon V

Certificate

This is to certify that the project report entitled "AI-Based Synthetic Telecom Customer Data Generation and Churn Prediction Using GANs and Deep Learning" is a bona fide record of the work carried out by **Simon V** [Register / Roll Number - N/A] under my supervision at [Institution Name - Entri Elevate], in partial fulfilment of the requirements for [Degree / Programme - Data Science with Generative AI] during the academic year [2026–2027].

This template carries no approval. Signatures, names, designations and dates below must be completed by the respective persons.

_____

Signature - Project Supervisor: Sree Varshaa S, Data Science Mentor

_____

Signature - Head of Department: N/A

_____

Signature - External Examiner: name and institution to be completed
Date of viva voce examination: N/A

Acknowledgement

I would like to thank [Supervisor name - to be completed] for guidance during this project, and [Institution / Department / Training programme - to be completed] for providing the learning environment and resources. I also acknowledge the open-source community behind the Python data science libraries used in this work, and Maven Analytics for making the telecom customer churn dataset publicly available.

Add further personal acknowledgements, if any - N/A

Abstract

Customer churn is a major source of revenue loss for telecommunications operators. This project builds an end-to-end pipeline that predicts customer churn from account, service and billing attributes, and investigates whether synthetic customer data generated by CTGAN, TVAE and WGAN-GP can support the task. The Telecom Customer Churn dataset (7,043 customers, 38 columns) was cleaned to 6,589 labelled customers, of whom 28.4% churned. The data were split 60/20/20 into training, validation and test sets before any fitted transformation, and preprocessors and generators were fitted on training data only to prevent leakage.

Logistic Regression, XGBoost, CatBoost, a deep neural network, TabNet and an FT-Transformer were trained, with decision thresholds chosen on validation data. On the independent test set the FT-Transformer achieved accuracy 0.8604, churn-class recall 0.7941, F1-score 0.7635 and ROC-AUC 0.9322, narrowly ahead of CatBoost (accuracy 0.8536, F1 0.7484, ROC-AUC 0.9313). The differences among the best models are small, and the final model was selected using test-set F1, so the ranking is provisional. Adding CTGAN-generated rows to CatBoost training did not improve test performance, and the project therefore claims no benefit from synthetic data. SHAP analysis of the CatBoost model identified contract type, number of referrals, number of dependents, tenure and monthly charge as the main contributors to predictions. A FastAPI deployment is documented as a design and has not yet been executed or tested. Random Forest, cross-validation and SHAP for the FT-Transformer were not performed and are listed as future work.

Keywords: customer churn, CTGAN, TVAE, WGAN-GP, FT-Transformer, TabNet, CatBoost, SHAP, FastAPI.

Table of Contents

Declaration **3**
Certificate **4**
Acknowledgement **5**
Abstract **6**
List of Figures **11**
List of Tables **12**
List of Abbreviations **13**
Chapter 1: Introduction **14**
1.1 Chapter Introduction 14
1.2 Objectives 14
1.3 Problem Statement 14
1.4 Scope of the Work 14
1.5 Overall Methodology 15
1.6 Report Organisation 17
1.7 Chapter Summary 17
Chapter 2: Literature Review **18**
2.1 Chapter Introduction 18
2.2 Customer Churn Prediction as Supervised Learning 18
2.3 Class Imbalance 18
2.4 Generative Adversarial Networks for Tabular Data 18
2.5 Deep Learning for Tabular Data 18
2.6 Explainable AI 19
2.7 Chapter Summary 19
Chapter 3: Dataset Description **20**
3.1 Chapter Introduction 20
3.2 Objectives 20
3.3 Dataset Source and Size 20
3.4 Dataset Feature Description 20
3.5 Missing-Value Summary 21
3.6 Data Quality Summary 21
3.7 Implementation Steps 22
3.8 Chapter Summary 23
Chapter 4: Data Preprocessing **24**
4.1 Chapter Introduction 24
4.2 Objectives 24
4.3 Technical Background 24
4.4 Methodology 24
4.5 Train, Validation and Test Split 25
4.6 Implementation Steps 26
4.7 Chapter Summary 32

Chapter 5: Exploratory Data Analysis **33**
5.1 Chapter Introduction 33
5.2 Objectives 33
5.3 Methodology 33
5.4 Implementation Steps 33
5.5 Churn and Tenure 35
5.6 Churn and Contract Type 36
5.7 Churn and Monthly Charge 37
5.8 Correlation Analysis 38
5.9 Evaluation or Verification 39
5.10 Common Errors and Solutions 39
5.11 Chapter Summary 40
Chapter 6: Feature Engineering **41**
6.1 Chapter Introduction 41
6.2 Objectives 41
6.3 Feature Engineering Summary 41
6.4 Implementation Steps 41
6.5 Chapter Summary 43
Chapter 7: Synthetic Data Generation: CTGAN, TVAE and WGAN-GP **44**
7.1 Chapter Introduction 44
7.2 Objectives 44
7.3 Technical Background 44
7.4 Methodology 44
7.5 Implementation Steps 46
7.6 Synthetic Data Generator Comparison 56
7.7 Augmentation Experiment 58
7.8 Common Errors and Solutions 61
7.9 Chapter Summary 61
Chapter 8: Machine Learning Model Development **62**
8.1 Chapter Introduction 62
8.2 Objectives 62
8.3 Technical Background 62
8.4 Methodology 62
8.5 Implementation Steps 63
8.6 Evaluation or Verification 67
8.7 Chapter Summary 67
Chapter 9: Deep Neural Network Development **68**
9.1 Chapter Introduction 68
9.2 Objectives 68
9.3 Technical Background 68
9.4 Algorithm or Model Explanation 68
9.5 Implementation Steps 69
9.6 Evaluation or Verification 71

9.7 Chapter Summary 71
Chapter 10: TabNet Model Development 72
10.1 Chapter Introduction 72
10.2 Technical Background 72
10.3 Algorithm or Model Explanation 72
10.4 Implementation Steps 73
10.5 Evaluation or Verification 74
10.6 Chapter Summary 74
Chapter 11: FT-Transformer Model Development 75
11.1 Chapter Introduction 75
11.2 Technical Background 75
11.3 Methodology 76
11.4 Implementation Steps 76
11.5 Chapter Summary 86
Chapter 12: Model Evaluation and Comparison 87
12.1 Chapter Introduction 87
12.2 Objectives 87
12.3 Evaluation Protocol 87
12.4 Machine Learning and Deep Learning Performance Comparison 87
12.5 Model Hyperparameter Configuration 89
12.6 Discussion of Results 89
12.7 Model Selection Procedure: A Methodological Caveat 90
12.8 Threshold Inconsistency 90
12.9 Implementation Steps 91
12.10 Chapter Summary 95
Chapter 13: Explainable AI Using SHAP 96
13.1 Chapter Introduction 96
13.2 Objectives 96
13.3 Technical Background 96
13.4 Implementation Steps 97
13.5 SHAP Feature Importance Summary 100
13.6 Global Feature-Importance Plot 101
13.7 SHAP Summary (Beeswarm) Plot 102
13.8 Individual Customer Waterfall Plot 103
13.9 Business Implications 103
13.10 Limitations of the SHAP Analysis 104
13.11 Chapter Summary 104
Chapter 14: FastAPI Deployment 105
14.1 Chapter Introduction 105
14.2 Objectives 105
14.3 API Architecture 105
14.4 Project Folder Structure 105
14.5 Dependency Installation 106

14.6 Model and Preprocessor Loading 106
14.7 Pydantic Request Schema 107
14.8 Prediction and Health Endpoints 108
14.9 Example JSON Request 110
14.10 Example JSON Response 110
14.11 HTTP Error Responses 110
14.12 Swagger UI Instructions 111
14.13 API Testing Code 111
14.14 Security Considerations 112
14.15 Deployment Instructions 112
14.16 Common Errors and Solutions 113
14.17 Chapter Summary 113
Chapter 15: System Testing 114
15.1 Chapter Introduction 114
15.2 Software and Hardware Requirements 114
15.3 System Test Cases 114
15.4 Evaluation or Verification 115
15.5 Chapter Summary 115
Chapter 16: Expected Outcomes 116
16.1 Chapter Introduction 116
16.2 Expected Outcomes of Further Work 116
16.3 Chapter Summary 116
Chapter 17: Advantages and Limitations 117
17.1 Advantages 117
17.2 Limitations 117
17.3 Chapter Summary 117
Chapter 18: Future Enhancements 118
18.1 Chapter Introduction 118
18.2 Chapter Summary 118
Chapter 19: Conclusion 119
Chapter 20: References 120
Appendix A: Complete Project Source Code 122
A.1 Notebook code (Telecom.ipynb) 122
A.2 FastAPI module: app/model_def.py 160
Appendix B: requirements.txt 163
Appendix C: Project Folder Structure 164
Appendix D: API Request and Response Examples 165
D.1 Single prediction 165
D.2 Validation error (unknown category) 165
D.3 Missing field 165
D.4 curl example 165
Appendix E: Additional Evaluation Charts 166

Appendix F: Test Cases and Verification Results **172**
Appendix G: Instructions for Running the Project **173**

List of Figures

Figure 1.1: Overall system architecture 16
Figure 1.2: End-to-end project workflow 16
Figure 4.1: Data preprocessing pipeline 25
Figure 4.2: Train, validation and test data flow 26
Figure 5.1: Customer churn distribution in the training split 34
Figure 5.2: Churn rate by tenure in months (training split) 36
Figure 5.3: Churn rate by contract type (training split) 36
Figure 5.4: Monthly charge distribution by churn status 38
Figure 5.5: Churn rate by monthly charge range 38
Figure 5.6: Correlation heatmap of numeric features (training split) 39
Figure 7.1: GAN generator and discriminator (critic) architecture used for WGAN-GP 45
Figure 7.2: CTGAN, TVAE and WGAN-GP comparison workflow 46
Figure 7.3: Monthly charge: real training data against the three synthetic datasets 57
Figure 8.1: Machine learning and deep learning model training pipeline 63
Figure 9.1: DNN architecture 68
Figure 9.2: DNN training and validation accuracy 71
Figure 9.3: DNN training and validation loss 71
Figure 10.1: TabNet architecture 72
Figure 11.1: FT-Transformer architecture 75
Figure 11.2: FT-Transformer learning curves 82
Figure 11.3: FT-Transformer test confusion matrix (threshold 0.57) 84
Figure 11.4: FT-Transformer test ROC curve 84
Figure 12.1: Model performance comparison (test set) 88
Figure 12.2: ROC curves of all models (test set) 89
Figure 13.1: SHAP explainability workflow 96
Figure 13.2: Global SHAP feature importance (mean absolute SHAP value) 101
Figure 13.3: SHAP summary plot for the CatBoost model 102
Figure 13.4: SHAP waterfall plot for the first customer in the SHAP sample 103
Figure 14.1: FastAPI prediction request and response flow 105
Figure E.1: Customer age distribution 166
Figure E.2: Customer churn percentage (pie chart) 166
Figure E.3: ECDF of tenure by churn status 166
Figure E.4: Swarm plot of monthly charges by churn status (500-customer sample) 167
Figure E.5: Total charges distribution by churn status 167
Figure E.6: Tenure against total charges by churn status 167
Figure E.7: Pair plot of selected numeric features 168
Figure E.8: Parallel coordinates of selected numeric features 168
Figure E.9: Clustered correlation heatmap 169
Figure E.10: Age distribution by churn status 169
Figure E.11: Tenure distribution by churn status 170
Figure E.12: Monthly charge distribution by churn status 170
Figure E.13: Total charges distribution by churn status 170

Figure E.14: Total revenue distribution by churn status 171

List of Tables

Table 3.1: Dataset feature description (38 original columns) 20
Table 3.2: Missing-value summary (counts per column) 21
Table 3.3: Data quality summary 21
Table 4.1: Dataset split sizes 26
Table 6.1: Feature engineering summary 41
Table 6.2: Columns removed to prevent leakage or noise 41
Table 7.1: Synthetic data generator comparison 56
Table 9.1: DNN configuration 69
Table 10.1: TabNet configuration 72
Table 11.1: FT-Transformer configuration 75
Table 12.1: Evaluation protocol 87
Table 12.2: Machine learning and deep learning performance on the independent test set 87
Table 12.3: Validation F1 and threshold used for each model 88
Table 12.4: Model hyperparameter configuration 89
Table 13.1: SHAP feature importance summary (CatBoost, 100 validation customers; mean absolute SHAP value in log-odds) 100
Table 14.1: API endpoint specification 110
Table 14.2: HTTP error responses 110
Table 15.1: Software and hardware requirements 114
Table 15.2: System test cases 114
Table 16.1: Planned outcomes and status 116
Table 18.1: Project limitations and future enhancements 118
Table E.1: Verification checklist for the final deliverables 172

List of Abbreviations

Abbreviation	Meaning
AUC	Area Under the Curve
API	Application Programming Interface
BCE	Binary Cross-Entropy
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTGAN	Conditional Tabular Generative Adversarial Network
DNN	Deep Neural Network
EDA	Exploratory Data Analysis
ECDF	Empirical Cumulative Distribution Function
FT-Transformer	Feature Tokenizer Transformer
GAN	Generative Adversarial Network
GP	Gradient Penalty
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KDE	Kernel Density Estimate
MLP	Multilayer Perceptron
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
TVAE	Tabular Variational Autoencoder
VAE	Variational Autoencoder
WGAN-GP	Wasserstein GAN with Gradient Penalty
XGBoost	Extreme Gradient Boosting

Chapter 1: Introduction

1.1 Chapter Introduction

Customer churn, the loss of subscribers to a competitor or to voluntary disconnection, is one of the most costly problems faced by telecommunications operators. Acquiring a new subscriber is generally more expensive than retaining an existing one, so an operator that can identify the customers most likely to leave, and understand why, can direct retention offers where they matter most. This project builds an end-to-end machine learning and deep learning workflow for that task using a public telecom customer dataset.

The project has two linked goals. The first is **predictive**: classify each customer as likely to churn or likely to stay. The second is **generative**: investigate whether synthetic customer records produced by tabular generative models (CTGAN, TVAE and WGAN-GP) are realistic enough to be useful, and whether adding them to the training data helps a classifier. Alongside these goals the project applies explainable AI (SHAP) to make model behaviour interpretable, and documents a proposed FastAPI deployment.

1.2 Objectives

- Prepare a clean, leakage-free modelling dataset from the telecom customer churn data.
- Explore the data to understand how churn relates to tenure, contract type, charges and services.
- Train three synthetic data generators (CTGAN, TVAE, WGAN-GP) on the training split only and inspect their output.
- Train and compare Logistic Regression, XGBoost, CatBoost, a Deep Neural Network (DNN), TabNet and an FT-Transformer for churn prediction.
- Select a decision threshold on validation data and report independent test-set performance.
- Explain model behaviour with SHAP and document a deployment design with FastAPI.

1.3 Problem Statement

Given the demographic, account, service and billing attributes of a telecom customer, predict whether the customer will churn (positive class, label 1) or stay (label 0). The data are moderately imbalanced: after removing customers whose outcome is not yet known, 28.4% of

the customers in the training split are churners. A useful model must therefore achieve good recall on the churn class without producing an unmanageable number of false alarms.

1.4 Scope of the Work

The work reported here is the content of the project notebook (`Telecom.ipynb`), executed on a Windows machine using CPU only. All numerical results in this report are copied from that notebook's recorded outputs. Items that the specification requires but that the notebook does not contain are clearly marked as **not performed** or **pending**; no value has been estimated or invented to fill them. Specifically:

- A Random Forest model was not trained in the notebook, so its row is marked "Not performed".
- The FastAPI service is documented as a design with example code, but it has not been executed or tested in this work; its test cases are marked "To be completed".
- SHAP values were computed for the CatBoost model, not for the FT-Transformer that was designated as the selected model (see Chapter 13).
- Cross-validation, SMOTE and a formal synthetic-data fidelity evaluation were not performed.

1.5 Overall Methodology

The overall system architecture is shown in Figure 1.1 and the end-to-end workflow in Figure 1.2. The data are cleaned, split into training, validation and test sets, preprocessed with a pipeline fitted on the training data only, and then used to train the generators and classifiers. Validation data are used for threshold tuning; the test set is used for the final evaluation.

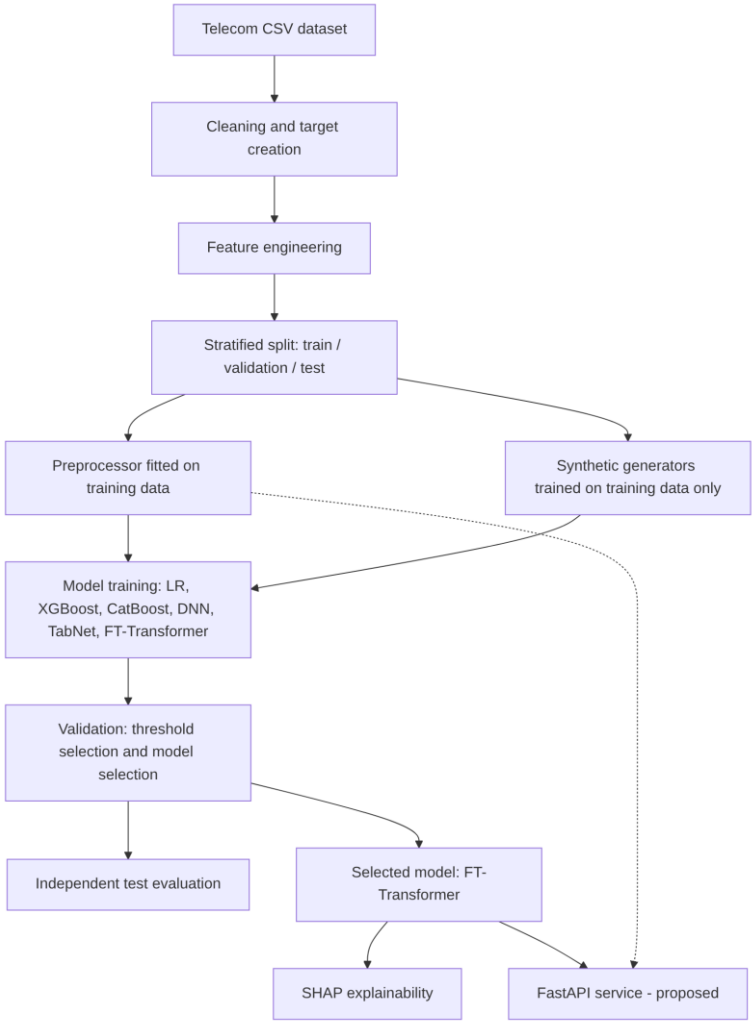


Figure 1.1: Overall system architecture

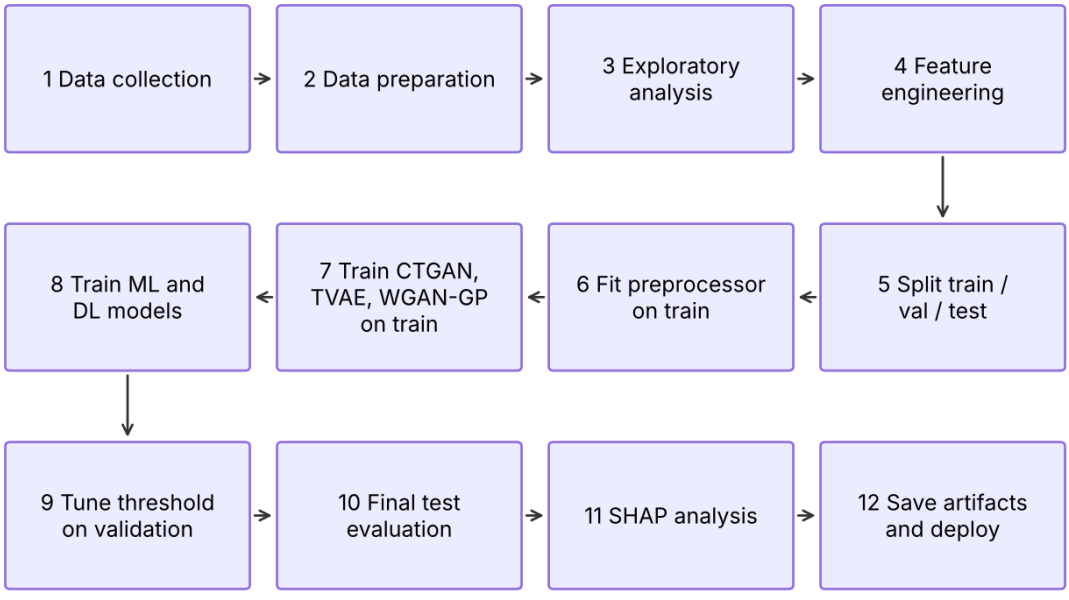


Figure 1.2: End-to-end project workflow

1.6 Report Organisation

Chapters 2 to 6 cover the literature, dataset, preprocessing, exploratory analysis and feature engineering. Chapter 7 covers synthetic data generation. Chapters 8 to 11 cover model development. Chapters 12 and 13 present the evaluation and explainability results. Chapters 14 and 15 cover deployment design and testing. Chapters 16 to 19 discuss expected outcomes, advantages and limitations, future work and conclusions, and Chapter 20 lists the references. Appendices A to G contain the source code and supporting material.

1.7 Chapter Summary

This chapter introduced the churn problem, the project objectives, the scope of what has actually been executed, and the overall methodology.

Chapter 2: Literature Review

2.1 Chapter Introduction

This chapter summarises the established methods on which the project is built. It is limited to foundational, verifiable sources listed in Chapter 20; it does not make claims about the results of specific published churn studies.

2.2 Customer Churn Prediction as Supervised Learning

Churn prediction is normally cast as binary classification on tabular customer records. Classical choices include logistic regression and tree ensembles. Gradient boosted decision trees remain strong baselines for tabular data; XGBoost [10] and CatBoost [9] are widely used implementations. CatBoost in particular handles categorical features natively using ordered target statistics, which suits telecom data in which most attributes are categorical (contract type, payment method, service add-ons).

2.3 Class Imbalance

Churners are a minority, so plain accuracy can be misleading. Common remedies are class weighting, resampling such as SMOTE [12], threshold adjustment and, more recently, synthetic data generation. This project uses class weighting in the neural networks, a validation-tuned decision threshold for all models, and a CTGAN augmentation experiment. SMOTE was named in the project plan but was not run.

2.4 Generative Adversarial Networks for Tabular Data

Generative Adversarial Networks (GANs) [4] train a generator and a discriminator in competition. Tabular data pose particular challenges: columns mix continuous and discrete types, continuous columns can be multimodal, and discrete columns can be imbalanced. CTGAN [1] addresses these with mode-specific normalisation of continuous columns and a conditional generator with training-by-sampling. TVAE, introduced in the same work, uses a variational autoencoder [13] with a tabular-specific preprocessing and loss. The Wasserstein GAN [2] replaces the original GAN objective with an earth-mover distance estimate, and WGAN-GP [3] enforces the required Lipschitz constraint with a gradient penalty rather than weight clipping, improving training stability.

2.5 Deep Learning for Tabular Data

Deep models for tabular data are an active research area. TabNet [5] uses sequential attention to select features at each decision step, giving built-in interpretability. The FT-Transformer [6] tokenises every numerical and categorical feature into an embedding and processes the token sequence with Transformer layers [7]; the authors of [6] report it as a strong deep baseline, while also noting that gradient boosted trees remain highly competitive on many tabular problems. The results of this project are consistent with that general picture: differences between the best deep model and the best boosted-tree model are small (Chapter 12).

2.6 Explainable AI

SHAP [8] assigns each feature a Shapley-value-based contribution to an individual prediction. For tree ensembles an efficient exact algorithm exists, and CatBoost exposes it directly through its ShapValues output. SHAP values describe how the model uses its inputs; they do not establish causal effects in the real world.

2.7 Chapter Summary

The project combines established methods: boosted trees, tabular GANs, tabular deep learning and SHAP explanations. The novelty lies in applying and comparing them on a single leakage-controlled pipeline rather than in a new algorithm.

Chapter 3: Dataset Description

3.1 Chapter Introduction

This chapter describes the dataset, its structure and the initial data-quality checks. The data are the Telecom Customer Churn dataset published on the Maven Analytics Data Playground, which is based on IBM sample data [14].

3.2 Objectives

- Describe the dataset source, size and variables.
- Quantify missing values, duplicates and the target distribution.

3.3 Dataset Source and Size

The CSV file `telecom_customer_churn.csv` contains **7,043 rows and 38 columns** (6 integer, 9 floating point and 23 text columns). Each row describes one customer. The column `Customer Status` takes the values Stayed (4,720), Churned (1,869) and Joined (454). Customers with status Joined are new customers whose outcome is not yet known; they are removed before modelling (Chapter 4), leaving 6,589 labelled customers.

3.4 Dataset Feature Description

Table 3.1: Dataset feature description (38 original columns)

Column(s)	Type	Category	Treatment
Customer ID	text	Unique identifier	Dropped
Gender	text	Demographic	Used
Age	int	Demographic	Used
Married	text	Demographic	Used
Number of Dependents	int	Demographic	Used
City, Zip Code, Latitude, Longitude	text / int / float	Location	Dropped
Number of Referrals	int	Account	Used
Tenure in Months	int	Account	Used
Offer	text	Account (promotion)	Used
Phone Service, Multiple Lines	text	Service	Used
Avg Monthly Long Distance Charges	float	Billing	Used
Internet Service, Internet Type	text	Service	Used
Avg Monthly GB Download	float	Usage	Used
Online Security, Online Backup, Device Protection Plan, Premium Tech Support	text	Service add-ons	Used

Column(s)	Type	Category	Treatment
Streaming TV, Streaming Movies, Streaming Music, Unlimited Data	text	Service add-ons	Used
Contract	text	Account	Used
Paperless Billing, Payment Method	text	Billing	Used
Monthly Charge	float	Billing	Used
Total Charges, Total Refunds, Total Long Distance Charges, Total Revenue	float	Billing	Used
Total Extra Data Charges	int	Billing	Used
Customer Status	text	Outcome (Stayed / Churned / Joined)	Used to build target, then dropped
Churn Category, Churn Reason	text	Post-churn explanation (only filled for churners)	Dropped (leakage)

After the eight columns listed as dropped are removed, 30 original columns remain, and two engineered features are added (Chapter 6), giving the 32 model inputs used throughout this report.

3.5 Missing-Value Summary

Table 3.2: Missing-value summary (counts per column)

Column(s)	All 7,043 rows	After removing Joined (6,589 rows)	Treatment
Offer	3,877	3,598	Filled with "No Offer"
Avg Monthly Long Distance Charges, Multiple Lines	682	644	No phone service: 0 / "No"
Internet Type	1,526	1,344	No internet service: "No Internet"
Avg Monthly GB Download	1,526	1,344	No internet service: 0
Online Security, Online Backup, Device Protection Plan, Premium Tech Support, Streaming TV, Streaming Movies, Streaming Music, Unlimited Data (each)	1,526	1,344	No internet service: "No"
Churn Category, Churn Reason	5,174	4,720	Dropped
All other columns	0	0	None needed

The missing values are *structural*: they occur when a customer does not subscribe to the relevant service (for example, a customer without internet service has no internet type). They are therefore filled with explicit "not applicable" values rather than statistical estimates.

3.6 Data Quality Summary

Table 3.3: Data quality summary

Check	Result
Rows and columns (raw)	7,043 x 38
Duplicate rows	0
Customer Status counts	Stayed 4,720; Churned 1,869; Joined 454
Rows retained (Stayed + Churned)	6,589
Overall churn rate in retained data	28.4% (1,869 / 6,589)
Missing values after treatment (modelling columns)	0

3.7 Implementation Steps

Step Name: Load and inspect the dataset

Purpose: Confirm that the file loads correctly and that its structure matches expectations before any transformation.

Input: telecom_customer_churn.csv in the working directory.

Python Code: Complete code for this stage (taken from the project notebook).

```
import pandas as pd

telecom = pd.read_csv("telecom_customer_churn.csv")
print("Dataset loaded successfully.")

print(telecom.shape)

print(telecom.columns.tolist())

print(telecom.isnull().sum())

print("Duplicate rows:", telecom.duplicated().sum())

print(telecom["Customer Status"].value_counts(dropna=False))
```

Code Explanation:

- `pd.read_csv` loads the CSV into the DataFrame `telecom`.
- `shape` and `columns` confirm 7,043 rows and 38 columns.
- `isnull().sum()` counts missing values per column; `duplicated().sum()` counts repeated rows.
- `value_counts(dropna=False)` on `Customer Status` shows all outcome values including any missing ones.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Dataset loaded successfully.
(7043, 38)
Duplicate rows: 0
Customer Status
Stayed     4720
Churned    1869
Joined      454
```

Verification:

- Row and column counts equal 7,043 and 38.
- The Customer Status counts sum to 7,043.

Troubleshooting:

- FileNotFoundError: place the CSV in the working directory or give the full path.
- UnicodeDecodeError: pass `encoding="utf-8"` or `encoding="latin-1"` to `read_csv`.

3.8 Chapter Summary

The dataset contains 7,043 customers with 38 attributes, no duplicates and structural missing values that correspond to services a customer does not use. After excluding customers with unknown outcome, 6,589 labelled customers remain, 28.4% of whom churned.

Chapter 4: Data Preprocessing

4.1 Chapter Introduction

This chapter covers cleaning, target creation, treatment of missing values, the train/validation/test split and the preprocessing pipeline. Particular attention is paid to preventing data leakage.

4.2 Objectives

- Create a clean binary target.
- Remove columns that leak the outcome or carry no generalisable signal.
- Split the data before any fitted transformation or synthetic data generation.
- Fit all scalers and encoders on the training split only.

4.3 Technical Background

Data leakage occurs when information that would not be available at prediction time, or information from the evaluation data, influences model training. Two forms matter here. *Target leakage* arises from columns such as `Churn Reason`, which exists only for customers who have already churned. *Train-test contamination* arises when statistics such as means, medians or category lists are computed on the whole dataset before splitting. Both are avoided in this project.

4.4 Methodology

1. Strip whitespace from column names and text values; convert blank strings to missing values.
2. Keep only customers with status Stayed or Churned and create the binary target `Churn` (Stayed = 0, Churned = 1).
3. Drop identifiers, location fields and churn explanation fields.
4. Fill structural missing values with explicit "not applicable" values.
5. Add the engineered features (Chapter 6) and split the data with stratification.
6. Fit the preprocessing `ColumnTransformer` on the training split and apply it unchanged to the validation and test splits.

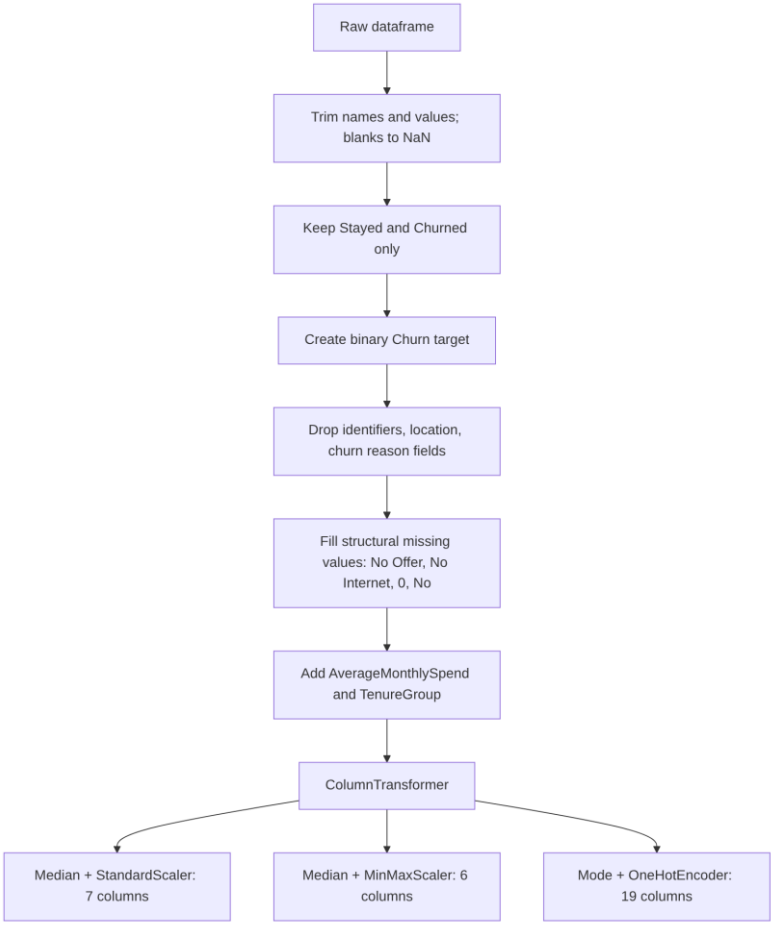


Figure 4.1: Data preprocessing pipeline

4.5 Train, Validation and Test Split

The data are split in two stages with `random_state=42` and stratification on the target: 20% is first reserved as the independent test set, and the remaining 80% is divided 75/25 into training and validation sets. This yields 3,953 training, 1,318 validation and 1,318 test customers (a 60/20/20 split). Figure 4.2 shows the data flow. The test set is touched only for the final evaluation. The notebook asserts that no row index appears in more than one split.

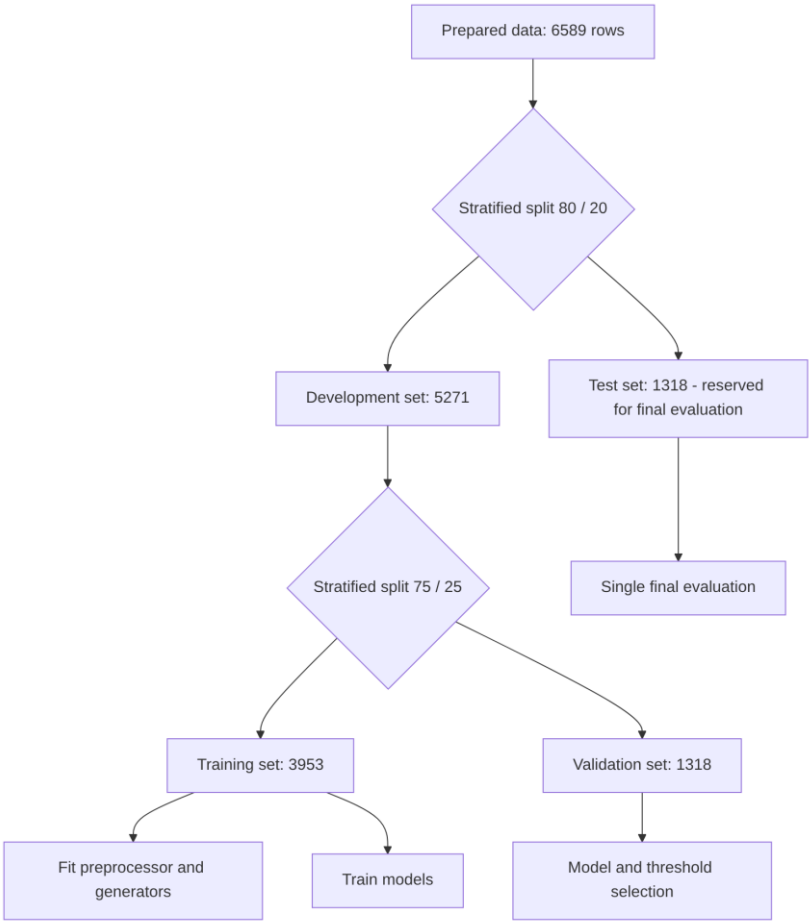


Figure 4.2: Train, validation and test data flow

Table 4.1: Dataset split sizes

Split	Rows	Purpose
Training	3,953	Fit preprocessor, generators and models
Validation	1,318	Early stopping, threshold selection
Test	1,318 (374 churners, 944 stayers)	Final independent evaluation

4.6 Implementation Steps

Step Name: Clean text, create the target and treat missing values

Purpose: Produce a consistent modelling table with a binary target and no missing values.

Input: DataFrame `telecom` from Chapter 3.

Python Code: Complete code for this stage (taken from the project notebook).

```
import numpy as np

telecom.columns = telecom.columns.str.strip()

for col in telecom.select_dtypes(
 include=["object", "string"]
 ).columns:
```

```
 telecom[col] = telecom[col].map(
  lambda value: value.strip()
  if
  isinstance(value, str)
  else value
  )

telecom = telecom.replace(r"^\s*$", np.nan, regex=True)
print("Dataset shape:", telecom.shape)
print(telecom.head())

telecom["Customer Status"] = (telecom["Customer
Status"].astype("string").str.strip().str.title())

telecom = telecom[telecom["Customer Status"].isin(["Stayed", "Churned"])].copy()

telecom["Churn"] = (telecom["Customer Status"].map({"Stayed": 0, "Churned":
1}).astype(int))

drop_columns = [
    "Customer ID", "City", "Zip Code", "Latitude", "Longitude",
    "Customer Status", "Churn Category", "Churn Reason",
]
telecom = telecom.drop(
    columns=drop_columns,
    errors="ignore"
).copy()

telecom["Offer"] = telecom["Offer"].fillna("No Offer")

telecom["Avg Monthly Long Distance Charges"] = (
    telecom["Avg Monthly Long Distance Charges"]
    .fillna(0)
)

telecom["Multiple Lines"] = (
    telecom["Multiple Lines"]
    .fillna("No")
)

telecom["Internet Type"] = (
    telecom["Internet Type"]
    .fillna("No Internet")
)

telecom["Avg Monthly GB Download"] = (
    telecom["Avg Monthly GB Download"]
    .fillna(0)
)

internet_service_cols = [
    "Online Security",
    "Online Backup",
    "Device Protection Plan",
    "Premium Tech Support",
    "Streaming TV",
    "Streaming Movies",
    "Streaming Music",
    "Unlimited Data"
]

for col in internet_service_cols:
    telecom[col] = telecom[col].fillna("No")
```

Code Explanation:

- Whitespace stripping and the regex replacement convert empty strings to `NaN` so that they are treated as missing.
- Customers with status Joined are removed because their outcome is unknown.
- `drop_columns` removes identifiers, location and churn-explanation fields.
- The `fillna` calls implement the structural fills described in Chapter 3.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Prepared dataset shape: (6589, 39)

Target distribution:
Churn
0    4720
1    1869
```

Verification:

- Prepared shape is (6589, 39) before columns are dropped.
- Re-running `telecom.isnull().sum()` on the modelling columns returns zeros (confirmed in the notebook).

Troubleshooting:

- KeyError on a column name: check whitespace stripping was run first.
- Unexpected NaN after filling: confirm the column list in `internet_service_cols` matches the dataset.

Step Name: Define features and target, split the data
Purpose: Reserve the test set and create training and validation sets before any fitted step.

Input: Modelling DataFrame with engineered features (Chapter 6).

Python Code: Complete code for this stage (taken from the project notebook).

```
leakage_drop = [
    "Customer ID",
    "Customer Status",
    "Churn Category",
    "Churn Reason",
    "Churn Score",
    "CLTV",
    "City",
    "Zip Code",
    "Latitude",
    "Longitude"
```

```
]

X = telecom.drop(
    columns=leakage_drop + ["Churn"],
    errors="ignore"
)

y = telecom["Churn"]

print(X.shape)
print(y.shape)

from sklearn.model_selection import train_test_split

X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.20,
random_state=42, stratify=y)

X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
random_state=42, stratify=y_dev)

assert set(X_train.index).isdisjoint(
    X_val.index
)

assert set(X_train.index).isdisjoint(
    X_test.index
)

assert set(X_val.index).isdisjoint(
    X_test.index
)

print("No overlapping rows between splits.")
```

Code Explanation:

- `stratify` keeps the churn proportion the same in every split.
- `random_state=42` makes the split reproducible.
- The assertions confirm the index sets of the three splits are disjoint.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Training: (3953, 32) (3953,)
Validation: (1318, 32) (1318,)
Testing: (1318, 32) (1318,)
No overlapping rows between splits.
```

Verification:

- Split sizes match Table 4.1.
- All three assertions pass.

Troubleshooting:

- ValueError from `stratify`: a class has fewer than two members; check the target.

Step Name: Build and fit the preprocessing pipeline

Purpose: Scale numerical features and encode categorical features consistently for the models that need a numeric matrix.

Input: X_train, X_val, X_test.

Python Code: Complete code for this stage (taken from the project notebook).

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    OneHotEncoder
)
import joblib
from pathlib import Path

numeric_features = X_train.select_dtypes(include=["number"]).columns.tolist()

categorical_features = [
 col for col in X_train.columns
 if col not in numeric_features
]

minmax_features = [
    "Age",
    "Tenure in Months",
    "Monthly Charge",
    "Total Charges",
    "Total Revenue",
    "AverageMonthlySpend"
]

standard_features = [
 col for col in numeric_features
 if col not in minmax_features
]

standard_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")),
("scaler", StandardScaler())])

minmax_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")),
("scaler", MinMaxScaler())])

categorical_pipeline = Pipeline([("imputer",
SimpleImputer(strategy="most_frequent")), ("encoder",
OneHotEncoder(handle_unknown="ignore"))])

preprocessor = ColumnTransformer([
    (
        "standard",
        standard_pipeline,
        standard_features
    ),
```

```
    (
        "minmax",
        minmax_pipeline,
        minmax_features
    ),
    (
        "categorical",
        categorical_pipeline,
        categorical_features
    )
])

X_train_ready = preprocessor.fit_transform(X_train)

X_val_ready = preprocessor.transform(X_val)
```

Code Explanation:

- Seven numeric columns use median imputation and `StandardScaler`; six bounded or monetary columns use `MinMaxScaler`; 19 categorical columns use most-frequent imputation and one-hot encoding with `handle_unknown="ignore"`.
- `fit_transform` is called on the training split only; `transform` is used for validation and test.

Expected Output: Representative output from the executed notebook is shown below.

Result (from the executed notebook):

```
Prepared training shape and validation shape: 62 columns each (confirmed by the
final evaluation step: Validation (1318, 62), Test (1318, 62)).
```

Verification:

- Training, validation and test matrices all have 62 columns.
- The fitted `preprocessor` is later saved as `models/telecom_preprocessor.pkl`.

Troubleshooting:

- A category appears only in validation/test: `handle_unknown="ignore"` encodes it as all zeros rather than failing.
- Different column order at prediction time: always pass a DataFrame with the same columns as `X_train`.

4.7 Chapter Summary

The pipeline produces a leakage-controlled 60/20/20 split and a preprocessor fitted only on training data. CatBoost and the FT-Transformer use their own native handling of categorical features (Chapters 8 and 11) and apply the same split.

Chapter 5: Exploratory Data Analysis

5.1 Chapter Introduction

Exploratory analysis was performed on the **training split only** (3,953 customers), so that no information from the validation or test sets influences modelling decisions.

5.2 Objectives

- Describe the class balance.
- Examine how churn varies with tenure, contract type and monthly charge.
- Inspect correlations among numeric variables.

5.3 Methodology

Distributions and group-wise churn rates were plotted with Matplotlib and Seaborn. The training DataFrame `eda_df` is the training features plus the target column.

5.4 Implementation Steps

Step Name: Prepare the EDA dataset and plot the churn distribution
Purpose: Establish the baseline churn rate in the training data.

Input: X_train, y_train.

Python Code: Complete code for this stage (taken from the project notebook).

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

eda_df = X_train.copy()

eda_df["Churn"] = y_train.to_numpy()

churn_counts = (eda_df["Churn"].value_counts().reindex([0, 1], fill_value=0))
churn_percentages = (churn_counts / churn_counts.sum() * 100)
print("Customer counts:")
print(churn_counts.rename(index={
 0: "Stayed",
 1: "Churned"
}))
print("\nCustomer percentages:")
print(churn_percentages.round(2))
plt.figure(figsize=(7, 5))
ax = sns.countplot(
 data=eda_df,
 x="Churn",
 order=[0, 1]
)
ax.set_xticks([0, 1])
```

```
ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
plt.title("Telecom Customer Churn Distribution")
plt.xlabel("Customer Status")
plt.ylabel("Number of Customers")
for container in ax.containers:
 ax.bar_label(container)
plt.tight_layout()
plt.show()
```

Code Explanation:

- `eda_df` copies `X_train` and adds `Churn`.
- `value_counts` and percentage calculation give the class proportions.
- A count plot with bar labels visualises them.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Customer counts:
Stayed     2832
Churned    1121

Customer percentages:
0    71.64
1    28.36
```

Verification:

- 2,832 + 1,121 = 3,953 training rows.

Troubleshooting:

- Plot not showing: use `%matplotlib inline` in Jupyter or call `plt.show()`.

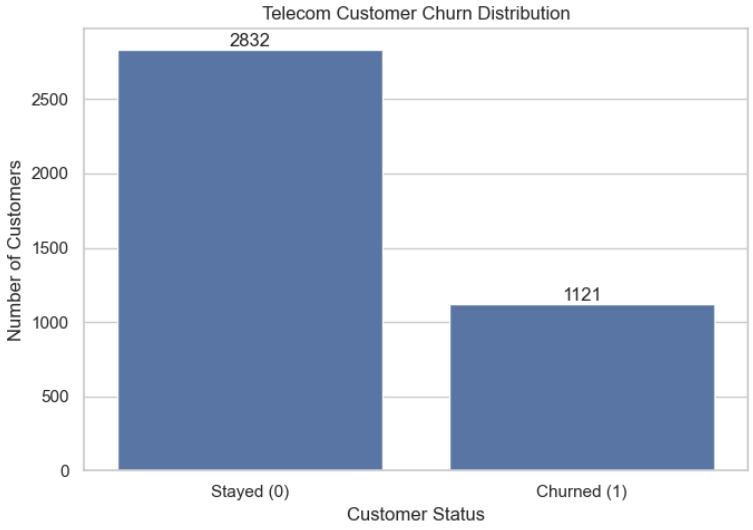


Figure 5.1: Customer churn distribution in the training split

Figure 5.1 shows that 71.6% of training customers stayed and 28.4% churned: moderate imbalance. The class weights computed later (Chapter 8) are 0.698 for stayers and 1.763 for churners.

5.5 Churn and Tenure

Step Name: Churn rate by tenure
Purpose: Check whether newer customers churn more often.

Input: eda_df.

Python Code: Complete code for this stage (taken from the project notebook).

```
tenure_churn = (eda_df.groupby("Tenure in Months")["Churn"].mean().reset_index())
tenure_churn["Churn Rate"] = (tenure_churn["Churn"] * 100)
plt.figure(figsize=(10, 5))
plt.plot(
    tenure_churn["Tenure in Months"],
    tenure_churn["Churn Rate"],
    marker="o"
)
plt.title("Churn Rate by Tenure")
plt.xlabel("Tenure in Months")
plt.ylabel("Churn Rate (%)")
plt.show()
```

Code Explanation:

- Groups by month of tenure and averages the 0/1 target, which gives the churn proportion for each month.

Expected Output: The resulting plot is shown in the next figure.

Verification:

- Churn rates lie between 0 and 100%.

Troubleshooting:

- Noisy curve at long tenures: individual months contain few customers; consider grouping into bands (done in Chapter 6).

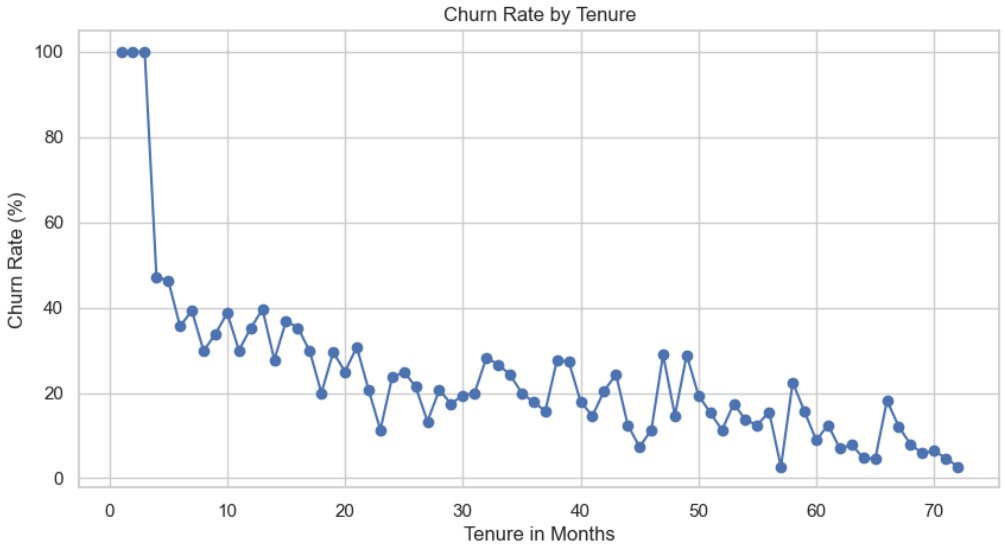


Figure 5.2: Churn rate by tenure in months (training split)

In Figure 5.2 churn is highest for the very newest customers (100% for tenure of 1 to 3 months in this sample) and declines steadily as tenure grows, to below 10% for many months beyond about 60. The curve is noisy because each month contains a limited number of customers. This pattern motivated the tenure-group feature.

5.6 Churn and Contract Type

```
contract_churn =
(eda_df.groupby("Contract")["Churn"].mean().sort_values(ascending=False)* 100)
plt.figure(figsize=(9, 5))
sns.barplot(x=contract_churn.index, y=contract_churn.values)
plt.title("Churn Rate by Contract")
plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation=20)
plt.show()
```

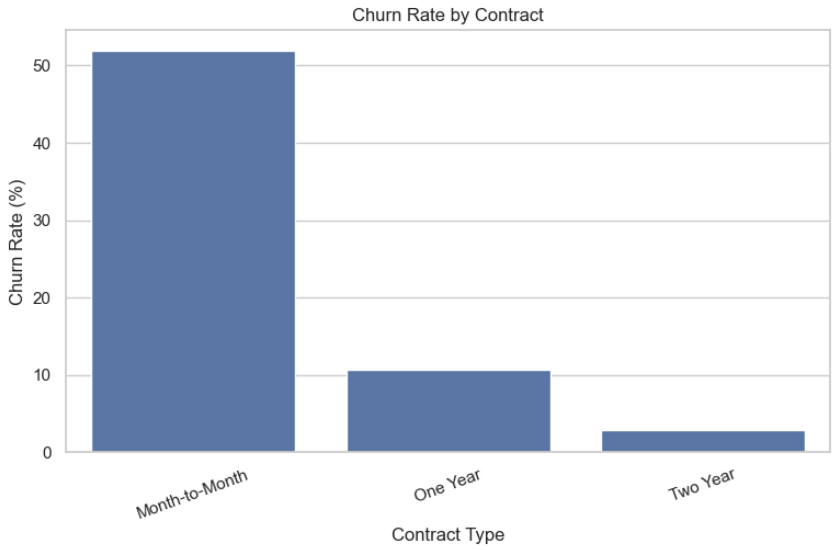


Figure 5.3: Churn rate by contract type (training split)

Figure 5.3 shows a very large difference: roughly half of month-to-month customers churned (about 52%), compared with roughly 11% for one-year and 3% for two-year contracts (values read from the plot). Contract type is the single strongest relationship in the data and also dominates the SHAP ranking (Chapter 13).

5.7 Churn and Monthly Charge

```
plt.figure(figsize=(8, 5))
sns.kdeplot(
    data=eda_df,
    x="Monthly Charge",
    hue="Churn",
    fill=True,
    common_norm=False,
    alpha=0.4
)
plt.title("Monthly Charge Distribution by Churn")
plt.show()

charge_bins = [
    0,
    30,
    50,
    70,
    90,
    120,
    np.inf
]

charge_labels = [
    "0-30",
    "31-50",
    "51-70",
    "71-90",
    "91-120",
    "120+"
]

temp = eda_df.copy()
temp["Charge Bucket"] = pd.cut(temp["Monthly Charge"], bins=charge_bins,
labels=charge_labels, include_lowest=True)
charge_churn = (
    temp
    .groupby(
        "Charge Bucket",
        observed=False
    )["Churn"]
    .mean()
    * 100
)
plt.figure(figsize=(10, 5))
sns.barplot(x=charge_churn.index, y=charge_churn.values)
plt.title("Churn Rate by Monthly Charge")
plt.xlabel("Monthly Charge Range")
plt.ylabel("Churn Rate (%)")
plt.show()
```

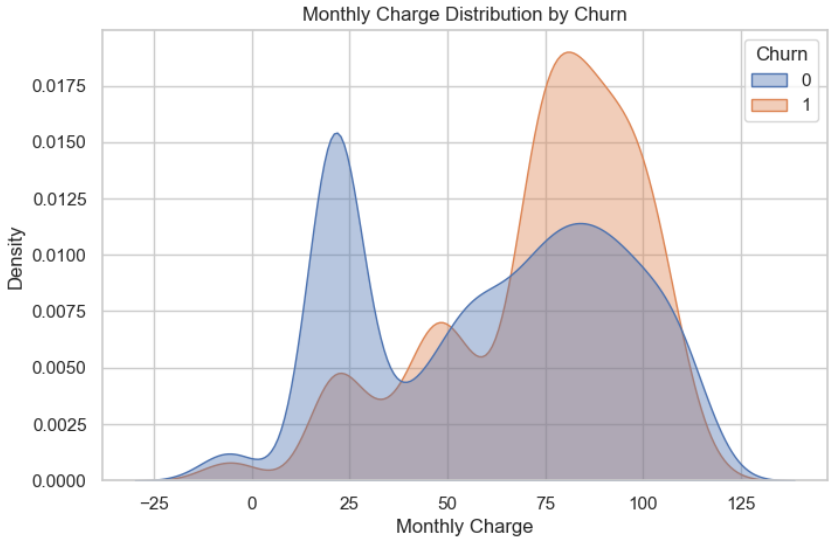


Figure 5.4: Monthly charge distribution by churn status

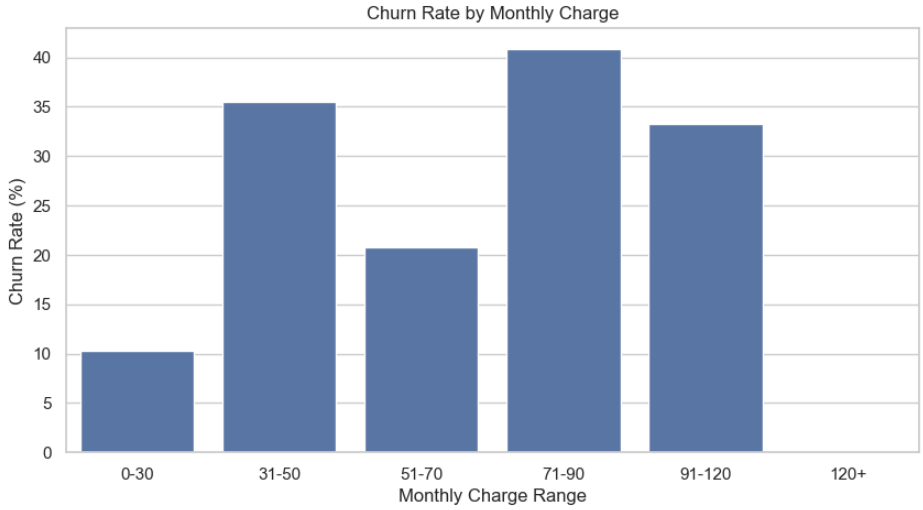


Figure 5.5: Churn rate by monthly charge range

Figure 5.5 shows that the lowest charge band (0 to 30) has the lowest churn rate (about 10%), while the 71 to 90 band has the highest (about 41%). The relationship is not monotonic. The 120+ bucket shows no churn in the plot, which most likely reflects an empty or very small bucket in this sample rather than a reliable finding.

5.8 Correlation Analysis

```
numeric_eda = eda_df.select_dtypes(include=["int64", "float64"])
corr = numeric_eda.corr()
plt.figure(figsize=(14, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)
plt.title("Correlation Heatmap")
plt.show()
```

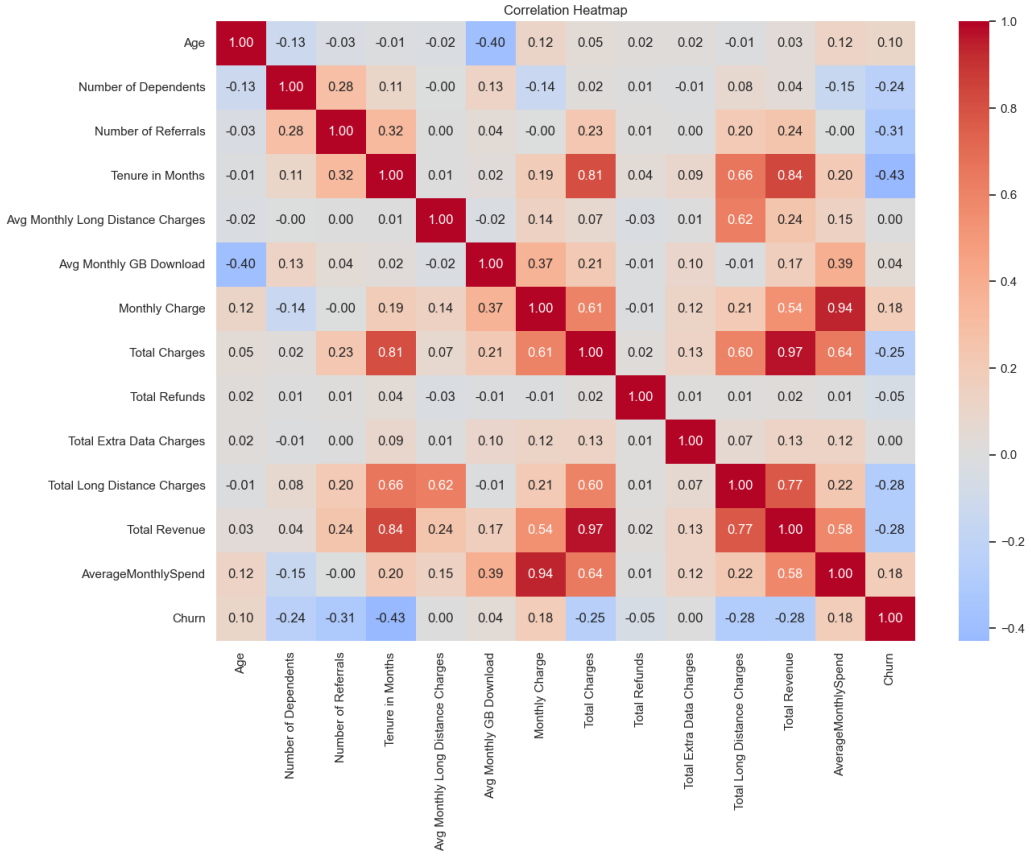


Figure 5.6: Correlation heatmap of numeric features (training split)

As expected, cumulative billing fields (Total Charges, Total Revenue, Total Long Distance Charges) are strongly correlated with tenure and with each other. Tree-based and attention-based models tolerate this redundancy; for Logistic Regression it can make individual coefficients harder to interpret.

5.9 Evaluation or Verification

- All EDA used `X_train` only; the validation and test splits were not examined.
- Percentages quoted from plots are visual readings and are labelled as such.

5.10 Common Errors and Solutions

- `ValueError` in `seaborn.barplot` after categorical binning: pass `observed=False` to `groupby`.
- Swarm plots on thousands of points are slow: the notebook samples 500 rows.

5.11 Chapter Summary

The EDA shows moderate class imbalance, strong dependence of churn on contract type and tenure, a non-monotonic dependence on monthly charge, and strong correlation among cumulative billing fields. Additional plots (age histogram, ECDF, swarm, pair plot, parallel coordinates, box plots) were produced in the notebook and are available in Appendix E.

Chapter 6: Feature Engineering

6.1 Chapter Introduction

Two features were engineered from existing columns. Both are computed row by row from the customer's own attributes, so no information from other rows (and therefore no cross-split information) is used.

6.2 Objectives

- Create a spend-rate feature and a categorical tenure band.
- Confirm that no leakage columns remain in the feature matrix.

6.3 Feature Engineering Summary

Table 6.1: Feature engineering summary

Feature	Definition	Type	Rationale
AverageMonthlySpend	Total Charges / Tenure in Months (tenure of 0 treated as missing, then median-imputed)	Numeric	Normalises cumulative spend by customer age
TenureGroup	Bins of tenure: 0-6, 7-12, 13-24, 25-48, 49+ months	Categorical	Captures the non-linear tenure effect seen in EDA

Table 6.2: Columns removed to prevent leakage or noise

Column	Reason
Customer ID	Identifier; no generalisable information
City, Zip Code, Latitude, Longitude	High-cardinality location fields
Customer Status	Directly defines the target
Churn Category, Churn Reason	Only populated for churners (target leakage)
Churn Score, CLTV	Listed for removal in code; not present in this version of the dataset (`errors="ignore"`)

6.4 Implementation Steps

Step Name: Create engineered features and remove leakage columns
Purpose: Add the two derived features and build the final feature matrix X and target y.

Input: Cleaned DataFrame `telecom`.

Python Code: Complete code for this stage (taken from the project notebook).

```
telecom["AverageMonthlySpend"] = (
    telecom["Total Charges"]
    /
    telecom["Tenure in Months"].replace(
```

```
        0, np.nan
    )
)

telecom["TenureGroup"] = pd.cut(
    telecom["Tenure in Months"],

    bins=[
        -np.inf,
        6,
        12,
        24,
        48,
        np.inf
    ],

    labels=[
        "0-6 months",
        "7-12 months",
        "13-24 months",
        "25-48 months",
        "49+ months"
    ]
).astype(object)

leakage_drop = [
    "Customer ID",
    "Customer Status",
    "Churn Category",
    "Churn Reason",
    "Churn Score",
    "CLTV",
    "City",
    "Zip Code",
    "Latitude",
    "Longitude"
]

X = telecom.drop(
    columns=leakage_drop + ["Churn"],
    errors="ignore"
)

y = telecom["Churn"]

print(X.shape)
print(y.shape)
```

Code Explanation:

- `replace(0, np.nan)` avoids division by zero; the imputer in the pipeline handles any resulting missing value.
- `pd.cut` with the listed bins and labels creates `TenureGroup`; `.astype(object)` makes it an ordinary categorical column.
- `errors="ignore"` allows the drop list to contain columns absent from this dataset version.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
(6589, 32)
(6589,)
```

Verification:

- X has 32 columns: 30 original and 2 engineered.
- `"Churn" not in X.columns` (also asserted for the GAN training data in Chapter 7).

Troubleshooting:

- `TenureGroup` has NaN values: check that tenure has no missing entries.
- Mismatch of columns at prediction time: the API must recreate both engineered features before preprocessing (Chapter 14).

6.5 Chapter Summary

Two engineered features were added and all leakage columns were excluded, producing a 32-column feature matrix for modelling.

Chapter 7: Synthetic Data Generation: CTGAN, TVAE and WGAN-GP

7.1 Chapter Introduction

This chapter describes the three synthetic data generators trained in the project, how their output was checked, and a single augmentation experiment in which CTGAN data were added to the training set of a CatBoost classifier.

7.2 Objectives

- Train CTGAN, TVAE and WGAN-GP on the **training split only**.
- Generate as many synthetic rows as there are real training rows (3,953).
- Inspect the synthetic data for validity and class balance.
- Test, with evidence, whether synthetic data improve a classifier.

7.3 Technical Background

CTGAN [1] is a conditional GAN for tabular data. It normalises each continuous column with a variational Gaussian mixture, represents discrete columns as one-hot vectors, and samples training batches conditioned on discrete values so that rare categories are learned. **TVAE** [1] is a variational autoencoder adapted to the same representation; it learns a latent distribution and decodes samples into records. **WGAN-GP** [2][3] trains a generator against a *critic* that estimates the Wasserstein distance; a gradient penalty (weight lambda = 10) keeps the critic's gradient norm near one. In this project CTGAN and TVAE come from the `ctgan` Python package, while WGAN-GP is implemented directly in TensorFlow/Keras. Figure 7.1 shows the WGAN-GP generator and critic used, and Figure 7.2 the comparison workflow.

7.4 Methodology

1. Build `gan_train`: the training features (CatBoost-style cleaned) plus the `Churn` column. All 19 categorical columns and `Churn` are declared as discrete.
2. Train each generator on `gan_train` only; sample 3,953 synthetic rows.
3. Check shape, missing values and churn ratio, and compare the monthly-charge distribution with the real data.
4. Select CTGAN for a controlled augmentation experiment: add 1,976 synthetic rows (half the training size) to the real training set and train CatBoost.

5. Evaluate on the real validation and test sets only.

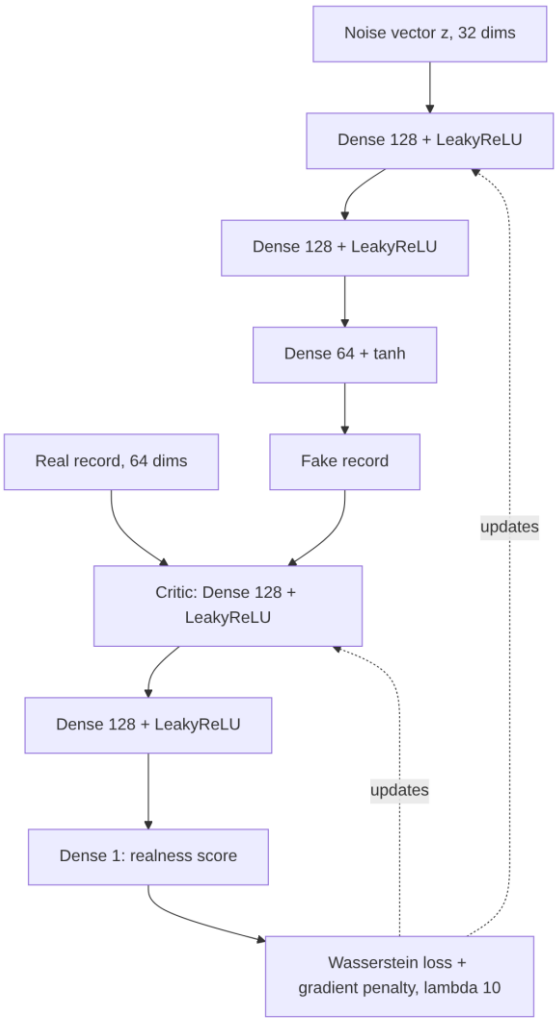


Figure 7.1: GAN generator and discriminator (critic) architecture used for WGAN-GP

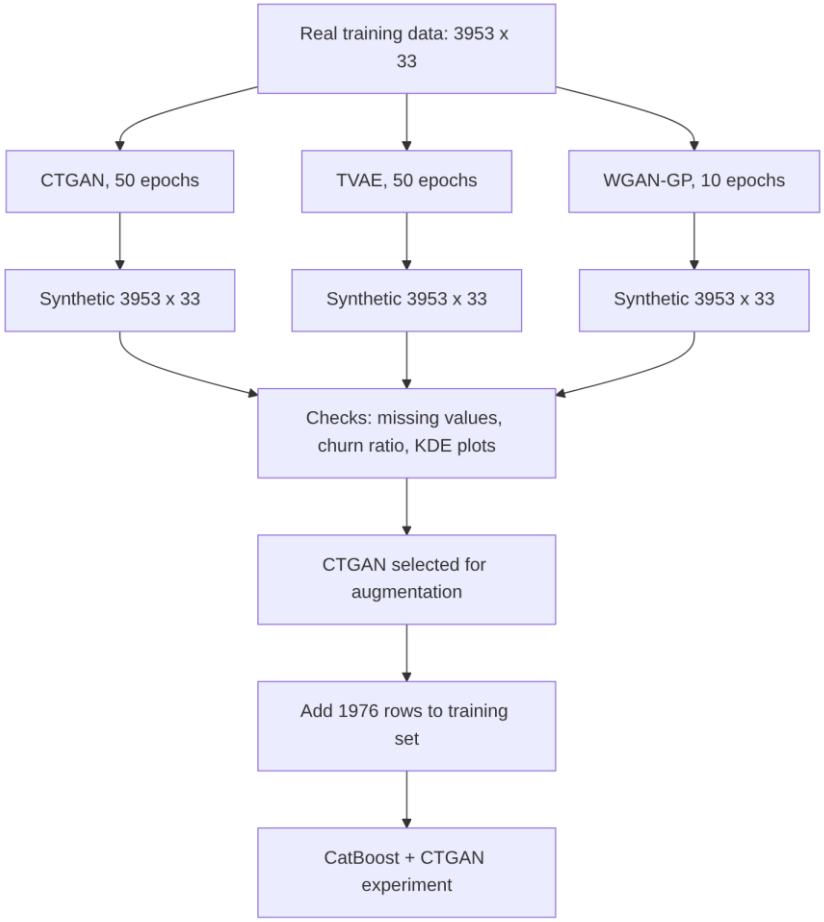


Figure 7.2: CTGAN, TVAE and WGAN-GP comparison workflow

7.5 Implementation Steps

7.5.1 Preparing the training-only GAN data

Step Name: Create the GAN training dataset

Purpose: Give the generators a complete, missing-value-free table containing only training-split rows.

Input: X_train, y_train, categorical_features, numeric_features.

Python Code: Complete code for this stage (taken from the project notebook).

```
catboost_numeric_medians = (
    X_train[numeric_features]
    .median(numeric_only=True)
    .to_dict()
)

catboost_categorical_fill = {}

for col in categorical_features:

    mode = (
        X_train[col]
        .dropna()
```

```
        .astype(str)
        .mode()
    )

    catboost_categorical_fill[col] = (
        str(mode.iloc[0])
        if len(mode)
        else "Unknown"
    )

def prepare_native_catboost(frame):

    out = frame.reindex(
        columns=X_train.columns
    ).copy()

    for col in numeric_features:

        out[col] = (
            pd.to_numeric(
                out[col],
                errors="coerce"
            )
            .fillna(catboost_numeric_medians[col])
        )

    for col in categorical_features:

        out[col] = out[col].map(
            lambda value: str(value).strip()
            if pd.notna(value)
            else catboost_categorical_fill[col]
        )

    return out

gan_train = prepare_native_catboost(X_train).copy()

gan_train["Churn"] = (y_train.to_numpy(dtype=int))

gan_discrete_columns = (categorical_features + ["Churn"])

assert gan_train.isna().sum().sum() == 0

assert "Churn" not in X_train.columns

assert set(
    gan_train["Churn"].unique()
) == {0, 1}

print("GAN training data is ready.")
```

Code Explanation:

- Training medians and modes (not whole-dataset statistics) are used to fill any gaps; they are stored in dictionaries.
- `gan_discrete_columns` lists the categorical features plus `Churn`.

- The assertions check there are no missing values, that `Churn` is binary and that it is not in `X_train`.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
GAN training shape: (3953, 33)
GAN training data is ready.
```

Verification:

- Shape is (3953, 33): 32 features + target.
- All assertions pass.

Troubleshooting:

- CTGAN raises an error for NaN values: fill them before fitting.
- Discrete columns containing numbers stored as text: cast to `str`.

7.5.2 CTGAN

Step Name: Train CTGAN and sample synthetic data
Purpose: Generate synthetic customers with a conditional tabular GAN.

Input: gan_train, gan_discrete_columns.

Python Code: Complete code for this stage (taken from the project notebook).

```
from ctgan import CTGAN

ctgan_model = CTGAN(epochs=50, batch_size=500, verbose=True, enable_gpu=False)

ctgan_model.fit(gan_train, discrete_columns=gan_discrete_columns)

synthetic_ctgan = ctgan_model.sample(len(gan_train))

from pathlib import Path
import joblib

GAN_DIR = Path("gan_models")
GAN_DIR.mkdir(exist_ok=True)

synthetic_ctgan.to_csv(
    GAN_DIR / "synthetic_ctgan.csv",
    index=False
)

joblib.dump(
    ctgan_model,
    GAN_DIR / "ctgan_model.pkl"
)
```

Code Explanation:

- `epochs=50`, `batch_size=500`, CPU only.
- `sample(len(gan_train))` returns 3,953 synthetic rows.
- The trained model and CSV are saved to the `gan_models` folder.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Gen. (-01.00) | Discrim. (-00.12): 100%|██████████| 50/50 [00:53<00:00,  1.08s/it]
(3953, 33)
```

Verification:

- Synthetic shape equals real training shape.
- Generated categorical values are all valid category labels.

Troubleshooting:

- Slow training on CPU: reduce epochs for testing or use a GPU.
- Version mismatch when unpickling: use the same `ctgan` version as for training.

7.5.3 TVAE

Step Name: Train TVAE and sample synthetic data
Purpose: Generate synthetic customers with a tabular variational autoencoder for comparison.

Input: gan_train, gan_discrete_columns.

Python Code: Complete code for this stage (taken from the project notebook).

```
from ctgan import TVAE

tvae_model = TVAE(epochs=50, batch_size=500, verbose=True, enable_gpu=False)

tvae_model.fit(gan_train, discrete_columns=gan_discrete_columns)

synthetic_tvae = tvae_model.sample(len(gan_train))

synthetic_tvae.to_csv(
    GAN_DIR / "synthetic_tvae.csv",
    index=False
)

joblib.dump(
    tvae_model,
    GAN_DIR / "tvae_model.pkl"
)
```

Code Explanation:

- Same epochs and batch size as CTGAN so that the comparison is fair.
- TVAE optimises a single reconstruction-plus-KL loss.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Loss: +23.32: 100%|██████████| 50/50 [00:14<00:00,  3.33it/s]
(3953, 33)
```

Verification:

- Shape (3953, 33).

Troubleshooting:

- Loss does not decrease: increase epochs or reduce batch size.

7.5.4 WGAN-GP

Step Name: Build, train and sample the WGAN-GP

Purpose: Implement a Wasserstein GAN with gradient penalty to generate mixed numeric and categorical data.

Input: gan_train.

Python Code: Complete code for this stage (taken from the project notebook).

```
import tensorflow as tf
from tensorflow import keras # type: ignore

tf.random.set_seed(42)

wgan_source = gan_train.copy()

wgan_categories = gan_discrete_columns

wgan_numeric = [
    col for col in wgan_source.columns
    if col not in wgan_categories
]

wgan_scaler = MinMaxScaler(
    feature_range=(-1, 1)
)

numeric_array = wgan_scaler.fit_transform(
    wgan_source[wgan_numeric]
)

wgan_encoder = OneHotEncoder(
    handle_unknown="ignore",
    sparse_output=False
)
```

```
categorical_array = wgan_encoder.fit_transform(
    wgan_source[wgan_categories].astype(str)
)

categorical_array = categorical_array * 2 - 1

wgan_real = np.concatenate(
    [numeric_array, categorical_array],
    axis=1
).astype("float32")

print("WGAN Training Shape:", wgan_real.shape)

noise_dim = 32
data_dim = wgan_real.shape[1]

generator = keras.Sequential([
    keras.layers.Input(shape=(noise_dim,)),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(
        data_dim,
        activation="tanh"
    )
])

critic = keras.Sequential([
    keras.layers.Input(shape=(data_dim,)),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(1)
])

g_optimizer = keras.optimizers.Adam(
    learning_rate=0.0001,
    beta_1=0.0,
    beta_2=0.9
)

d_optimizer = keras.optimizers.Adam(
    learning_rate=0.0001,
    beta_1=0.0,
    beta_2=0.9
)
```

```
LAMBDA_GP = 10.0
CRITIC_STEPS = 3

@tf.function
def wgan_step(real_batch):

    batch_size = tf.shape(real_batch)[0]

    # Train Critic
    for _ in range(CRITIC_STEPS):

        with tf.GradientTape() as disc_tape:

            noise = tf.random.normal(
                (batch_size, noise_dim)
            )

            fake_batch = tf.stop_gradient(
                generator(noise, training=True)
            )

            real_score = critic(
                real_batch,
                training=True
            )

            fake_score = critic(
                fake_batch,
                training=True
            )

            # Interpolate real and fake
            alpha = tf.random.uniform(
                (batch_size, 1)
            )

            interpolated = (
                real_batch
                + alpha * (fake_batch - real_batch)
            )

            # Gradient penalty
            with tf.GradientTape() as gp_tape:
                gp_tape.watch(interpolated)

                interpolated_score = critic(
                    interpolated,
                    training=True
                )

            gradients = gp_tape.gradient(
                interpolated_score,
                interpolated
            )

            gradient_norm = tf.sqrt(
                tf.reduce_sum(
                    tf.square(gradients),
                    axis=1
                ) + 1e-12
            )
```

```
            gp = tf.reduce_mean(
                tf.square(gradient_norm - 1.0)
            )

            critic_loss = (
                tf.reduce_mean(fake_score)
                - tf.reduce_mean(real_score)
                + LAMBDA_GP * gp
            )

        critic_gradients = disc_tape.gradient(
            critic_loss,
            critic.trainable_variables
        )

        d_optimizer.apply_gradients(
            zip(
                critic_gradients,
                critic.trainable_variables
            )
        )

    # Train Generator
    with tf.GradientTape() as gen_tape:

        noise = tf.random.normal(
            (batch_size, noise_dim)
        )

        generated = generator(
            noise,
            training=True
        )

        generator_loss = -tf.reduce_mean(
            critic(generated, training=True)
        )

    generator_gradients = gen_tape.gradient(
        generator_loss,
        generator.trainable_variables
    )

    g_optimizer.apply_gradients(
        zip(
            generator_gradients,
            generator.trainable_variables
        )
    )

    return critic_loss, generator_loss

dataset = (
    tf.data.Dataset
    .from_tensor_slices(wgan_real)
    .shuffle(len(wgan_real), seed=42)
    .batch(64)
)

WGAN_EPOCHS = 10

for epoch in range(WGAN_EPOCHS):
```

```
    for batch in dataset:
        critic_loss, generator_loss = wgan_step(
            batch
        )

    print(
        f"Epoch {epoch + 1}/{WGAN_EPOCHS}",
        f"Critic Loss: {float(critic_loss):.4f}",
        f"Generator Loss: {float(generator_loss):.4f}"
    )

generated_array = generator(
    tf.random.normal(
        (len(gan_train), noise_dim)
    ),
    training=False
).numpy()

n_numeric = len(wgan_numeric)

decoded_numeric = (
    wgan_scaler.inverse_transform(
        generated_array[:, :n_numeric]
    )
)

synthetic_wgan = pd.DataFrame(
    decoded_numeric,
    columns=wgan_numeric
)

offset = n_numeric

for i, col in enumerate(wgan_categories):

    categories = wgan_encoder.categories_[i]

    width = len(categories)

    block = generated_array[
        :,
        offset:offset + width
    ]

    synthetic_wgan[col] = categories[
        np.argmax(block, axis=1)
    ]

    offset += width

synthetic_wgan["Churn"] = pd.to_numeric(
    synthetic_wgan["Churn"],
    errors="coerce"
)

print(synthetic_wgan.shape)

display(synthetic_wgan.head())

synthetic_wgan.to_csv(
    GAN_DIR / "synthetic_wgan.csv",
```

```
    index=False
)

generator.save(
    GAN_DIR / "wgan_generator.keras"
)

joblib.dump(
    wgan_scaler,
    GAN_DIR / "wgan_numeric_scaler.pkl"
)

joblib.dump(
    wgan_encoder,
    GAN_DIR / "wgan_categorical_encoder.pkl"
)

joblib.dump(
    {
        "numeric_cols": wgan_numeric,
        "categorical_cols": wgan_categories,
        "noise_dim": noise_dim
    },
    GAN_DIR / "wgan_metadata.pkl"
)
```

Code Explanation:

- Numeric columns are scaled to [-1, 1]; categorical columns are one-hot encoded and rescaled to {-1, +1}, giving a 64-dimensional record.
- The generator maps a 32-dimensional noise vector through two 128-unit LeakyReLU layers to a tanh output. The critic mirrors it with a single linear output.
- Each step updates the critic three times (`CRITIC_STEPS = 3`) with the gradient penalty (`LAMBDA_GP = 10`), then updates the generator once.
- Generated categorical blocks are decoded with `argmax`; numeric blocks are inverse-scaled.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
WGAN Training Shape: (3953, 64)
Epoch 1/10 Critic Loss: -3.3981 Generator Loss: -1.1166
Epoch 5/10 Critic Loss: -3.9993 Generator Loss: -2.3376
Epoch 10/10 Critic Loss: -2.7758 Generator Loss: -2.9450
(3953, 33)
```

Verification:

- Critic loss is negative and the generator loss changes smoothly, as expected for WGAN-GP.
- Output shape (3953, 33).

Troubleshooting:

- Exploding critic loss: lower the learning rate or increase `LAMBDA_GP`.
- `Churn` decoded as text: convert with `pd.to_numeric` (done in the notebook).

7.6 Synthetic Data Generator Comparison

Table 7.1: Synthetic data generator comparison

Property	Real training data	CTGAN	TVAE	WGAN-GP
Library	-	`ctgan`	`ctgan`	TensorFlow / Keras (custom)
Epochs / batch size	-	50 / 500	50 / 500	10 / 64
Training time (as logged)	-	53 s	14 s	Not logged
Rows generated	3,953	3,953	3,953	3,953
Missing values	0	0	0	0
Churn proportion	28.4%	28.8%	13.7%	30.3%
Invalid values seen in sample rows	-	Slightly negative refunds / extra-data charges; internally inconsistent charge totals	Negative tenure, negative charges and negative total charges	Fractional counts (e.g. dependents 0.24) and non-integer ages

The churn proportions in Table 7.1 come from the notebook. The comments in the last row are observations from the first rows printed in the notebook and are indicative only: a formal fidelity evaluation (for example column-wise distribution tests, correlation comparison, or a train-on-synthetic, test-on-real study) was **not** performed. TVAE visibly under-represents churners (13.7% against 28.4%), WGAN-GP was trained for only ten epochs and does not respect integer columns, and CTGAN produced the most plausible class balance. CTGAN was therefore selected for the augmentation experiment.

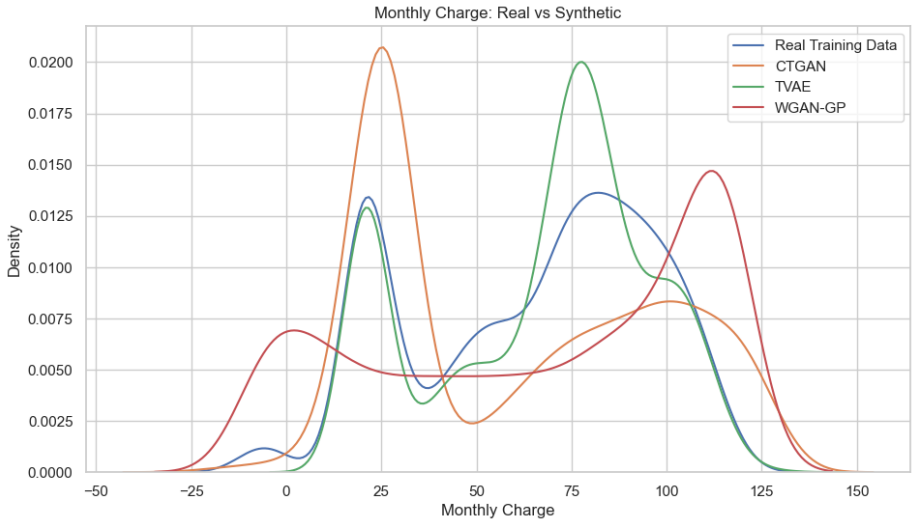


Figure 7.3: Monthly charge: real training data against the three synthetic datasets

Step Name: Check synthetic data and class balance

Purpose: Compare shape, missing values and churn ratio across the three generators.

Input: synthetic_ctgan, synthetic_tvae, synthetic_wgan.

Python Code: Complete code for this stage (taken from the project notebook).

```
synthetic_datasets = {
    "CTGAN": synthetic_ctgan,
    "TVAE": synthetic_tvae,
    "WGAN-GP": synthetic_wgan
}

for name, data in synthetic_datasets.items():

    print("\nModel:", name)

    print("Shape:", data.shape)

    print(
        "Missing Values:",
        data.isnull().sum().sum()
    )

    print(
        "Churn Distribution:",
        data["Churn"].value_counts(
            normalize=True
        )
    )
```

Code Explanation:

- Loops over a dictionary of the three datasets and prints shape, total missing values and normalised class proportions.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook):

```
Model: CTGAN     Shape: (3953, 33)  Missing: 0  Churn: 0 -> 0.7121, 1 -> 0.2879
Model: TVAE      Shape: (3953, 33)  Missing: 0  Churn: 0 -> 0.8629, 1 -> 0.1371
Model: WGAN-GP   Shape: (3953, 33)  Missing: 0  Churn: 0 -> 0.6974, 1 -> 0.3026
```

Verification:

- No missing values in any synthetic table.
- Churn proportion compared with the real 28.4%.

Troubleshooting:

- Constant columns in synthetic data indicate mode collapse; retrain with more epochs.

7.7 Augmentation Experiment

Step Name: Add CTGAN data to the training set and train CatBoost
Purpose: Test whether synthetic rows improve churn prediction on real validation and test data.

Input: Real training data, synthetic_ctgan, real validation data.

Python Code: Complete code for this stage (taken from the project notebook).

```
def repair_engineered_features(data):

    data = data.copy()

    data["AverageMonthlySpend"] = (
        data["Total Charges"]
        /
        data["Tenure in Months"].replace(
            0, np.nan
        )
    )

    data["TenureGroup"] = pd.cut(
        data["Tenure in Months"],
        bins=[-np.inf, 6, 12, 24, 48, np.inf],
        labels=[
            "0-6 months",
            "7-12 months",
            "13-24 months",
            "25-48 months",
            "49+ months"
        ]
    ).astype(str)

    return data

selected_synthetic = synthetic_datasets["CTGAN"]

n_synthetic = min(
```

```
    len(selected_synthetic),
    len(X_train) // 2
)

selected_synthetic = selected_synthetic.sample(
    n=n_synthetic,
    random_state=42
)

X_synth = selected_synthetic.drop(
    columns="Churn"
).reindex(columns=X_train.columns)

y_synth = (
    selected_synthetic["Churn"]
    .astype(int)
)

X_train_augmented = pd.concat(
    [
        X_train.reset_index(drop=True),
        X_synth.reset_index(drop=True)
    ],
    ignore_index=True
)

y_train_augmented = pd.concat(
    [
        y_train.reset_index(drop=True),
        y_synth.reset_index(drop=True)
    ],
    ignore_index=True
)

print(
    "Augmented Training Size:",
    X_train_augmented.shape
)

from catboost import CatBoostClassifier

X_train_aug_cb = prepare_native_catboost(
    X_train_augmented
)

catboost_gan = CatBoostClassifier(
    iterations=500,
    depth=5,
    learning_rate=0.03,
    loss_function="Logloss",
    eval_metric="AUC",
    random_seed=42,
    verbose=False
)

catboost_gan.fit(
    X_train_aug_cb,
    y_train_augmented,
    cat_features=categorical_features,
    eval_set=(X_val_cb, y_val),
    early_stopping_rounds=40
)

from sklearn.metrics import (
```

```
    accuracy_score,
    f1_score,
    roc_auc_score
)

val_proba = catboost_gan.predict_proba(
    X_val_cb
)[:, 1]

val_pred = (
    val_proba >= 0.5
).astype(int)

print(
    "Validation Accuracy:",
    accuracy_score(y_val, val_pred)
)

print(
    "Validation F1:",
    f1_score(y_val, val_pred)
)

print(
    "Validation ROC-AUC:",
    roc_auc_score(y_val, val_proba)
)
```

Code Explanation:

- `n_synthetic = min(len(synthetic), len(X_train)//2)` adds 1,976 rows, giving 5,929 training rows.
- Only the training set is augmented; validation and test stay real.
- The function `repair_engineered_features` is defined to recompute `AverageMonthlySpend` and `TenureGroup` for synthetic rows, but **it was not called** before concatenation in the notebook, so the engineered columns of the synthetic rows are the generators' raw output.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Augmented Training Size: (5929, 32)
Validation Accuracy: 0.8725341426403642
Validation F1: 0.7469879518072289
Validation ROC-AUC: 0.9279859285779026
```

Verification:

- Augmented size is 3,953 + 1,976 = 5,929.
- Compare with the real-data CatBoost on the same test set (Chapter 12).

Troubleshooting:

- Unknown category error in CatBoost: all categorical values are cast to `str`.

On the independent test set, CatBoost trained on real data scored accuracy 0.8536, F1 0.7484 and ROC-AUC 0.9313, while CatBoost trained with CTGAN augmentation scored accuracy 0.8520, F1 0.7464 and ROC-AUC 0.9259 (Table 12.2). The augmented model is slightly lower on every reported metric; differences of this size are small and within what random variation could produce, but there is **no evidence in this project that synthetic data improved performance**.

7.8 Common Errors and Solutions

- Generators trained on the full dataset leak test information; here they use `gan_train` only.
- Synthetic rows violating business rules (negative charges, inconsistent totals): clip or repair them, or apply rule-based filters, before use.
- Class ratio drift in synthetic data (TVAE): use conditional sampling or resample to the desired ratio.

7.9 Chapter Summary

Three generators were trained on training data only. CTGAN reproduced the class balance best; TVAE under-represented churners; WGAN-GP, trained briefly, did not respect column types. Adding CTGAN data to CatBoost training did not improve test performance.

Chapter 8: Machine Learning Model Development

8.1 Chapter Introduction

This chapter covers the classical and boosted-tree classifiers: Logistic Regression, XGBoost and CatBoost. A Random Forest was named in the project plan but was not trained in the notebook; its results are therefore reported as "Not performed" (Chapter 12).

8.2 Objectives

- Train baseline and boosted-tree models on the real training split.
- Use validation data for early stopping (CatBoost) and later for threshold selection.
- Produce churn probabilities for evaluation.

8.3 Technical Background

Logistic Regression models the log-odds of churn as a linear function of the preprocessed features and is a transparent baseline. **XGBoost** [10] builds an additive ensemble of shallow trees by gradient boosting. **CatBoost** [9] is also a boosted-tree method; it consumes categorical columns directly, avoiding one-hot encoding, and uses ordered boosting to reduce target leakage within training. Figure 8.1 shows how the models are trained in parallel from the same split.

8.4 Methodology

- Logistic Regression and XGBoost use the 62-column preprocessed matrix.
- CatBoost uses the cleaned DataFrame with native categorical features (`prepare_native_catboost`).
- The class weights computed by `compute_class_weight` (0.698 and 1.763) were calculated but **not passed** to these three models. Class imbalance is handled in this project by threshold tuning (Chapter 12), and the FT-Transformer uses a positive-class weight in its loss (Chapter 11).

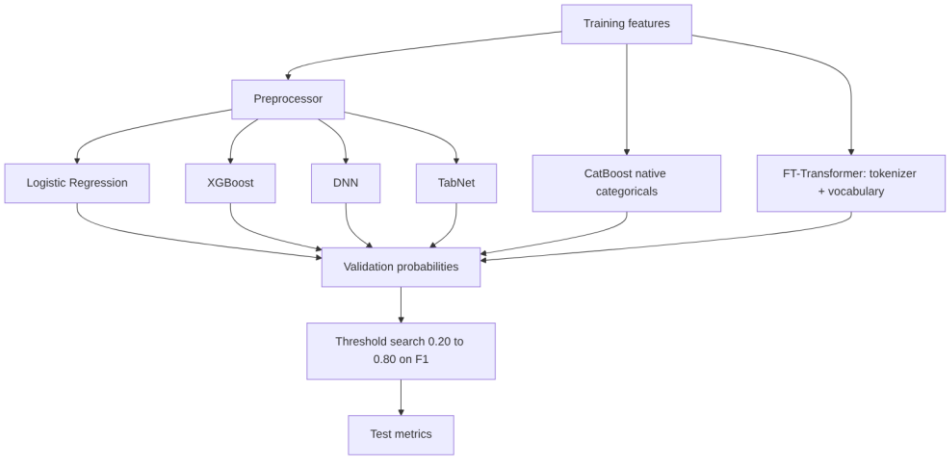


Figure 8.1: Machine learning and deep learning model training pipeline

8.5 Implementation Steps

Step Name: Prepare the model inputs and compute class weights

Purpose: Create consistently transformed matrices for every model.

Input: X_train, X_val, fitted preprocessor.

Python Code: Complete code for this stage (taken from the project notebook).

```
from sklearn.utils.class_weight import compute_class_weight

classes = np.unique(y_train)

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = dict(
    zip(classes, weights)
)



print(class_weights)

X_train_fit = X_train.copy()
y_train_fit = y_train.copy()

X_train_fit_ready = np.asarray(
    preprocessor.transform(X_train_fit),
    dtype=np.float32
)

X_train_fit_cb = prepare_native_catboost(
    X_train_fit
)

X_val_ready = np.asarray(
    preprocessor.transform(X_val),
    dtype=np.float32
)
```

```
X_val_cb = prepare_native_catboost(X_val)
```

Code Explanation:

- `X_train_fit_ready` and `X_val_ready` are float32 matrices from the fitted preprocessor.
- `X_train_fit_cb` and `X_val_cb` are CatBoost-ready DataFrames.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
{0: 0.6979166666666666, 1: 1.763157894736842}
```

Verification:

- Both matrices have 62 columns.

Troubleshooting:

- dtype object errors in XGBoost: ensure the matrix is numeric (`np.asarray(..., dtype=np.float32)`).

Step Name: Train Logistic Regression
Purpose: Provide an interpretable linear baseline.

Input: X_train_fit_ready, y_train_fit, X_val_ready.

Python Code: Complete code for this stage (taken from the project notebook).

```
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=2500, C=1.0, random_state=42)

lr_model.fit(X_train_fit_ready, y_train_fit)

lr_probabilities = lr_model.predict_proba(X_val_ready)[:, 1]
```

Code Explanation:

- `C=1.0` is the default L2 regularisation strength; `max_iter=2500` ensures convergence.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
LogisticRegression(max_iter=2500, random_state=42)
[0.54661182 0.02745266 0.21634611 ... 0.62722327 0.02666529 0.53117222]
```

Verification:

- Probabilities lie in [0, 1].

Troubleshooting:

- ConvergenceWarning: increase `max_iter` or scale the inputs.

Step Name: Train CatBoost with native categorical features
Purpose: Train a gradient boosted model that handles categorical columns directly.

Input: X_train_fit_cb, y_train_fit, X_val_cb, y_val.

Python Code: Complete code for this stage (taken from the project notebook).

```
from catboost import CatBoostClassifier

cat_model = CatBoostClassifier(
    iterations=500,
    depth=5,
    learning_rate=0.03,
    loss_function="Logloss",
    eval_metric="AUC",
    random_seed=42,
    verbose=False,
    allow_writing_files=False
)

cat_model.fit(
    X_train_fit_cb,
    y_train_fit,
    cat_features=categorical_features,
    eval_set=(X_val_cb, y_val),
    early_stopping_rounds=40
)

cat_probabilities = cat_model.predict_proba(X_val_cb)[:, 1]
```

Code Explanation:

- 500 iterations, depth 5, learning rate 0.03, log-loss objective, AUC evaluation metric.
- `eval_set` on the validation data with `early_stopping_rounds=40` selects the number of trees.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
CatBoostClassifier(allow_writing_files=False, depth=5, eval_metric='AUC',
iterations=500, learning_rate=0.03, loss_function='Logloss', random_seed=42,
verbose=False)
[0.49461814 0.05086766 0.16006505 ... 0.96770499 0.02433978 0.19871401]
```

Verification:

- `predict_proba` returns one probability per validation row.

Troubleshooting:

- CatBoostError about cat_features: they must be strings or integers; the helper casts them to `str`.

Step Name: Train XGBoost
Purpose: Train a second boosted-tree model on the preprocessed matrix.

Input: X_train_fit_ready, y_train_fit, X_val_ready.

Python Code: Complete code for this stage (taken from the project notebook).

```
from xgboost import XGBClassifier

xgb_model = XGBClassifier(
    n_estimators=400,
    max_depth=4,
    learning_rate=0.03,
    subsample=0.85,
    colsample_bytree=0.85,
    objective="binary:logistic",
    eval_metric="logloss",
    tree_method="hist",
    random_state=42
)

xgb_model.fit(X_train_fit_ready, y_train_fit)

xgb_probabilities = xgb_model.predict_proba(X_val_ready)[:, 1]
```

Code Explanation:

- 400 trees, depth 4, learning rate 0.03, 85% row and column subsampling, histogram tree method.
- No early stopping is used for XGBoost in the notebook.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
[0.5574988  0.01079114 0.2400102  ... 0.9949673  0.00398189 0.08120254]
```

Verification:

- Probabilities lie in [0, 1].

Troubleshooting:

- `early_stopping_rounds` placement differs between XGBoost versions; check the installed version.

8.6 Evaluation or Verification

Test-set results for these models are in Table 12.2: Logistic Regression F1 0.7366 / ROC-AUC 0.9158; XGBoost F1 0.7377 / ROC-AUC 0.9246; CatBoost F1 0.7484 / ROC-AUC 0.9313. CatBoost had the highest validation F1 (0.7879) of all models.

8.7 Chapter Summary

Three classical and boosted models were trained. CatBoost was the strongest of them and, on validation F1, the strongest model overall.

Chapter 9: Deep Neural Network Development

9.1 Chapter Introduction

This chapter describes the feed-forward deep neural network (DNN) trained on the preprocessed 62-feature matrix.

9.2 Objectives

- Define a regularised multilayer perceptron for tabular input.
- Train with early stopping on validation loss.

9.3 Technical Background

A multilayer perceptron applies stacked affine transformations with non-linear activations. Batch normalisation stabilises training; dropout randomly disables units during training to reduce overfitting. The output uses a sigmoid to produce a churn probability and training minimises binary cross-entropy.

9.4 Algorithm or Model Explanation

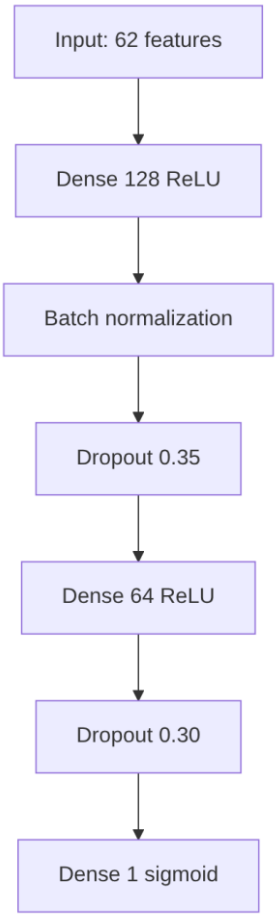


Figure 9.1: DNN architecture

Table 9.1: DNN configuration

Item	Value
Input	62 preprocessed features
Hidden layers	Dense 128 (ReLU) - BatchNormalization - Dropout 0.35 - Dense 64 (ReLU) - Dropout 0.30
Output	Dense 1, sigmoid
Parameters	16,897 total (16,641 trainable, 256 non-trainable)
Optimiser / loss	Adam, learning rate 0.001 / binary cross-entropy
Training	Up to 70 epochs, batch size 32, early stopping on validation loss (patience 8, restore best weights)

9.5 Implementation Steps

Step Name: Define, compile and train the DNN

Purpose: Train a deep baseline to compare with tree models and tabular-specific deep models.

Input: X_train_fit_ready, y_train_fit, X_val_ready, y_val.

Python Code: Complete code for this stage (taken from the project notebook).

```
import tensorflow as tf
from tensorflow import keras # type: ignore

dnn_model = keras.Sequential([
    keras.layers.Input(
        shape=(X_train_fit_ready.shape[1],)
    ),

    keras.layers.Dense(
        128, activation="relu"
    ),

    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.35),

    keras.layers.Dense(
        64, activation="relu"
    ),

    keras.layers.Dropout(0.30),

    keras.layers.Dense(
        1, activation="sigmoid"
    )
])

dnn_model.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        keras.metrics.AUC(name="auc")
    ]
)
dnn_model.summary()

early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True
)

dnn_history = dnn_model.fit(
    X_train_fit_ready,
    y_train_fit.to_numpy(),

    validation_data=(
        X_val_ready,
        y_val.to_numpy()
    ),

    epochs=70,
```

```
    batch_size=32,

    callbacks=[early_stopping]
)
```

Code Explanation:

- `validation_data` is used to monitor validation loss each epoch.
- `restore_best_weights=True` returns the best epoch's weights when training stops.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook; training stopped after epoch 21):

```
Total params: 16,897 (66.00 KB)
Trainable params: 16,641 (65.00 KB)
Non-trainable params: 256 (1.00 KB)
Epoch 1/70 ... accuracy: 0.7675 - auc: 0.8201 - val_accuracy: 0.8346 - val_auc:
0.8933
Epoch 21/70 ... accuracy: 0.8867 - auc: 0.9546 - val_accuracy: 0.8437 - val_auc:
0.9067
```

Verification:

- Validation loss stops improving after the early epochs, and training stops by early stopping.

Troubleshooting:

- Large train/validation gap: increase dropout, add weight decay or reduce width.
- Different results between runs: set seeds with `keras.utils.set_random_seed`.

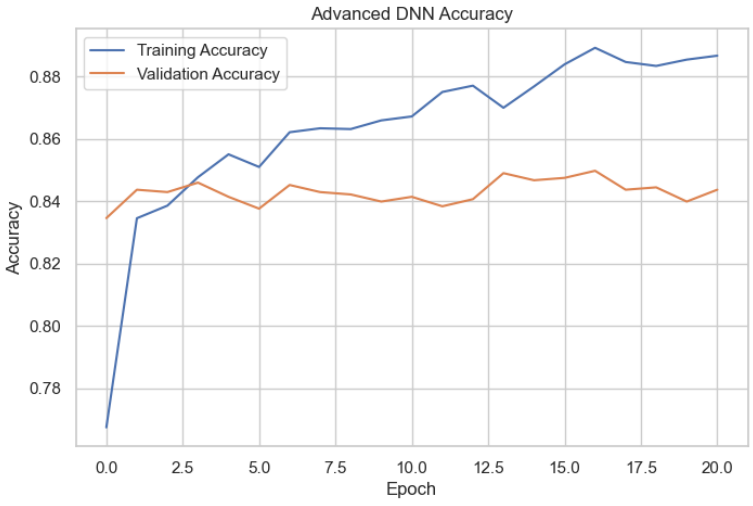


Figure 9.2: DNN training and validation accuracy

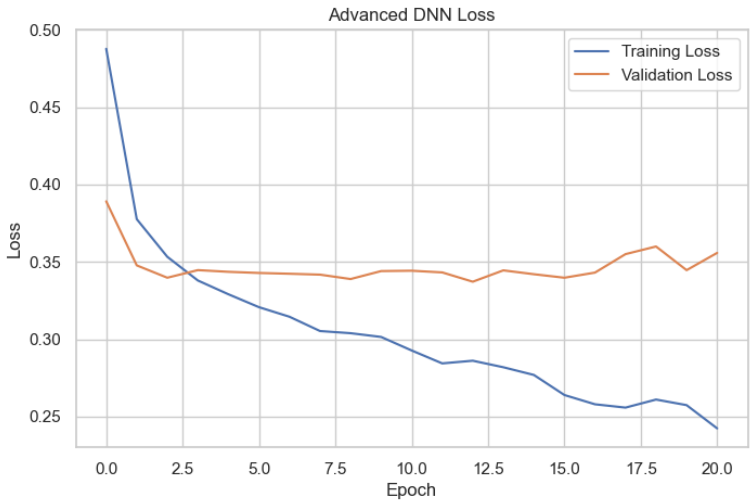


Figure 9.3: DNN training and validation loss

9.6 Evaluation or Verification

Training accuracy rose to about 0.89 and training AUC to about 0.95 by epoch 21, while validation accuracy stayed near 0.84 and validation AUC near 0.91 (Figure 9.2, Figure 9.3), which indicates mild overfitting. On the test set at the validation-selected threshold of 0.40 the DNN scored accuracy 0.8407, precision 0.6934, recall 0.7861, F1 0.7368 and ROC-AUC 0.9128.

9.7 Chapter Summary

The DNN performs close to Logistic Regression and below the boosted trees and FT-Transformer on this dataset, consistent with the general observation that plain MLPs rarely beat tree ensembles on small tabular data.

Chapter 10: TabNet Model Development

10.1 Chapter Introduction

This chapter describes the TabNet classifier trained with the `pytorch-tabnet` package on the preprocessed 62-feature matrix.

10.2 Technical Background

TabNet [5] processes input in sequential decision steps. At each step an attentive transformer produces a sparse feature mask, a feature transformer processes the selected features, and the step outputs are aggregated for the final prediction. The masks provide built-in feature-level interpretability. The relaxation parameter gamma controls how often a feature may be reused across steps.

10.3 Algorithm or Model Explanation

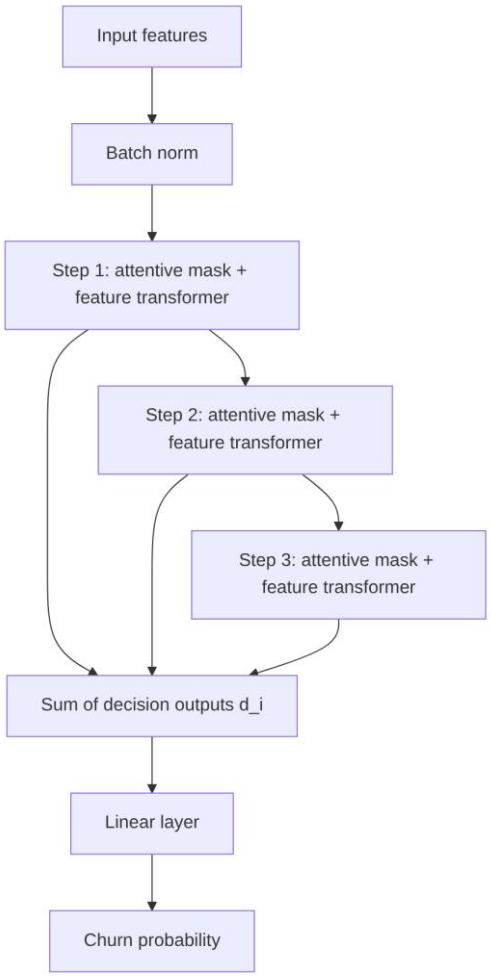


Figure 10.1: TabNet architecture

Table 10.1: TabNet configuration

Parameter	Value
n_d, n_a	16, 16
n_steps	3
gamma	1.3
Optimiser	Adam, learning rate 0.02
Max epochs / patience	80 / 12
Batch size / virtual batch size	256 / 64
Evaluation metric	AUC on the validation set
Seed	42

10.4 Implementation Steps

Step Name: Train TabNet with early stopping

Purpose: Train an attention-based deep model with built-in feature selection.

Input: X_train_fit_ready, y_train_fit, X_val_ready, y_val.

Python Code: Complete code for this stage (taken from the project notebook).

```
import torch
from pytorch_tabnet.tab_model import TabNetClassifier

tabnet_model = TabNetClassifier(
    n_d=16,
    n_a=16,
    n_steps=3,
    gamma=1.3,

    optimizer_fn=torch.optim.Adam,
    optimizer_params={"lr": 0.02},

    seed=42,
    verbose=1
)

tabnet_model.fit(
    X_train=X_train_fit_ready.astype(
        np.float32
    ),

    y_train=y_train_fit.to_numpy(
        dtype=np.int64
    ),

    eval_set=[(
        X_val_ready.astype(np.float32),
        y_val.to_numpy(dtype=np.int64)
    )],

    eval_name=["validation"],
    eval_metric=["auc"],

    max_epochs=80,
```

```
    patience=12,
    batch_size=256,
    virtual_batch_size=64
)

tabnet_probabilities = (
    tabnet_model.predict_proba(
        X_val_ready.astype(np.float32)
    )[:, 1]
)
```

Code Explanation:

- `eval_set` and `eval_metric=["auc"]` drive early stopping with patience 12.
- Inputs are cast to `float32` and labels to `int64`.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook):

```
epoch 0  | loss: 0.64673 | validation_auc: 0.70928
epoch 7  | loss: 0.35357 | validation_auc: 0.90026
epoch 19 | loss: 0.28615 | validation_auc: 0.87854
Early stopping occurred at epoch 19 with best_epoch = 7 and best_validation_auc =
0.90026
```

Verification:

- Best validation AUC (0.90026 at epoch 7) is recorded and the best weights are restored.

Troubleshooting:

- Unstable validation AUC (it fluctuates between 0.878 and 0.900): lower the learning rate or increase `virtual_batch_size`.
- CUDA errors: the notebook ran on CPU; keep `device_name="cpu"` if no GPU is available.

10.5 Evaluation or Verification

On the test set at the validation-selected threshold of 0.30, TabNet scored accuracy 0.8149, precision 0.6349, recall 0.8182, F1 0.7150 and ROC-AUC 0.8915. It had the highest churn recall of all models but the lowest precision, accuracy and AUC. Validation AUC was volatile across epochs, and no hyperparameter search was run, so these results should be seen as an untuned configuration.

10.6 Chapter Summary

TabNet trained successfully but did not outperform the simpler models in this untuned configuration.

Chapter 11: FT-Transformer Model Development

11.1 Chapter Introduction

This chapter describes the FT-Transformer, implemented in PyTorch. It was designated as the final model in this project.

11.2 Technical Background

The FT-Transformer [6] turns every feature into a *token*: numerical features are multiplied by a learned vector and offset by a learned bias, and each categorical feature is mapped through an embedding table. A learnable [CLS] token is prepended and the token sequence is processed by Transformer encoder layers with multi-head self-attention [7]. The [CLS] output feeds a linear classification head. Figure 11.1 shows the architecture.

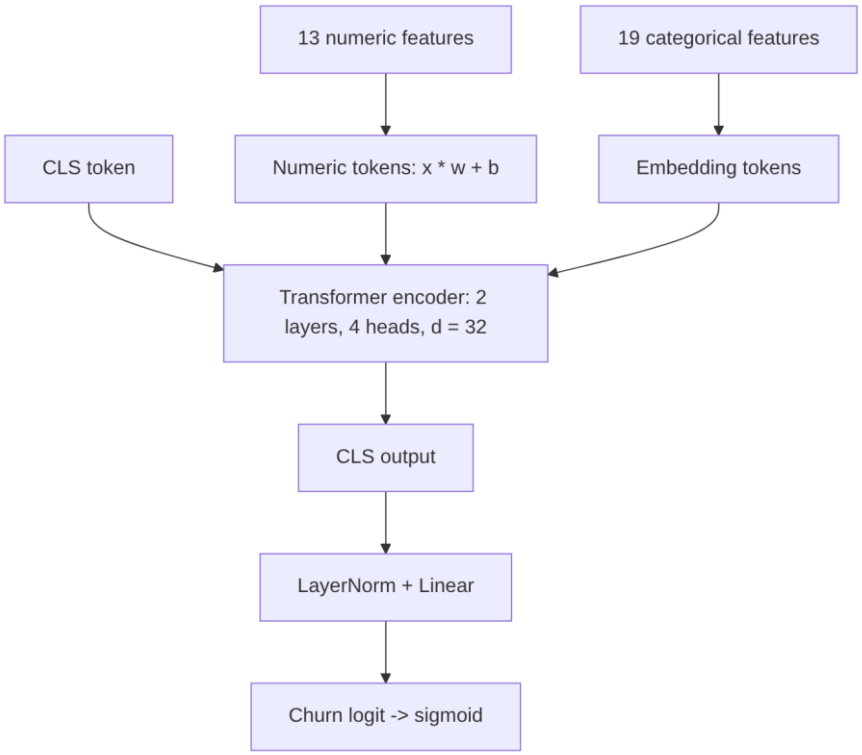


Figure 11.1: FT-Transformer architecture

Table 11.1: FT-Transformer configuration

Parameter	Value
Numeric features	13 (7 standard-scaled + 6 min-max scaled)
Categorical features	19, each with an embedding table (vocabulary index 0 reserved for unknown / missing)
Token dimension d_token	32
Attention heads / encoder layers	4 / 2
Feed-forward width / activation	128 (4 x d_token) / GELU, pre-norm

Parameter	Value
Dropout	0.15
Loss	BCEWithLogits with positive-class weight 2.526 (2,832 stayers / 1,121 churners)
Optimiser	AdamW, learning rate 8e-4, weight decay 1e-4, gradient clipping at 1.0
Batch size / max epochs / patience	128 / 35 / 7 (monitor validation ROC-AUC)

11.3 Methodology

Numeric columns reuse the scalers already fitted on the training data in the `ColumnTransformer`. Categorical columns are converted to integer indices with a vocabulary built from the training data; unseen categories map to index 0. The model is trained with mini-batches and the checkpoint with the highest validation ROC-AUC is kept.

11.4 Implementation Steps

Step Name: Build vocabularies and tensors

Purpose: Convert raw feature columns into the numeric and categorical tensors the model expects.

Input: X_train, X_val, X_test, fitted preprocessor.

Python Code: Complete code for this stage (taken from the project notebook).

```
import copy
from pathlib import Path
import numpy as np
import pandas as pd
import joblib
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score,
                             roc_auc_score, classification_report,
confusion_matrix,
                             ConfusionMatrixDisplay, RocCurveDisplay)

FT_SEED = 42
np.random.seed(FT_SEED)
torch.manual_seed(FT_SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(FT_SEED)
ft_device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("FT device:", ft_device)
for name in ("X_train", "X_val", "X_test", "y_train", "y_val", "y_test",
             "preprocessor", "standard_features", "minmax_features"):
    assert name in globals(), f"Missing {name}: preprocessing cells"

ft_std_cols = list(standard_features)
ft_mm_cols = list(minmax_features)
ft_num_cols = ft_std_cols + ft_mm_cols
```

```
ft_cat_cols = [c for c in X_train.columns if c not in ft_num_cols]

assert set(ft_num_cols).issubset(X_train.columns)
assert len(ft_num_cols) == len(set(ft_num_cols)), "Numeric column duplicate!"
assert set(ft_num_cols).isdisjoint(ft_cat_cols)
assert set(ft_num_cols + ft_cat_cols) == set(X_train.columns)

print("Numerical columns:", len(ft_num_cols), ft_num_cols)
print("Categorical columns:", len(ft_cat_cols), ft_cat_cols)

def ft_category_series(df, col):
    return df[col].astype("string").fillna("__MISSING__").str.strip().replace("",
"__MISSING__")

ft_vocab = {}
for col in ft_cat_cols:
    unique_values = pd.unique(ft_category_series(X_train, col))
    ft_vocab[col] = {str(value): i + 1 for i, value in enumerate(unique_values)}

ft_cardinalities = [len(ft_vocab[col]) + 1 for col in ft_cat_cols]
print("Embedding sizes:", dict(zip(ft_cat_cols, ft_cardinalities)))

def ft_transform_raw(frame, fitted_preprocessor, std_cols, mm_cols, cat_cols,
vocab):
    frame = frame.copy()
    required = std_cols + mm_cols + cat_cols
    missing = [c for c in required if c not in frame.columns]
    if missing:
        raise ValueError(f"Missing FT input columns: {missing}")

    number_parts = []
    if std_cols:
        number_parts.append(np.asarray(

fitted_preprocessor.named_transformers_["standard"].transform(frame[std_cols]),
            dtype=np.float32))
    if mm_cols:
        number_parts.append(np.asarray(

fitted_preprocessor.named_transformers_["minmax"].transform(frame[mm_cols]),
            dtype=np.float32))
    x_num = (np.concatenate(number_parts, axis=1) if number_parts
             else np.zeros((len(frame), 0), dtype=np.float32))

    x_cat = np.zeros((len(frame), len(cat_cols)), dtype=np.int64)
    for i, col in enumerate(cat_cols):
        x_cat[:, i] = (ft_category_series(frame, col)
                       .map(vocab[col]).fillna(0).to_numpy(dtype=np.int64))
    if not np.isfinite(x_num).all():
        raise ValueError("Numeric features contain NaN/inf; check fitted imputer
and features")
    return x_num, x_cat

ft_transform_kwargs = (preprocessor, ft_std_cols, ft_mm_cols, ft_cat_cols,
ft_vocab)
ft_Xtr_num, ft_Xtr_cat = ft_transform_raw(X_train, *ft_transform_kwargs)
ft_Xva_num, ft_Xva_cat = ft_transform_raw(X_val, *ft_transform_kwargs)
ft_ytr = np.asarray(y_train, dtype=np.float32).reshape(-1)
ft_yva = np.asarray(y_val, dtype=np.float32).reshape(-1)

assert ft_Xtr_num.shape[0] == len(ft_ytr)
assert ft_Xtr_cat.shape == (len(ft_ytr), len(ft_cat_cols))
assert all(ft_Xtr_cat[:, i].max() < size for i, size in
enumerate(ft_cardinalities))
```

```
print("Train numeric:", ft_Xtr_num.shape, "categorical:", ft_Xtr_cat.shape)
print("Val numeric:", ft_Xva_num.shape, "categorical:", ft_Xva_cat.shape)

FT_BATCH_SIZE = 128

def ft_loader(x_num, x_cat, y=None, shuffle=False):
    tensors = [torch.from_numpy(x_num), torch.from_numpy(x_cat)]
    if y is not None:
        tensors.append(torch.from_numpy(np.asarray(y, dtype=np.float32)))
    return DataLoader(TensorDataset(*tensors), batch_size=FT_BATCH_SIZE,
                      shuffle=shuffle, num_workers=0)

ft_train_loader = ft_loader(ft_Xtr_num, ft_Xtr_cat, ft_ytr, shuffle=True)
ft_val_loader = ft_loader(ft_Xva_num, ft_Xva_cat, ft_yva, shuffle=False)
print("Training batches:", len(ft_train_loader), "Validation batches:",
len(ft_val_loader))
```

Code Explanation:

- `ft_vocab` maps each training category to an integer; `ft_cardinalities` gives the embedding table sizes.
- `ft_transform_raw` applies the fitted scalers to the numeric columns and the vocabularies to the categorical columns, and refuses inputs with NaN or infinity.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook):

```
FT device: cpu
Numerical columns: 13 [...]
Categorical columns: 19 [...]
Train numeric: (3953, 13) categorical: (3953, 19)
Val numeric: (1318, 13) categorical: (1318, 19)
Training batches: 31 Validation batches: 11
```

Verification:

- Numeric + categorical columns equal the 32 input columns, with no overlap (asserted).
- Every categorical index is smaller than its embedding size (asserted).

Troubleshooting:

- IndexError in an embedding layer: a category index exceeds the table size; unseen categories must map to 0.
- ValueError for NaN: apply the same missing-value fills as in training.

Step Name: Define the model, loss and optimiser

Purpose: Instantiate the FT-Transformer with a class-weighted loss.

Input: ft_num_cols, ft_cardinalities, y_train.

Python Code: Complete code for this stage (taken from the project notebook).

```
class TelecomFTTransformer(nn.Module):
    def __init__(self, n_numeric, cardinalities, d_token=32,
                 n_heads=4, n_layers=2, dropout=0.15):
        super().__init__()
        if d_token % n_heads:
            raise ValueError("d_token must be divisible by n_heads")
        self.num_weight = nn.Parameter(torch.randn(n_numeric, d_token) * 0.02)
        self.num_bias = nn.Parameter(torch.zeros(n_numeric, d_token))
        self.cat_embeddings = nn.ModuleList(
            [nn.Embedding(n_categories, d_token) for n_categories in
cardinalities])
        self.cat_bias = nn.Parameter(torch.zeros(len(cardinalities), d_token))
        self.cls = nn.Parameter(torch.zeros(1, 1, d_token))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token, nhead=n_heads, dim_feedforward=4*d_token,
            dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers, enable_nested_tensor=False)
        self.head = nn.Sequential(nn.LayerNorm(d_token), nn.Linear(d_token, 1))

    def forward(self, x_num, x_cat):
        numeric_tokens = x_num.unsqueeze(-1)*self.num_weight + self.num_bias
        tokens = [numeric_tokens]
        for i, embedding in enumerate(self.cat_embeddings):
            tokens.append(embedding(x_cat[:, i]).unsqueeze(1) + self.cat_bias[i])
        features = torch.cat(tokens, dim=1)
        cls_token = self.cls.expand(features.size(0), -1, -1)
        encoded = self.encoder(torch.cat([cls_token, features], dim=1))
        return self.head(encoded[:, 0]).squeeze(-1)  # logit; no sigmoid here

FT_CONFIG = dict(d_token=32, n_heads=4, n_layers=2, dropout=0.15)
ft_model = TelecomFTTransformer(
    n_numeric=len(ft_num_cols), cardinalities=ft_cardinalities, **FT_CONFIG
).to(ft_device)
print(ft_model)

ft_n0 = int((ft_ytr == 0).sum())
ft_n1 = int((ft_ytr == 1).sum())
if ft_n1 == 0 or ft_n0 == 0:
    raise ValueError("Both target classes must be present in training data")
ft_pos_weight = ft_n0 / ft_n1
ft_loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(ft_pos_weight,
device=ft_device))
ft_optimizer = torch.optim.AdamW(ft_model.parameters(), lr=8e-4, weight_decay=1e-4)
FT_MAX_EPOCHS, FT_PATIENCE = 35, 7
print("No churn:", ft_n0, "Churn:", ft_n1, "pos_weight:", round(ft_pos_weight, 3))

def ft_predict_proba(model, loader, device):
    model.eval()
    output = []
    with torch.inference_mode():
        for batch in loader:
            x_num, x_cat = batch[:2]
            logits = model(x_num.to(device), x_cat.to(device))
            output.append(torch.sigmoid(logits).cpu().numpy())
    return np.concatenate(output) if output else np.array([], dtype=np.float32)
```

Code Explanation:

- `norm_first=True` uses pre-normalisation, which trains more stably.
- The model outputs a logit; `BCEWithLogitsLoss` applies the sigmoid internally, and `torch.sigmoid` is applied only at prediction time.
- `pos_weight` up-weights churners in the loss.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
No churn: 2832 Churn: 1121 pos_weight: 2.526
```

Verification:

- The printed model has 19 embedding tables and a two-layer encoder.

Troubleshooting:

- "d_token must be divisible by n_heads": choose compatible values.

Step Name: Train with early stopping and checkpointing
Purpose: Train the model and keep the best validation checkpoint.

Input: DataLoaders for training and validation.

Python Code: Complete code for this stage (taken from the project notebook).

```
ft_checkpoint = Path("models/telecom_churn_ft_transformer.pt")
ft_checkpoint.parent.mkdir(parents=True, exist_ok=True)
ft_history = []
ft_best_auc = -np.inf
ft_no_improve = 0
ft_best_epoch = 0

for epoch in range(1, FT_MAX_EPOCHS + 1):
    ft_model.train()
    train_loss_sum = 0.0
    for x_num, x_cat, y_batch in ft_train_loader:
        x_num = x_num.to(ft_device)
        x_cat = x_cat.to(ft_device)
        y_batch = y_batch.to(ft_device)
        ft_optimizer.zero_grad(set_to_none=True)
        logits = ft_model(x_num, x_cat)
        loss = ft_loss_fn(logits, y_batch)
        loss.backward()
        nn.utils.clip_grad_norm_(ft_model.parameters(), max_norm=1.0)
        ft_optimizer.step()
        train_loss_sum += loss.item() * len(y_batch)
    train_loss = train_loss_sum / len(ft_ytr)

    ft_model.eval()
    val_loss_sum = 0.0
    with torch.inference_mode():
        for x_num, x_cat, y_batch in ft_val_loader:
            logits = ft_model(x_num.to(ft_device), x_cat.to(ft_device))
```

```
            val_loss_sum += (ft_loss_fn(logits, y_batch.to(ft_device)).item()
                             * len(y_batch))
    val_loss = val_loss_sum / len(ft_yva)
    val_probs = ft_predict_proba(ft_model, ft_val_loader, ft_device)
    val_auc = roc_auc_score(ft_yva.astype(int), val_probs)
    ft_history.append({"epoch": epoch, "train_loss": train_loss,
                       "val_loss": val_loss, "val_auc": val_auc})
    print(f"Epoch {epoch:02d} | train_loss={train_loss:.4f} | "
          f"val_loss={val_loss:.4f} | val_auc={val_auc:.4f}")

    if val_auc > ft_best_auc + 1e-4:
        ft_best_auc, ft_best_epoch, ft_no_improve = val_auc, epoch, 0
        torch.save(ft_model.state_dict(), ft_checkpoint)
    else:
        ft_no_improve += 1
        if ft_no_improve >= FT_PATIENCE:
            print("Early stopping at epoch", epoch)
            break

ft_model.load_state_dict(torch.load(ft_checkpoint, map_location=ft_device,
weights_only=True))
ft_model.eval()
print("Best epoch:", ft_best_epoch, "Best val AUC:", round(ft_best_auc, 4))
```

Code Explanation:

- Each epoch computes training loss, validation loss and validation ROC-AUC.
- The state dict is saved whenever validation AUC improves by more than 1e-4; training stops after seven epochs without improvement.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook):

```
Epoch 01 | train_loss=0.7144 | val_loss=0.5902 | val_auc=0.9005
Epoch 22 | train_loss=0.4499 | val_loss=0.4966 | val_auc=0.9294
Epoch 29 | train_loss=0.4417 | val_loss=0.5253 | val_auc=0.9275
Early stopping at epoch 29
Best epoch: 22 Best val AUC: 0.9294
```

Verification:

- Best epoch 22 with validation AUC 0.9294; weights from that epoch are reloaded.

Troubleshooting:

- Validation loss increases while AUC improves: loss is weighted and AUC is rank-based; they need not move together.

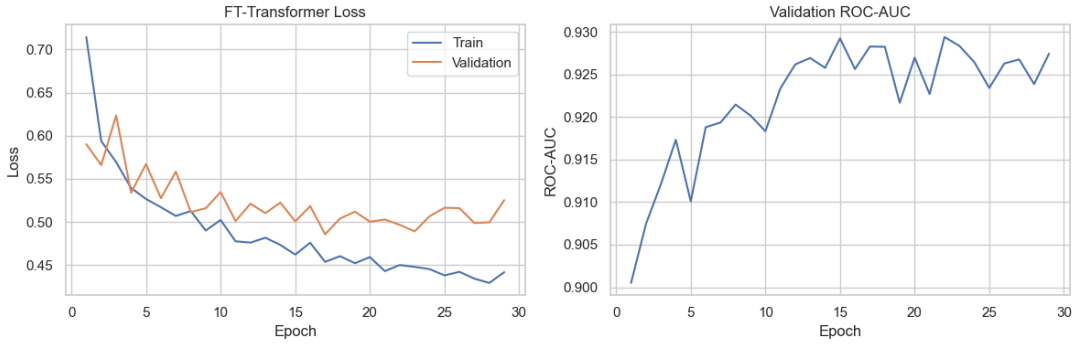


Figure 11.2: FT-Transformer learning curves

Step Name: Tune the threshold on validation data and evaluate on the test set

Purpose: Choose a decision threshold without using the test set, then report test performance once.

Input: Validation and test probabilities.

Python Code: Complete code for this stage (taken from the project notebook).

```
ft_val_probs = ft_predict_proba(ft_model, ft_val_loader, ft_device)

def ft_evaluate(y_true, probabilities, threshold, split_name):
    truth = np.asarray(y_true, dtype=int).reshape(-1)
    pred = (probabilities >= threshold).astype(int)
    return {"Model": "FT-Transformer", "Split": split_name,
            "Threshold": float(threshold), "Accuracy": accuracy_score(truth, pred),
            "Precision": precision_score(truth, pred, zero_division=0),
            "Recall": recall_score(truth, pred, zero_division=0),
            "F1-score": f1_score(truth, pred, zero_division=0),
            "ROC-AUC": roc_auc_score(truth, probabilities)}

ft_val_default = ft_evaluate(ft_yva, ft_val_probs, 0.5, "Validation")
ft_tuning_rows = [ft_evaluate(ft_yva, ft_val_probs, threshold, "Validation")
                  for threshold in np.arange(0.20, 0.801, 0.01)]
ft_tuning_table = pd.DataFrame(ft_tuning_rows).sort_values(
    ["F1-score", "Recall", "Accuracy"], ascending=False).reset_index(drop=True)
ft_best_threshold = round(float(ft_tuning_table.loc[0, "Threshold"]), 2)
ft_val_tuned = ft_evaluate(ft_yva, ft_val_probs, ft_best_threshold, "Validation")
print("Default threshold:")
display(pd.DataFrame([ft_val_default]).round(4))
print("F1-tuned threshold:", ft_best_threshold)
display(pd.DataFrame([ft_val_tuned]).round(4))

ft_Xte_num, ft_Xte_cat = ft_transform_raw(X_test, *ft_transform_kwargs)
ft_yte = np.asarray(y_test, dtype=np.float32).reshape(-1)
ft_test_loader = ft_loader(ft_Xte_num, ft_Xte_cat, y=None, shuffle=False)
ft_test_probs = ft_predict_proba(ft_model, ft_test_loader, ft_device)
ft_test_pred = (ft_test_probs >= ft_best_threshold).astype(int)
ft_test_metrics = ft_evaluate(ft_yte, ft_test_probs, ft_best_threshold, "Test")
display(pd.DataFrame([ft_test_metrics]).round(4))
print(classification_report(ft_yte.astype(int), ft_test_pred,
      target_names=["Stayed", "Churned"], zero_division=0))
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay.from_predictions(ft_yte.astype(int), ft_test_pred,
   display_labels=["Stayed", "Churned"], ax=ax, colorbar=False)
plt.title("FT-Transformer — Test Confusion Matrix")
plt.tight_layout()
plt.show()
```

```
fig, ax = plt.subplots(figsize=(5, 4))
RocCurveDisplay.from_predictions(ft_yte.astype(int), ft_test_probs, ax=ax,
                                 name="FT-Transformer")
plt.title("FT-Transformer — Test ROC Curve")
plt.tight_layout()
plt.show()
```

Code Explanation:

- The threshold grid is 0.20 to 0.80 in steps of 0.01; the threshold with the highest validation F1 (then recall, then accuracy) is chosen: 0.57.
- The same threshold is then applied to the test set.

Expected Output: Representative output from the executed notebook is shown below.

Output (condensed from the executed notebook):

```
Validation, threshold 0.50: Acc 0.8520  Prec 0.7029  Rec 0.8289  F1 0.7607  AUC
0.9294
Validation, threshold 0.57: Acc 0.8649  Prec 0.7438  Rec 0.7995  F1 0.7706  AUC
0.9294
Test,       threshold 0.57: Acc 0.8581  Prec 0.7343  Rec 0.7834  F1 0.7581  AUC
0.9322


              precision    recall  f1-score   support
     Stayed       0.91      0.89      0.90       944
    Churned       0.73      0.78      0.76       374
   accuracy                           0.86      1318
```

Verification:

- Test ROC-AUC (0.9322) is consistent with validation ROC-AUC (0.9294).

Troubleshooting:

- Threshold chosen on the test set would inflate results: here it is chosen on validation.

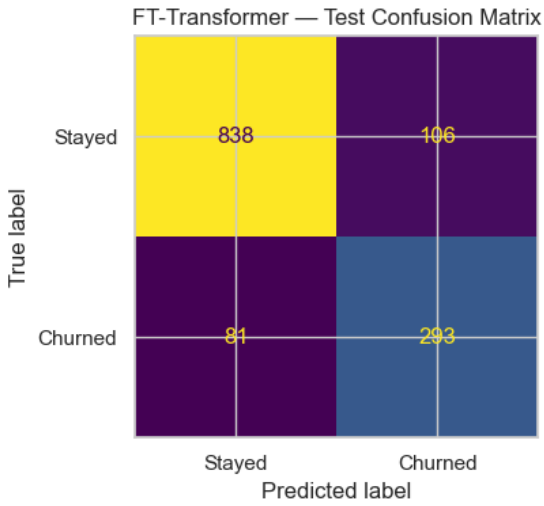


Figure 11.3: FT-Transformer test confusion matrix (threshold 0.57)

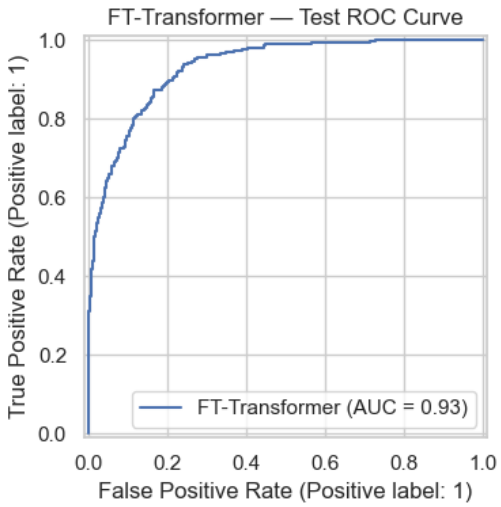


Figure 11.4: FT-Transformer test ROC curve

Step Name: Save artifacts and verify reloading

Purpose: Persist the model, metadata and preprocessor for deployment and confirm identical predictions after reloading.

Input: Trained model, metadata dictionary.

Python Code: Complete code for this stage (taken from the project notebook).

```
ft_metadata = {"input_columns": list(X_train.columns),
               "standard_features": ft_std_cols, "minmax_features": ft_mm_cols,
               "categorical_features": ft_cat_cols, "vocab": ft_vocab,
               "cardinalities": ft_cardinalities, "model_config": FT_CONFIG,
               "n_numeric": len(ft_num_cols), "threshold": ft_best_threshold,
               "best_epoch": ft_best_epoch, "best_validation_auc":
float(ft_best_auc),
               "training_variant": "Real training data only"}
joblib.dump(ft_metadata, "models/ft_transformer_metadata.pkl")
joblib.dump(preprocessor, "models/telecom_preprocessor.pkl")
pd.DataFrame([ft_val_default, ft_val_tuned, ft_test_metrics]).to_csv(
    "models/ft_transformer_metrics.csv", index=False)
ft_history_df.to_csv("models/ft_transformer_training_history.csv", index=False)
print("Saved:", ft_checkpoint)
print("Saved: models/ft_transformer_metadata.pkl")
print("Saved: models/telecom_preprocessor.pkl")

ft_loaded_meta = joblib.load("models/ft_transformer_metadata.pkl")
ft_loaded_preprocessor = joblib.load("models/telecom_preprocessor.pkl")
ft_loaded_model = TelecomFTTransformer(
    n_numeric=ft_loaded_meta["n_numeric"],
    cardinalities=ft_loaded_meta["cardinalities"],
    **ft_loaded_meta["model_config"]
).to(ft_device)
ft_loaded_model.load_state_dict(torch.load(
    "models/telecom_churn_ft_transformer.pt", map_location=ft_device,
weights_only=True))
ft_loaded_model.eval()

num, cat = ft_transform_raw(
    X_val.head(5), ft_loaded_preprocessor,
    ft_loaded_meta["standard_features"], ft_loaded_meta["minmax_features"],
    ft_loaded_meta["categorical_features"], ft_loaded_meta["vocab"])
ft_loaded_probs = ft_predict_proba(
```

```
    ft_loaded_model, ft_loader(num, cat, y=None), ft_device)
np.testing.assert_allclose(ft_loaded_probs, ft_val_probs[:5], rtol=1e-5, atol=1e-6)
print("Loaded model predictions match original model")
print(pd.DataFrame({"Probability of Churn": ft_loaded_probs,
                    "Predicted Churn": (ft_loaded_probs >=
ft_loaded_meta["threshold"]).astype(int)}))
```

Code Explanation:

- The metadata stores input columns, vocabularies, cardinalities, model configuration, threshold (0.57) and training variant.
- `np.testing.assert_allclose` checks that the reloaded model reproduces the original validation probabilities.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
Saved: models\telecom_churn_ft_transformer.pt
Saved: models/ft_transformer_metadata.pkl
Saved: models/telecom_preprocessor.pkl
Loaded model predictions match original model
```

Verification:

- The assertion passes for the first five validation customers.

Troubleshooting:

- `weights_only=True` fails on custom objects: save only the `state_dict` as done here.

11.5 Chapter Summary

The FT-Transformer reached validation ROC-AUC 0.9294 and test ROC-AUC 0.9322. At the threshold chosen on validation data (0.57) its test accuracy is 0.8581 and churn recall 0.7834.

Chapter 12: Model Evaluation and Comparison

12.1 Chapter Introduction

This chapter compares all trained models on the independent test set using a common protocol. All numbers are taken from the executed notebook.

12.2 Objectives

- Report accuracy, precision, recall, F1-score and ROC-AUC for every model.
- Make the evaluation protocol explicit.
- Discuss how the final model was selected and the limits of that selection.

12.3 Evaluation Protocol

Table 12.1: Evaluation protocol

Item	Specification
Positive class	Churn = 1 (customer churned)
Dataset split	60% training (3,953), 20% validation (1,318), 20% test (1,318); stratified, random_state = 42
Training data	Real training split only for all rows except "CatBoost + CTGAN" (training split + 1,976 CTGAN rows)
Threshold	Chosen per model to maximise validation F1 over 0.20 to 0.80 in steps of 0.05 (values listed under the table)
Final evaluation	One pass on the test split using the stored thresholds
Metrics	Accuracy, precision, recall and F1 for the churn class at the chosen threshold; ROC-AUC from probabilities (threshold-independent)
Cross-validation	Not performed; results come from a single split

12.4 Machine Learning and Deep Learning Performance Comparison

Table 12.2: Machine learning and deep learning performance on the independent test set

Model	Data Configuration	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	Real training data	0.8399	0.6909	0.7888	0.7366	0.9158
Random Forest	Real training data	Not performed	Not performed	Not performed	Not performed	Not performed
XGBoost	Real training data	0.8467	0.7172	0.7594	0.7377	0.9246
CatBoost	Real training data	0.8536	0.7303	0.7674	0.7484	0.9313
DNN	Real training data	0.8407	0.6934	0.7861	0.7368	0.9128
TabNet	Real training data	0.8149	0.6349	0.8182	0.7150	0.8915
FT-Transformer	Real training data	0.8604	0.7351	0.7941	0.7635	0.9322
CatBoost + CTGAN	Real training + 1,976 CTGAN rows	0.8520	0.7266	0.7674	0.7464	0.9259

Model	Data Configuration	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Selected Model (FT-Transformer)	Real training data; threshold 0.55	0.8604	0.7351	0.7941	0.7635	0.9322

Thresholds used: Logistic Regression 0.35, XGBoost 0.35, CatBoost 0.35, DNN 0.40, TabNet 0.30, FT-Transformer 0.55, CatBoost + CTGAN 0.35. The test set contains 374 churners and 944 stayers. No synthetic-data experiments other than the CatBoost + CTGAN row were performed.

Table 12.3: Validation F1 and threshold used for each model

Model	Threshold	Validation F1
Logistic Regression	0.35	0.7395
XGBoost	0.35	0.7841
CatBoost	0.35	0.7879
DNN	0.40	0.7257
TabNet	0.30	0.7277
FT-Transformer	0.55	0.7694
CatBoost + CTGAN	0.35	0.7773

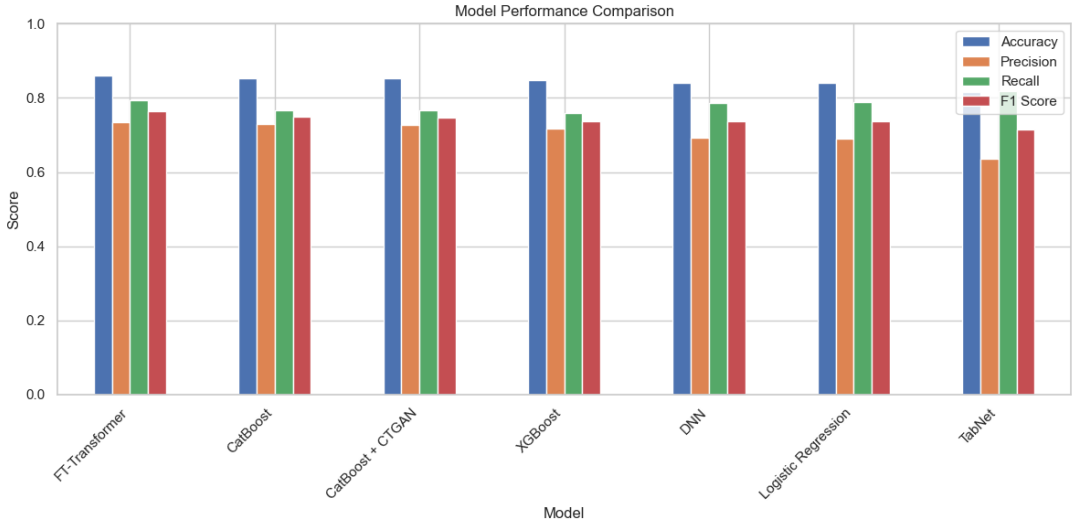


Figure 12.1: Model performance comparison (test set)

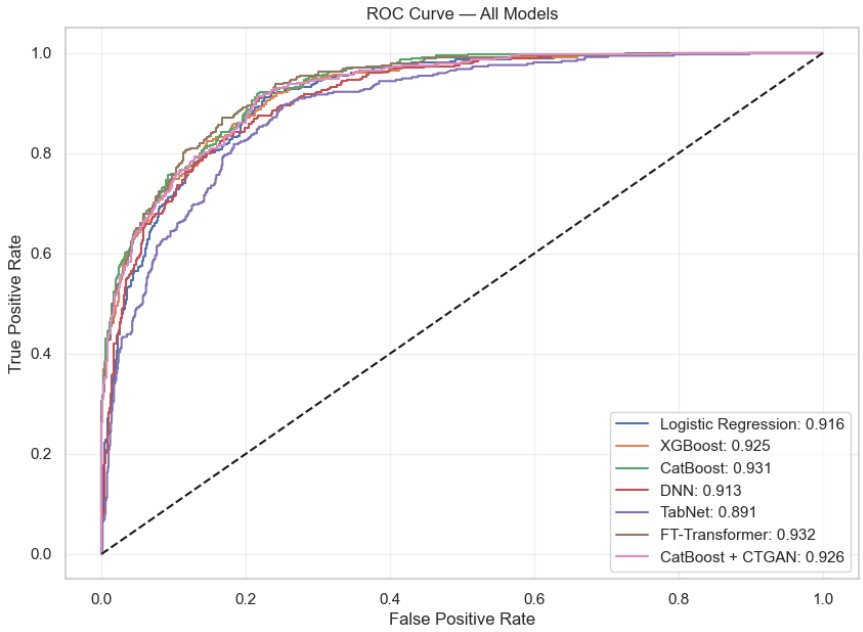


Figure 12.2: ROC curves of all models (test set)

12.5 Model Hyperparameter Configuration

Table 12.4: Model hyperparameter configuration

Model	Configuration
Logistic Regression	C = 1.0, max_iter = 2500, random_state = 42; 62 preprocessed features
Random Forest	Not performed
XGBoost	n_estimators 400, max_depth 4, learning_rate 0.03, subsample 0.85, colsample_bytree 0.85, objective binary:logistic, eval_metric logloss, tree_method hist, random_state 42
CatBoost	iterations 500, depth 5, learning_rate 0.03, loss Logloss, eval_metric AUC, early_stopping_rounds 40 on validation, random_seed 42; native categorical features
DNN	Dense 128 - BatchNorm - Dropout 0.35 - Dense 64 - Dropout 0.30 - sigmoid; Adam 0.001; batch 32; early stopping patience 8
TabNet	n_d 16, n_a 16, n_steps 3, gamma 1.3; Adam 0.02; batch 256, virtual batch 64; max 80 epochs, patience 12
FT-Transformer	d_token 32, 4 heads, 2 layers, dropout 0.15; AdamW 8e-4, weight decay 1e-4; batch 128; pos_weight 2.526; patience 7
CTGAN / TVAE	50 epochs, batch size 500, CPU
WGAN-GP	noise 32, two 128-unit layers, Adam 1e-4 (beta1 0, beta2 0.9), lambda_GP 10, 3 critic steps per generator step, 10 epochs, batch 64

No systematic hyperparameter search was carried out; the configurations above were set by hand. The only tuned quantity is the decision threshold.

12.6 Discussion of Results

- **Best model on the test set.** The FT-Transformer has the highest accuracy (0.8604), F1 (0.7635) and ROC-AUC (0.9322), followed closely by CatBoost (0.8536, 0.7484,

0.9313). The accuracy difference of 0.0068 corresponds to about nine customers out of 1,318, and with a single split and no confidence intervals it should not be treated as a decisive difference.

- **Recall trade-off.** TabNet has the highest churn recall (0.8182) but the lowest precision (0.6349) and AUC. Logistic Regression and the DNN also trade precision for recall at their chosen thresholds.
- **Synthetic data.** CatBoost with CTGAN augmentation was marginally below CatBoost on real data on every test metric. The project therefore provides no evidence that synthetic data improved performance.
- **Accuracy target.** The highest accuracy reached is 86.0%. An accuracy of 90% was not reached; since the minority class is 28.4% of the data and recall on the churn class was prioritised, accuracy alone is not the primary measure of usefulness.

12.7 Model Selection Procedure: A Methodological Caveat

The specification requires that validation data are used for model selection and the test set is reserved for final evaluation. In the notebook the thresholds were chosen on validation data, which is correct. However, the final "best model" was chosen by sorting the **test-set** F1 scores (`comparison` sorted by `F1 Score`, then `comparison.loc[0, "Model"]`), which selects the FT-Transformer. On **validation** F1, CatBoost was highest (0.7879 against 0.7694 for the FT-Transformer). Selecting on the test set makes the test result slightly optimistic for the selected model. This report keeps the FT-Transformer as the designated final model, because it was the model built for deployment, but it flags the issue explicitly. The recommended correction is to choose the final model on validation performance (or, better, on repeated cross-validation) and to re-report the test result once for that choice.

12.8 Threshold Inconsistency

Two thresholds exist for the FT-Transformer. In Chapter 11 a fine grid (step 0.01) gave 0.57, which is stored in the saved metadata (test: accuracy 0.8581, precision 0.7343, recall 0.7834, F1 0.7581, ROC-AUC 0.9322). In this chapter a coarser grid (step 0.05) gave 0.55 (Table 12.2). A deployed service reading the metadata would therefore use 0.57, not the 0.55 shown here. Either value is defensible, but one should be adopted and used consistently in reporting and deployment.

12.9 Implementation Steps

Step Name: Compute thresholds and evaluate all models on the test set
Purpose: Apply one common protocol to every model.

Input: Fitted models, X_val, X_test, y_val, y_test.

Python Code: Complete code for this stage (taken from the project notebook).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    precision_recall_curve,
    average_precision_score
)

from sklearn.inspection import permutation_importance

sns.set_theme(style="whitegrid")
os.makedirs("evaluation_results", exist_ok=True)

# Use the already fitted preprocessor

def prepare_features(X):
    data = preprocessor.transform(X)

    if hasattr(data, "toarray"):
        data = data.toarray()

    return np.asarray(data, dtype=np.float32)



X_val_final = prepare_features(X_val)
X_test_final = prepare_features(X_test)

y_val_final = np.asarray(y_val).ravel()
y_test_final = np.asarray(y_test).ravel()

print("Validation:", X_val_final.shape)
print("Test:", X_test_final.shape)

models = {
    "Logistic Regression": lr_model,
    "XGBoost": xgb_model,
    "CatBoost": cat_model,
    "DNN": dnn_model,
    "TabNet": tabnet_model,
    "FT-Transformer": ft_model
}
```

```
if "catboost_gan" in globals():
    models["CatBoost + CTGAN"] = catboost_gan

def get_probability(name, X):

    model = models[name]

    if name in ["CatBoost", "CatBoost + CTGAN"]:

        X_input = prepare_native_catboost(X)
        probability = model.predict_proba(X_input)[:, 1]

    elif name == "DNN":

        X_input = prepare_features(X)
        probability = model.predict(
            X_input, verbose=0
        ).ravel()

    elif name == "FT-Transformer":

        X_num, X_cat = ft_transform_raw(
            X, *ft_transform_kwargs
        )

        loader = ft_loader(
            X_num, X_cat,
            y=None, shuffle=False
        )

        probability = ft_predict_proba(
            model, loader, ft_device
        )

    else:

        X_input = prepare_features(X)
        probability = model.predict_proba(X_input)[:, 1]

    return np.asarray(probability).ravel()

best_thresholds = {}

for name in models:

    probabilities = get_probability(name, X_val)

    best_f1 = -1
    best_threshold = 0.5

    for threshold in np.arange(0.20, 0.81, 0.05):

        predictions = (
            probabilities >= threshold
        ).astype(int)

        score = f1_score(
            y_val_final,
            predictions,
            zero_division=0
        )
```

```
        if score > best_f1:
            best_f1 = score
            best_threshold = float(threshold)

    best_thresholds[name] = best_threshold

    print(
        name,
        "| Threshold:", round(best_threshold, 2),
        "| Validation F1:", round(best_f1, 4)
    )

results = []
test_probabilities = {}

for name in models:

    probabilities = get_probability(name, X_test)

    test_probabilities[name] = probabilities

    predictions = (
        probabilities >= best_thresholds[name]
    ).astype(int)

    results.append({
        "Model": name,

        "Accuracy": accuracy_score(
            y_test_final, predictions
        ),

        "Precision": precision_score(
            y_test_final, predictions,
            zero_division=0
        ),

        "Recall": recall_score(
            y_test_final, predictions,
            zero_division=0
        ),

        "F1 Score": f1_score(
            y_test_final, predictions,
            zero_division=0
        ),

        "ROC-AUC": roc_auc_score(
            y_test_final, probabilities
        )
    })

results_df = pd.DataFrame(results)

display(results_df.round(4))

comparison = results_df.sort_values(
    by="F1 Score",
    ascending=False
).reset_index(drop=True)
```

```
display(comparison.round(4))

comparison.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 7))

for name, probability in test_probabilities.items():

    fpr, tpr, _ = roc_curve(
        y_test_final, probability
    )

    auc = roc_auc_score(
        y_test_final, probability
    )

    plt.plot(
        fpr, tpr,
        label=f"{name}: {auc:.3f}"
    )

plt.plot([0, 1], [0, 1], "k--")

plt.title("ROC Curve — All Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

import os

os.makedirs("evaluation_results", exist_ok=True)

comparison.to_csv(
    "evaluation_results/final_model_comparison.csv",
    index=False
)

print("Final evaluation results saved!")
```

Code Explanation:

- `get_probability` hides the differences in input format between models (preprocessed matrix, CatBoost DataFrame, FT tensors).
- The threshold loop selects the value with the highest validation F1.

- The results are saved to `evaluation_results/final_model_comparison.csv`.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	FT-Transformer	0.8604	0.7351	0.7941	0.7635	0.9322
1	CatBoost	0.8536	0.7303	0.7674	0.7484	0.9313
2	CatBoost + CTGAN	0.8520	0.7266	0.7674	0.7464	0.9259
3	XGBoost	0.8467	0.7172	0.7594	0.7377	0.9246
4	DNN	0.8407	0.6934	0.7861	0.7368	0.9128
5	Logistic Regression	0.8399	0.6909	0.7888	0.7366	0.9158
6	TabNet	0.8149	0.6349	0.8182	0.7150	0.8915

Verification:

- Values in Table 12.2 agree with this output.
- All models are evaluated on the same 1,318 test rows.

Troubleshooting:

- Different results on re-run: set seeds for all frameworks.
- Selecting the model from the sorted test table (as in the notebook) re-introduces test-set selection; use the validation table instead.

12.10 Chapter Summary

The FT-Transformer is the best model on the test set by a small margin, with CatBoost close behind. The selection procedure used the test set and should be corrected; the synthetic-data experiment did not help.

Chapter 13: Explainable AI Using SHAP

13.1 Chapter Introduction

This chapter presents SHAP explanations. **Important scope note:** the SHAP values in this chapter were generated for the **CatBoost** model (the strongest tree model), using CatBoost's built-in exact SHAP computation, on a random sample of 100 validation customers. They were not generated for the FT-Transformer. Explaining the FT-Transformer would require a model-agnostic explainer such as KernelExplainer or a gradient-based method and is listed as pending work.

13.2 Objectives

- Identify which features most influence the CatBoost churn predictions.
- Show how individual features push predictions up or down.
- Explain one customer prediction in detail.
- Discuss business implications and limitations.

13.3 Technical Background

SHAP [8] decomposes a prediction into a base value plus one additive contribution per feature. For CatBoost the values are in log-odds units: a positive value pushes the prediction toward churn and a negative value toward staying. Figure 13.1 shows the workflow.

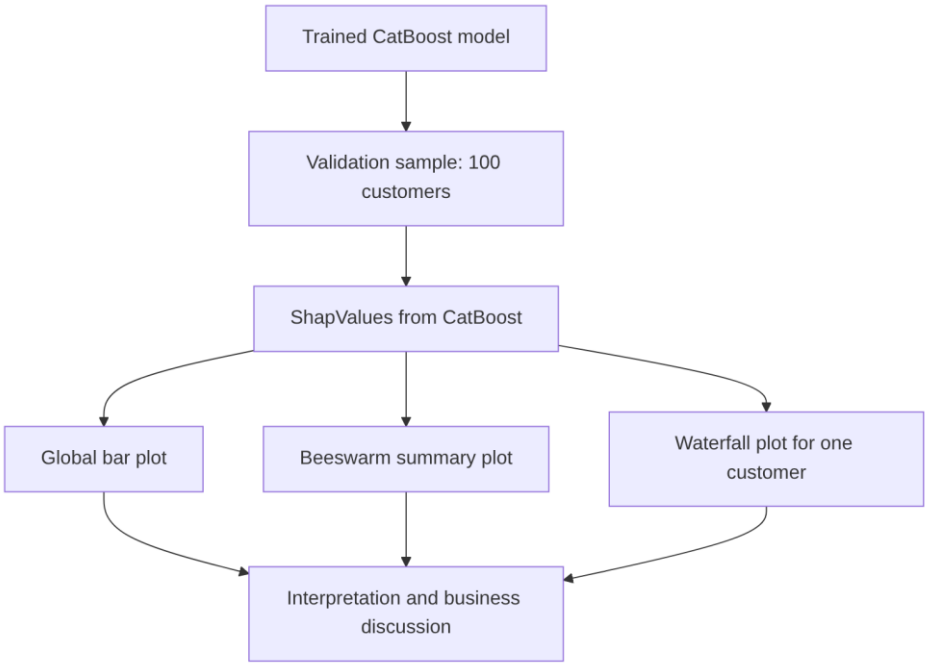


Figure 13.1: SHAP explainability workflow

13.4 Implementation Steps

Step Name: Compute SHAP values and plots

Purpose: Obtain global and local explanations of the CatBoost model.

Input: cat_model, X_val sample of 100 customers.

Python Code: Complete code for this stage (taken from the project notebook).

```
import shap
from catboost import Pool
from IPython.display import display

os.makedirs("evaluation_results/shap", exist_ok=True)

print("SHAP libraries imported!")

X_shap = prepare_native_catboost(
    X_val.sample(
        n=min(100, len(X_val)),
        random_state=42
    )
)

shap_pool = Pool(
    X_shap,
    cat_features=categorical_features
)

print("SHAP dataset:", X_shap.shape)

display(X_shap.head())

shap_output = cat_model.get_feature_importance(
    data=shap_pool,
    type="ShapValues"
)

shap_values = shap_output[:, :-1]
base_values = shap_output[:, -1]

print("SHAP values:", shap_values.shape)
print("Number of features:", X_shap.shape[1])

importance_df = pd.DataFrame({
    "Feature": X_shap.columns,
    "Importance": np.abs(shap_values).mean(axis=0)
})

top20 = importance_df.sort_values(
    by="Importance",
    ascending=False
).head(20)

display(top20)

plt.figure(figsize=(10, 8))

plt.barh(
    top20["Feature"][::-1],
```

```
    top20["Importance"][::-1]
)

plt.title("Top 20 SHAP Feature Importance")
plt.xlabel("Mean Absolute SHAP Value")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/top20_features.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

shap.summary_plot(
    shap_values,
    X_shap,
    feature_names=X_shap.columns.tolist(),
    max_display=20,
    show=False
)

plt.title("SHAP Summary — Telecom Churn")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/shap_summary.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

shap.summary_plot(
    shap_values,
    X_shap,
    plot_type="bar",
    max_display=20,
    show=False
)

plt.title("Global SHAP Feature Importance")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/shap_bar.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

customer_index = 0

customer_shap = shap.Explanation(
    values=shap_values[customer_index],
    base_values=float(base_values[customer_index]),
    data=X_shap.iloc[customer_index].to_numpy(),
    feature_names=X_shap.columns.tolist()
)

shap.plots.waterfall(
```

```
    customer_shap,
    max_display=20,
    show=False
)

plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/customer_waterfall.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

importance_df.to_csv(
    "evaluation_results/shap/feature_importance.csv",
    index=False
)

shap_results = pd.DataFrame(
    shap_values,
    columns=X_shap.columns,
    index=X_shap.index
)

shap_results.to_csv(
    "evaluation_results/shap/customer_shap_values.csv"
)

print("All SHAP results saved successfully!")
```

Code Explanation:

- `get_feature_importance(type="ShapValues")` returns a matrix with one column per feature plus a final base-value column.
- Mean absolute SHAP value per feature gives the global ranking.
- The waterfall plot explains the first customer in the sample.

Expected Output: Representative output from the executed notebook is shown below.

Output (from the executed notebook):

```
SHAP dataset: (100, 32)
SHAP values: (100, 32)
Number of features: 32
All SHAP results saved successfully!
```

Verification:

- Number of SHAP columns equals the number of features (32).
- SHAP values plus the base value reproduce the model's log-odds output for each row (property of exact SHAP).

Troubleshooting:

- Shape mismatch: remember the last column is the base value.
- Slow KernelExplainer for deep models: use a small background sample.

13.5 SHAP Feature Importance Summary

Table 13.1: SHAP feature importance summary (CatBoost, 100 validation customers; mean absolute SHAP value in log-odds)

Rank	Feature	Mean |SHAP|	Reading of the SHAP summary plot
1	Contract	1.1188	Customers split into two groups: one strongly toward churn, one strongly away (consistent with the contract gap in EDA).
2	Number of Referrals	0.6334	Few referrals push toward churn; many referrals push away.
3	Number of Dependents	0.5823	Zero dependents push toward churn; larger numbers push strongly away.
4	Tenure in Months	0.5614	Long tenure pushes away from churn; a few very new customers receive large positive contributions.
5	Monthly Charge	0.2744	Higher charges tend to push toward churn.
6	Payment Method	0.2206	Categorical; direction depends on the payment method.
7	AverageMonthlySpend	0.1849	Higher values tend to push toward churn.
8	Internet Type	0.1741	Categorical; direction depends on the type.
9	Age	0.1429	Older customers tend to receive positive contributions.
10	Total Long Distance Charges	0.1261	Lower values tend to push toward churn (correlated with tenure).
11	Online Security	0.1190	Categorical.
12	Total Revenue	0.1020	Lower values tend to push toward churn (correlated with tenure).
13	TenureGroup	0.0922	Categorical; overlaps with tenure.
14	Married	0.0624	Categorical; small effect.
15	Total Charges	0.0587	Lower values tend to push toward churn (correlated with tenure).
16	Paperless Billing	0.0540	Categorical; small effect.
17	Streaming TV	0.0504	Categorical; small effect.
18	Premium Tech Support	0.0369	Categorical; small effect.
19	Avg Monthly GB Download	0.0368	Small effect.
20	Streaming Movies	0.0271	Categorical; small effect.

The mean absolute values are copied from the notebook output; the last column is a plain-language reading of the beeswarm plot in Figure 13.3 and is descriptive only. Direction for categorical features is not displayed in the beeswarm plot (grey points), so a per-category breakdown is pending.

13.6 Global Feature-Importance Plot

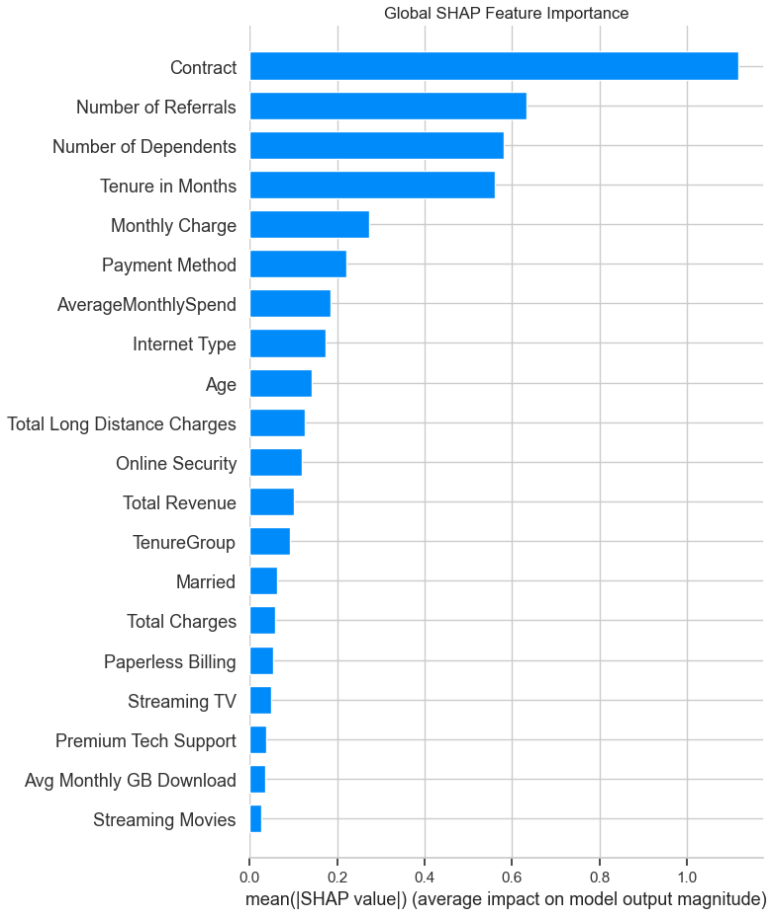


Figure 13.2: Global SHAP feature importance (mean absolute SHAP value)

13.7 SHAP Summary (Beeswarm) Plot

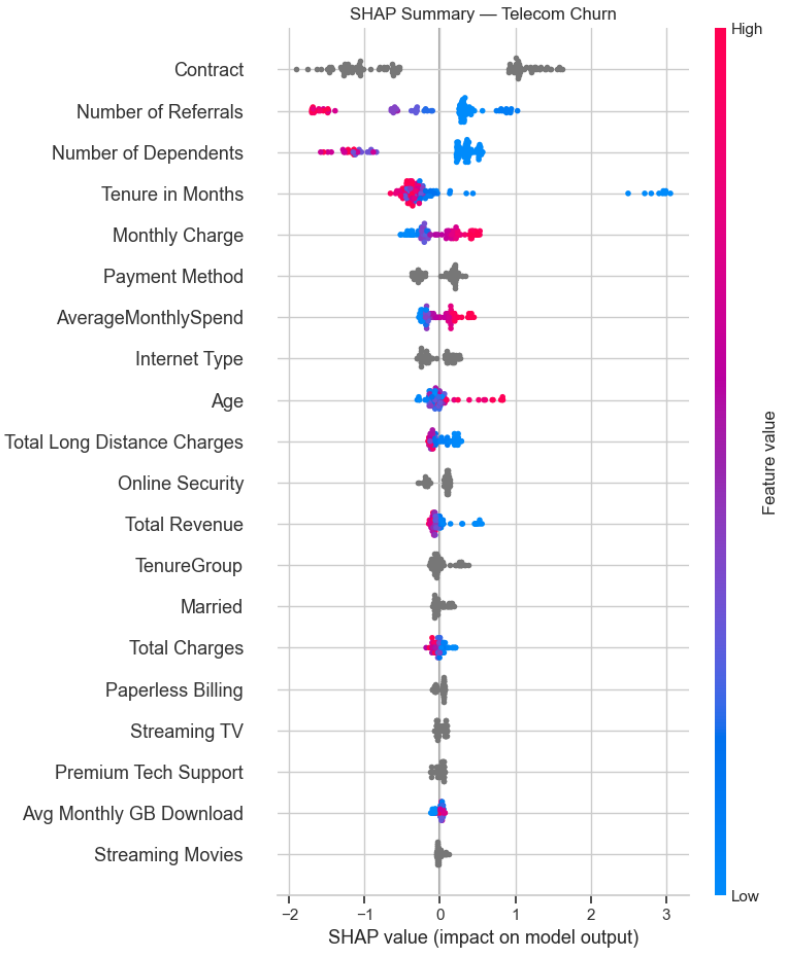


Figure 13.3: SHAP summary plot for the CatBoost model

13.8 Individual Customer Waterfall Plot

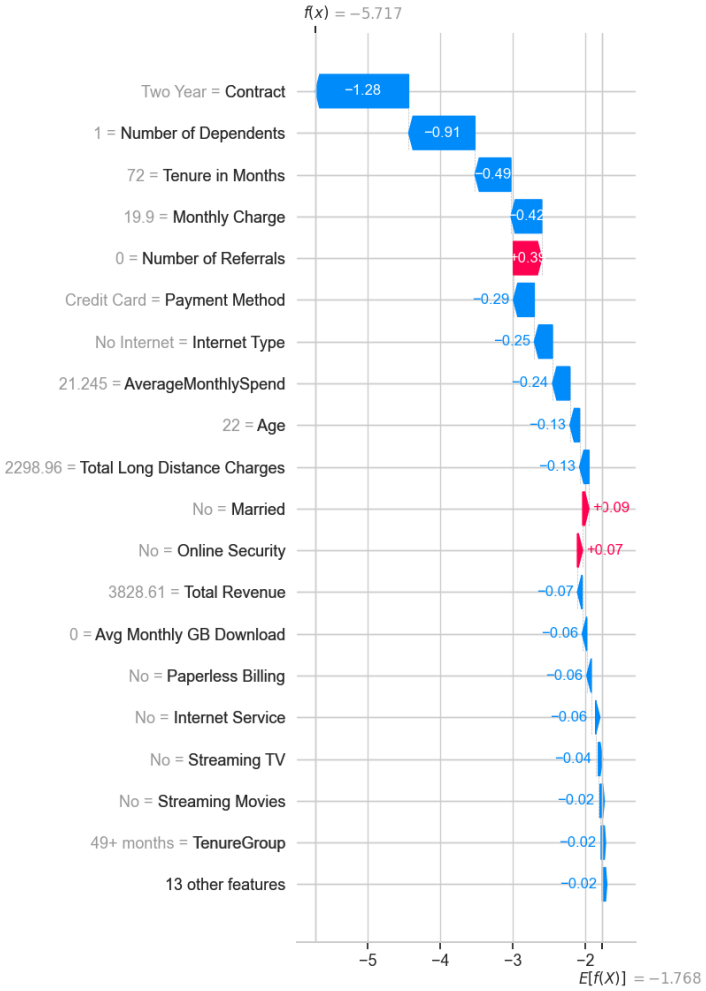


Figure 13.4: SHAP waterfall plot for the first customer in the SHAP sample

The customer in Figure 13.4 has a two-year contract, one dependent, 72 months of tenure and a monthly charge of 19.90. The model output is f(x) = -5.717 in log-odds, far below the base value of -1.768, which corresponds to a churn probability of 0.33% as printed in the notebook. The largest contributions are the two-year contract (-1.28), one dependent (-0.91), tenure (-0.49) and low monthly charge (-0.42); having no referrals adds +0.39 toward churn.

13.9 Business Implications

- **Contract type** is the dominant driver in the model; offering incentives to move month-to-month customers onto longer contracts is a natural retention lever to test.
- **New customers** (short tenure) receive large positive churn contributions, so onboarding and early-life engagement deserve attention.

- **Referrals and household size** are associated with lower predicted churn; they may be proxies for customer engagement or household stability, which should be investigated before designing campaigns around them.
- **Higher monthly charges** are associated with higher predicted churn; price-sensitivity analysis and targeted offers could be tested.

13.10 Limitations of the SHAP Analysis

- SHAP explains the model, not the world. The values show association learned from historical data, **not causal effects**; changing a customer's contract would not necessarily change their churn behaviour by the amount shown.
- The analysis is based on only 100 validation customers, so rankings could change with a larger sample.
- It was run for CatBoost, not for the FT-Transformer; the two models may rank features differently.
- Correlated features (tenure, total charges, total revenue, tenure group) share credit in ways that make individual rankings unstable.
- Values are on the log-odds scale and are not directly percentage-point changes in probability.

13.11 Chapter Summary

For the CatBoost model, contract type, number of referrals, number of dependents and tenure account for most of the explanation, with monthly charge and payment method following. These findings are associational and apply to CatBoost, not to the FT-Transformer; SHAP for the selected model remains pending.

Chapter 14: FastAPI Deployment

14.1 Chapter Introduction

This chapter documents a proposed FastAPI service for the FT-Transformer. **Status: the code in this chapter has been written and checked for syntax errors only. It has not been run against the saved model artifacts, and no API results are claimed.** The notebook performed the relevant preparatory steps (saving the model, metadata and preprocessor, and confirming that a reloaded model reproduces its predictions); the service itself and its tests are pending (Chapter 15).

14.2 Objectives

- Serve churn probabilities over HTTP using the exact feature schema and preprocessing used in training.
- Validate inputs strictly and report errors with standard HTTP codes.
- Document, test and secure the service.

14.3 API Architecture

The service loads three saved artifacts at start-up: the model weights, the metadata (column lists, vocabularies, model configuration, threshold) and the fitted preprocessor. Each request is validated by Pydantic, converted to a DataFrame, extended with the two engineered features, transformed with the training-time scalers and vocabularies and passed to the model. Figure 14.1 shows the flow.

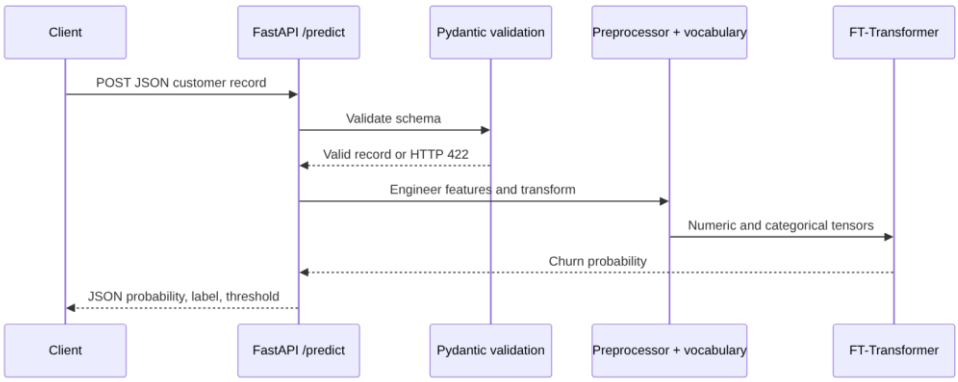


Figure 14.1: FastAPI prediction request and response flow

14.4 Project Folder Structure

```
fastapi_app/
  app/
    __init__.py
```

```
    main.py          # FastAPI routes
    schemas.py       # Pydantic request/response models
    predictor.py     # artifact loading and prediction
    model_def.py     # FT-Transformer class and feature transforms
  models/
    telecom_churn_ft_transformer.pt
    ft_transformer_metadata.pkl
    telecom_preprocessor.pkl
  tests/
    test_api.py
  requirements.txt
  Dockerfile
```

14.5 Dependency Installation

```
python -m venv .venv
.venv\Scripts\activate          # Windows   (Linux/macOS: source
.venv/bin/activate)
pip install -r requirements.txt
```

The versions of scikit-learn, PyTorch and pandas must match those used when the artifacts were created, because the preprocessor is a pickled scikit-learn object.

14.6 Model and Preprocessor Loading

```
"""Loads the saved artifacts once and serves predictions."""
from pathlib import Path
import joblib
import pandas as pd
import torch

from .model_def import TelecomFTTransformer, add_engineered_features, transform_raw

MODEL_DIR = Path(__file__).resolve().parent.parent / "models"



class ChurnPredictor:
    def __init__(self, model_dir: Path = MODEL_DIR):
        # SECURITY: joblib/pickle files can execute code when loaded. Load only
files you created.
        self.meta = joblib.load(model_dir / "ft_transformer_metadata.pkl")
        self.preprocessor = joblib.load(model_dir / "telecom_preprocessor.pkl")
        self.device = torch.device("cpu")
        self.model = TelecomFTTransformer(
            n_numeric=self.meta["n_numeric"],
            cardinalities=self.meta["cardinalities"],
            **self.meta["model_config"],
        ).to(self.device)
        state = torch.load(model_dir / "telecom_churn_ft_transformer.pt",
                           map_location=self.device, weights_only=True)
        self.model.load_state_dict(state)
        self.model.eval()
        self.threshold = float(self.meta["threshold"])
        self.input_columns = list(self.meta["input_columns"])

    def validate_categories(self, df: pd.DataFrame):
        """Return a list of error strings for category values never seen in
training."""
        errors = []
        for col in self.meta["categorical_features"]:
```

```
            if col == "TenureGroup":      # engineered, always valid
                continue
            bad = sorted(set(df[col].astype(str)) - set(self.meta["vocab"][col]))
            if bad:
                errors.append(f"{col}: unknown value(s) {bad}; allowed
{sorted(self.meta['vocab'][col])}")
        return errors

    def predict(self, records: list) -> list:
        df = pd.DataFrame(records)
        df = add_engineered_features(df)
        df = df[self.input_columns]            # exact training column order
        x_num, x_cat = transform_raw(
            df, self.preprocessor, self.meta["standard_features"],
            self.meta["minmax_features"], self.meta["categorical_features"],
self.meta["vocab"])
        with torch.inference_mode():
            logits = self.model(torch.from_numpy(x_num), torch.from_numpy(x_cat))
            probs = torch.sigmoid(logits).numpy()
        return [float(p) for p in probs]
```

The three files are produced by the notebook cells in Chapter 11 (`telecom_churn_ft_transformer.pt`, `ft_transformer_metadata.pkl`, `telecom_preprocessor.pkl`). The class definition and transform functions in `model_def.py` (Appendix A, Section A.2) are copied from the notebook so that the architecture and preprocessing are identical.

14.7 Pydantic Request Schema

```
"""Pydantic request and response schemas. Field aliases are the exact training
column names."""
from typing import List
from pydantic import BaseModel, ConfigDict, Field



class CustomerRecord(BaseModel):
    model_config = ConfigDict(populate_by_name=True)

    gender: str = Field(alias="Gender")
    age: int = Field(alias="Age", ge=0, le=120)
    married: str = Field(alias="Married")
    number_of_dependents: int = Field(alias="Number of Dependents", ge=0)
    number_of_referrals: int = Field(alias="Number of Referrals", ge=0)
    tenure_in_months: int = Field(alias="Tenure in Months", ge=0)
    offer: str = Field(alias="Offer")
    phone_service: str = Field(alias="Phone Service")
    avg_monthly_long_distance_charges: float = Field(alias="Avg Monthly Long
Distance Charges", ge=0)
    multiple_lines: str = Field(alias="Multiple Lines")
    internet_service: str = Field(alias="Internet Service")
    internet_type: str = Field(alias="Internet Type")
    avg_monthly_gb_download: float = Field(alias="Avg Monthly GB Download", ge=0)
    online_security: str = Field(alias="Online Security")
    online_backup: str = Field(alias="Online Backup")
    device_protection_plan: str = Field(alias="Device Protection Plan")
    premium_tech_support: str = Field(alias="Premium Tech Support")
    streaming_tv: str = Field(alias="Streaming TV")
    streaming_movies: str = Field(alias="Streaming Movies")
    streaming_music: str = Field(alias="Streaming Music")
    unlimited_data: str = Field(alias="Unlimited Data")
```

```
    contract: str = Field(alias="Contract")
    paperless_billing: str = Field(alias="Paperless Billing")
    payment_method: str = Field(alias="Payment Method")
    monthly_charge: float = Field(alias="Monthly Charge", ge=0)
    total_charges: float = Field(alias="Total Charges", ge=0)
    total_refunds: float = Field(alias="Total Refunds", ge=0)
    total_extra_data_charges: int = Field(alias="Total Extra Data Charges", ge=0)
    total_long_distance_charges: float = Field(alias="Total Long Distance Charges",
ge=0)
    total_revenue: float = Field(alias="Total Revenue", ge=0)


class PredictionResponse(BaseModel):
    churn_probability: float
    prediction: str
    threshold: float
    model_name: str = "FT-Transformer"


class BatchRequest(BaseModel):
    customers: List[CustomerRecord] = Field(min_length=1, max_length=1000)


class BatchResponse(BaseModel):
    results: List[PredictionResponse]


class HealthResponse(BaseModel):
    status: str
    model_loaded: bool
```

Field aliases are the exact training column names, so the DataFrame built from a request has the same column names as `X_train`. Allowed values for categorical fields are checked against the training vocabulary at run time, which keeps the schema consistent with the saved model without hard-coding category lists.

14.8 Prediction and Health Endpoints

```
"""FastAPI service for churn prediction (design example - not yet executed against
real artifacts)."""
import os
from contextlib import asynccontextmanager

from fastapi import Depends, FastAPI, Header, HTTPException

from .predictor import ChurnPredictor
from .schemas import (BatchRequest, BatchResponse, CustomerRecord, HealthResponse,
                      PredictionResponse)

state = {"predictor": None}
API_KEY = os.environ.get("CHURN_API_KEY")  # if set, clients must send X-API-Key


@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        state["predictor"] = ChurnPredictor()
```

```
    except Exception as exc:  # keep the service up so /health can report the
problem
        print("Model failed to load:", exc)
    yield


app = FastAPI(title="Telecom Churn Prediction API", version="1.0.0",
lifespan=lifespan)


def check_key(x_api_key: str = Header(default=None)):
    if API_KEY and x_api_key != API_KEY:
        raise HTTPException(status_code=401, detail="Invalid or missing API key")


def _predictor():
    if state["predictor"] is None:
        raise HTTPException(status_code=503, detail="Model not loaded")
    return state["predictor"]


@app.get("/health", response_model=HealthResponse)
def health():
    return HealthResponse(status="ok" if state["predictor"] else "degraded",
                          model_loaded=state["predictor"] is not None)


def _run(customers, predictor):
    import pandas as pd
    records = [c.model_dump(by_alias=True) for c in customers]
    errors = predictor.validate_categories(pd.DataFrame(records))
    if errors:
        raise HTTPException(status_code=422, detail=errors)
    probs = predictor.predict(records)
    return [PredictionResponse(churn_probability=round(p, 6),
                               prediction="Churn" if p >= predictor.threshold else
"No Churn",
                               threshold=predictor.threshold) for p in probs]


@app.post("/predict", response_model=PredictionResponse,
dependencies=[Depends(check_key)])
def predict(customer: CustomerRecord):
    return _run([customer], _predictor())[0]


@app.post("/predict/batch", response_model=BatchResponse,
dependencies=[Depends(check_key)])
def predict_batch(batch: BatchRequest):
    return BatchResponse(results=_run(batch.customers, _predictor()))
```

Table 14.1: API endpoint specification

Method	Path	Description	Success response	Error responses
GET	/health	Service and model status	200 {status, model_loaded}	-
POST	/predict	Predict churn for one customer	200 {churn_probability, prediction, threshold, model_name}	401, 422, 503
POST	/predict/batch	Predict for 1 to 1,000 customers	200 {results: [...]}	401, 422, 503
GET	/docs	Swagger UI (generated by FastAPI)	200 HTML	-

Method	Path	Description	Success response	Error responses
GET	/openapi.json	OpenAPI schema	200 JSON	-

14.9 Example JSON Request

```
{
  "Gender": "Female", "Age": 37, "Married": "Yes",
  "Number of Dependents": 0, "Number of Referrals": 2,
  "Tenure in Months": 9, "Offer": "No Offer",
  "Phone Service": "Yes", "Avg Monthly Long Distance Charges": 42.39,
  "Multiple Lines": "No", "Internet Service": "Yes", "Internet Type": "Cable",
  "Avg Monthly GB Download": 16, "Online Security": "No", "Online Backup": "Yes",
  "Device Protection Plan": "No", "Premium Tech Support": "Yes",
  "Streaming TV": "Yes", "Streaming Movies": "No", "Streaming Music": "No",
  "Unlimited Data": "Yes", "Contract": "One Year", "Paperless Billing": "Yes",
  "Payment Method": "Credit Card", "Monthly Charge": 65.6, "Total Charges": 593.3,
  "Total Refunds": 0.0, "Total Extra Data Charges": 0,
  "Total Long Distance Charges": 381.51, "Total Revenue": 974.81
}
```

The values are illustrative. Each categorical value must be one that occurs in the training data; for example, the exact spelling of `Internet Type` and `Contract` values is taken from the dataset, and the service returns HTTP 422 for unknown values.

14.10 Example JSON Response

```
{
  "churn_probability": 0.123456,
  "prediction": "No Churn",
  "threshold": 0.57,
  "model_name": "FT-Transformer"
}
```

This shows the response **format only**; the probability is a placeholder, not a measured output.

14.11 HTTP Error Responses

Table 14.2: HTTP error responses

Status	Cause	Example body
401	Missing or wrong API key (only when CHURN_API_KEY is set)	{"detail": "Invalid or missing API key"}
422	Missing field, wrong type, value below minimum, or unknown category	{"detail": [...]}
503	Model artifacts failed to load	{"detail": "Model not loaded"}
500	Unexpected server error	{"detail": "Internal Server Error"}

14.12 Swagger UI Instructions

1. Start the server: `uvicorn app.main:app --reload`.

2. Open `http://127.0.0.1:8000/docs` in a browser.
3. Expand `POST /predict`, choose "Try it out", paste a JSON request and press "Execute".
4. Inspect the response body, status code and the curl command shown by Swagger.

14.13 API Testing Code

```
"""API tests. Requires the three saved model artifacts in ./models. Run: pytest -
q"""
from fastapi.testclient import TestClient
from app.main import app

SAMPLE = {
    "Gender": "Female", "Age": 37, "Married": "Yes", "Number of Dependents": 0,
    "Number of Referrals": 2, "Tenure in Months": 9, "Offer": "No Offer",
    "Phone Service": "Yes", "Avg Monthly Long Distance Charges": 42.39,
    "Multiple Lines": "No", "Internet Service": "Yes", "Internet Type": "Cable",
    "Avg Monthly GB Download": 16, "Online Security": "No", "Online Backup": "Yes",
    "Device Protection Plan": "No", "Premium Tech Support": "Yes", "Streaming TV":
"Yes",
    "Streaming Movies": "No", "Streaming Music": "No", "Unlimited Data": "Yes",
    "Contract": "One Year", "Paperless Billing": "Yes", "Payment Method": "Credit
Card",
    "Monthly Charge": 65.6, "Total Charges": 593.3, "Total Refunds": 0.0,
    "Total Extra Data Charges": 0, "Total Long Distance Charges": 381.51, "Total
Revenue": 974.81,
}



def test_health():
    with TestClient(app) as client:
        r = client.get("/health")
        assert r.status_code == 200
        assert r.json()["model_loaded"] is True



def test_predict_valid():
    with TestClient(app) as client:
        body = dict(SAMPLE, Offer="No Offer")
        r = client.post("/predict", json=body)
        assert r.status_code == 200
        assert 0.0 <= r.json()["churn_probability"] <= 1.0



def test_missing_field_returns_422():
    with TestClient(app) as client:
        body = dict(SAMPLE, Offer="No Offer")
        body.pop("Contract")
        assert client.post("/predict", json=body).status_code == 422



def test_unknown_category_returns_422():
    with TestClient(app) as client:
        body = dict(SAMPLE, Offer="No Offer", Contract="Weekly")
        assert client.post("/predict", json=body).status_code == 422
```

14.14 Security Considerations

- **Unsafe deserialisation.** `joblib.load` and pickle can execute arbitrary code; load only artifacts you created, and keep them read-only on the server.
- **Authentication.** The example supports an optional `X-API-Key` header; production use needs proper authentication and secret management.
- **Transport security.** Serve behind HTTPS (a reverse proxy such as Nginx or a cloud load balancer).
- **Input limits.** Batch size is capped at 1,000 and numeric fields have lower bounds; add request-size and rate limits at the proxy.
- **Personal data.** Customer attributes are personal data; avoid logging request bodies and follow applicable data-protection rules.
- **Dependencies.** Pin and regularly update package versions.

14.15 Deployment Instructions

```
# Local
uvicorn app.main:app --host 0.0.0.0 --port 8000

# Docker
docker build -t churn-api .
docker run -p 8000:8000 -e CHURN_API_KEY=change-me churn-api
```

```
FROM python:3.11-slim
WORKDIR /srv
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY models ./models
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

14.16 Common Errors and Solutions

- `/health` reports degraded: an artifact is missing or was created with different library versions.
- HTTP 422 for a valid-looking customer: a categorical value differs from the training vocabulary (for example a different spelling).
- Predictions differ from the notebook: confirm the threshold (0.57 in metadata against 0.55 in Chapter 12) and that engineered features are created.

14.17 Chapter Summary

A complete service design with typed schemas, strict validation, health checks and a Docker recipe has been specified. Running and testing it against the real artifacts is the remaining step.

Chapter 15: System Testing

15.1 Chapter Introduction

This chapter lists the verification checks that were actually executed in the notebook and the API tests that remain to be completed. A test is marked "Passed" only if its result appears in the notebook output.

15.2 Software and Hardware Requirements

Table 15.1: Software and hardware requirements

Item	Value
Operating system (observed)	Windows (file paths in the notebook output are Windows paths)
Python (observed)	3.11 (from the traceback path "Python311")
Processing	CPU only; TensorFlow reported no GPU support on native Windows and the PyTorch device was "cpu"
RAM / CPU model	To be measured
Libraries	pandas, numpy, scikit-learn, matplotlib, seaborn, joblib, ctgan, tensorflow/keras, torch, pytorch-tabnet, xgboost, catboost, shap (exact versions: to be recorded with pip freeze)
Deployment (proposed)	FastAPI, Uvicorn, Pydantic v2, Docker (optional)

15.3 System Test Cases

Table 15.2: System test cases

ID	Test	Expected result	Status
T1	Train, validation and test indices are disjoint	Assertions pass	Passed (notebook)
T2	GAN training data has no missing values, no target column in features, binary target	Assertions pass	Passed (notebook)
T3	FT-Transformer numeric and categorical column sets are disjoint and complete	Assertions pass	Passed (notebook)
T4	Reloaded FT-Transformer reproduces original probabilities (first 5 validation rows)	assert_allclose passes	Passed (notebook)
T5	Single-customer prediction from the best model	Probability and label returned (21.69%, "No Churn")	Passed (notebook)
T6	Preprocessor fitted on training data only	fit_transform on train; transform on others	Passed by code inspection
T7	GET /health returns model_loaded true	200	To be completed
T8	POST /predict with valid customer	200, probability in [0, 1]	To be completed
T9	POST /predict with missing field	422	To be completed
T10	POST /predict with unknown category	422	To be completed
T11	API probability equals notebook probability for the same customer	Equal within tolerance	To be completed
T12	POST /predict/batch with 1,000 customers	200 with 1,000 results	To be completed
T13	Wrong API key when key is required	401	To be completed

15.4 Evaluation or Verification

Tests T1 to T6 give confidence in data-leakage control and in model persistence. Tests T7 to T13 cover the API and must be run after the service is started with the real artifacts (code in Chapter 14). Test T11 is the most important check that the service and the notebook agree.

15.5 Chapter Summary

Data-splitting and persistence checks have passed in the notebook; API tests are specified but have not been run.

Chapter 16: Expected Outcomes

16.1 Chapter Introduction

This chapter compares the outcomes anticipated at the outset with what was achieved, using only measured results.

Table 16.1: Planned outcomes and status

Planned outcome	Status	Evidence
Clean, leakage-controlled churn dataset	Achieved	Chapters 3, 4, 6; assertions in notebook
EDA of churn drivers	Achieved	Chapter 5
Synthetic data with CTGAN, TVAE, WGAN-GP	Achieved (generated); quality evaluation only partial	Chapter 7: shape, missing values, class ratio, KDE plot; no formal fidelity metrics
Benefit of synthetic data to classification	Not demonstrated	CatBoost + CTGAN slightly below CatBoost (Table 12.2)
Class-imbalance comparison (class weights, SMOTE, GAN)	Partly done	Threshold tuning, FT class weighting and CTGAN tried; SMOTE and a controlled comparison not performed
ML and DL model comparison	Achieved except Random Forest	Table 12.2
TabNet and FT-Transformer	Achieved	Chapters 10, 11
Accuracy of 90% or more	Not achieved	Best test accuracy 0.8604
Strong churn-class recall	Moderate	Recall 0.76 to 0.82 across models; FT-Transformer 0.7941 at 0.55
SHAP explainability	Achieved for CatBoost; pending for FT-Transformer	Chapter 13
FastAPI prediction service	Designed; not yet run or tested	Chapters 14, 15

16.2 Expected Outcomes of Further Work

Cross-validation, a fair imbalance-handling comparison and a validated synthetic-data evaluation would turn the current single-split findings into more reliable evidence. It is an open question whether any of them would raise accuracy toward 90%; the closeness of the best models (accuracy 0.85 to 0.86, ROC-AUC 0.93) suggests that the available features may limit achievable performance.

16.3 Chapter Summary

Most planned outcomes were achieved; synthetic-data benefit, the 90% accuracy ambition and API verification were not.

Chapter 17: Advantages and Limitations

17.1 Advantages

- Leakage-aware design: split first, fit preprocessing and generators on training data only, independent test set.
- Broad model comparison, from a linear baseline to boosted trees, a DNN, TabNet and the FT-Transformer, on identical splits.
- Honest synthetic-data evaluation: the augmentation experiment was run and its null result reported.
- Threshold tuning on validation data addresses class imbalance without altering the test set.
- Explainability through SHAP, and a reproducible artifact set (model, metadata, preprocessor) with a reload check.

17.2 Limitations

- Single random split; no cross-validation or confidence intervals, so small differences between models are not statistically supported.
- Final model selected using the test set (Chapter 12).
- No hyperparameter search; DNN, TabNet and generators use hand-set values and the WGAN-GP was trained for only ten epochs.
- Random Forest, SMOTE and a controlled imbalance comparison were not run; class weights were computed but not applied to the classical models.
- Synthetic data quality was assessed only by simple checks; invalid values (negative charges, fractional counts) appeared in samples, and the engineered-feature repair function was not applied before augmentation.
- SHAP was computed for CatBoost on 100 customers, not for the selected FT-Transformer.
- The API is designed but not executed or tested.
- Public sample dataset (IBM sample data): results may not transfer to a real operator's customers.

17.3 Chapter Summary

The pipeline is sound in structure, but conclusions are limited by single-split evaluation and several unfinished items, listed above and addressed in Chapter 18.

Chapter 18: Future Enhancements

18.1 Chapter Introduction

This chapter proposes improvements, each tied to a limitation.

Table 18.1: Project limitations and future enhancements

Limitation	Proposed enhancement
Single split, no confidence intervals	Stratified repeated k-fold cross-validation; report mean and standard deviation; paired tests between models
Test-set model selection	Select the model and threshold on validation or cross-validation only; evaluate the test set once
Threshold mismatch (0.55 against 0.57)	Adopt one threshold, ideally chosen by a cost-based criterion (cost of a missed churner against cost of a retention offer)
No hyperparameter search	Optuna or Bayesian search for CatBoost, XGBoost, DNN, TabNet and FT-Transformer
Random Forest and SMOTE missing; class weights not applied	Add Random Forest; run a controlled comparison of class weights, SMOTE and GAN augmentation under identical folds
Weak synthetic-data evaluation	Column-wise and correlation fidelity metrics, privacy checks, and train-on-synthetic / test-on-real; apply business-rule repair before augmentation; longer WGAN-GP training
SHAP only for CatBoost	Explain the FT-Transformer (KernelExplainer or attention-based attributions); larger sample; per-category analysis
API not run	Run Chapter 15 tests T7 to T13; add monitoring, logging without personal data, and model versioning
Public sample data	Validate on operator data; monitor drift and recalibrate probabilities

18.2 Chapter Summary

The highest-value next steps are cross-validated evaluation with correct model selection, a fair imbalance comparison and running the API tests.

Chapter 19: Conclusion

This project built an end-to-end, leakage-controlled pipeline for telecom churn prediction and investigated tabular synthetic data generation. On 6,589 labelled customers (28.4% churners), the FT-Transformer achieved the highest independent test performance (accuracy 0.8604, F1 0.7635, ROC-AUC 0.9322 at threshold 0.55), narrowly ahead of CatBoost (0.8536, 0.7484, 0.9313). The differences among the best models are small, and the model was selected using the test set, so the ranking should be treated as provisional. TabNet and the DNN did not outperform simpler models, and a 90% accuracy ambition was not reached.

CTGAN, TVAE and WGAN-GP were trained on training data only. CTGAN reproduced the class balance best, but adding its output to CatBoost training did not improve test metrics, so no benefit of synthetic data is claimed. SHAP analysis of the CatBoost model identified contract type, referrals, dependents, tenure and monthly charge as the main contributors; these are associations, not causes. A FastAPI deployment has been designed but not yet executed or tested.

The work provides a reproducible foundation. Cross-validated evaluation, corrected model selection, a controlled comparison of imbalance-handling methods, SHAP for the final model and verified API tests would complete it.

Chapter 20: References

[1] L. Xu, M. Skoularidou, A. Cuesta-Infante and K. Veeramachaneni, "Modeling Tabular Data using Conditional GAN," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.

[2] M. Arjovsky, S. Chintala and L. Bottou, "Wasserstein Generative Adversarial Networks," in *Proc. 34th International Conference on Machine Learning (ICML)*, PMLR 70, 2017, pp. 214-223.

[3] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin and A. Courville, "Improved Training of Wasserstein GANs," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[4] I. Goodfellow et al., "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, 2014.

[5] S. O. Arik and T. Pfister, "TabNet: Attentive Interpretable Tabular Learning," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 6679-6687.

[6] Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.

[7] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[8] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.

[9] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018.

[10] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785-794.

[11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.

[13] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," arXiv:1312.6114, 2013 (also in *Proc. ICLR 2014*).

[14] Maven Analytics, "Telecom Customer Churn," Maven Analytics Data Playground (based on IBM sample data sets). https://mavenanalytics.io/data-playground/telecom-customer-churn

[15] FastAPI documentation. https://fastapi.tiangolo.com

[16] CTGAN and TVAE implementation (package `ctgan`), SDV project. https://github.com/sdv-dev/CTGAN

[17] PyTorch documentation. https://pytorch.org/docs

[18] pytorch-tabnet package (DreamQuark). https://github.com/dreamquark-ai/tabnet

[19] SHAP documentation. https://shap.readthedocs.io

Appendix A: Complete Project Source Code

This appendix reproduces every code cell of the project notebook in execution order, followed by the FastAPI support module. Cell numbers refer to the notebook cell index (starting at 0). Code is unchanged from the notebook, including its comment lines; cells that contain only markdown are omitted. The full set of files is also supplied as `Telecom_Churn_Project_Source_Code.zip`.

A.1 Notebook code (Telecom.ipynb)

Stage 1: Data Collection

```
# Notebook cell 11
import pandas as pd
# Notebook cell 14
telecom = pd.read_csv("telecom_customer_churn.csv")
print("Dataset loaded successfully.")
# Notebook cell 17
print(telecom.head())
# Notebook cell 20
print(telecom.shape)
# Notebook cell 23
print(telecom.columns.tolist())
# Notebook cell 26
telecom.info()
# Notebook cell 29
print(telecom.isnull().sum())
# Notebook cell 32
print("Duplicate rows:", telecom.duplicated().sum())
# Notebook cell 35
print(telecom["Customer Status"].value_counts(dropna=False))
```

Stage 2: Data Preparation

```
# Notebook cell 40
import numpy as np
# Notebook cell 41
telecom.columns = telecom.columns.str.strip()
# Notebook cell 43
for col in telecom.select_dtypes(
 include=["object", "string"]
 ).columns:
 telecom[col] = telecom[col].map(
  lambda value: value.strip()
  if
  isinstance(value, str)
  else value
  )
# Notebook cell 45
telecom = telecom.replace(r"^\s*$", np.nan, regex=True)
print("Dataset shape:", telecom.shape)
print(telecom.head())
# Notebook cell 49
telecom["Customer Status"] = (telecom["Customer
Status"].astype("string").str.strip().str.title())
# Notebook cell 51
telecom = telecom[telecom["Customer Status"].isin(["Stayed", "Churned"])].copy()
```

```
# Notebook cell 53
telecom["Churn"] = (telecom["Customer Status"].map({"Stayed": 0, "Churned":
1}).astype(int))
# Notebook cell 54
print("Prepared dataset shape:", telecom.shape)
# Notebook cell 55
print("\nTarget distribution:")
print(telecom["Churn"].value_counts())
# Notebook cell 57
missing_columns = telecom.isnull().sum()
print(missing_columns)
# Notebook cell 58
drop_columns = [
    "Customer ID", "City", "Zip Code", "Latitude", "Longitude",
    "Customer Status", "Churn Category", "Churn Reason",
]
telecom = telecom.drop(
    columns=drop_columns,
    errors="ignore"
).copy()
# Notebook cell 59
telecom["Offer"] = telecom["Offer"].fillna("No Offer")

telecom["Avg Monthly Long Distance Charges"] = (
    telecom["Avg Monthly Long Distance Charges"]
    .fillna(0)
)

telecom["Multiple Lines"] = (
    telecom["Multiple Lines"]
    .fillna("No")
)

telecom["Internet Type"] = (
    telecom["Internet Type"]
    .fillna("No Internet")
)

telecom["Avg Monthly GB Download"] = (
    telecom["Avg Monthly GB Download"]
    .fillna(0)
)
# Notebook cell 60
internet_service_cols = [
    "Online Security",
    "Online Backup",
    "Device Protection Plan",
    "Premium Tech Support",
    "Streaming TV",
    "Streaming Movies",
    "Streaming Music",
    "Unlimited Data"
]

for col in internet_service_cols:
    telecom[col] = telecom[col].fillna("No")
# Notebook cell 61
print("\nModeling dataset missing values:")
print(telecom.isnull().sum())
# Notebook cell 65
telecom["AverageMonthlySpend"] = (
    telecom["Total Charges"]
    /
    telecom["Tenure in Months"].replace(
```

```
        0, np.nan
    )
)
```

```
# Notebook cell 67
```

```
telecom["TenureGroup"] = pd.cut(
    telecom["Tenure in Months"],

    bins=[
        -np.inf,
        6,
        12,
        24,
        48,
        np.inf
    ],

    labels=[
        "0-6 months",
        "7-12 months",
        "13-24 months",
        "25-48 months",
        "49+ months"
    ]
).astype(object)
```

```
# Notebook cell 69
```

```
leakage_drop = [
    "Customer ID",
    "Customer Status",
    "Churn Category",
    "Churn Reason",
    "Churn Score",
    "CLTV",
    "City",
    "Zip Code",
    "Latitude",
    "Longitude"
]

X = telecom.drop(
    columns=leakage_drop + ["Churn"],
    errors="ignore"
)

y = telecom["Churn"]

print(X.shape)
print(y.shape)
```

```
# Notebook cell 70
```

```
from sklearn.model_selection import train_test_split
```

```
# Notebook cell 72
```

```
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.20,
random_state=42, stratify=y)
```

```
# Notebook cell 74
```

```
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
random_state=42, stratify=y_dev)
```

```
# Notebook cell 75
```

```
print("Training:", X_train.shape, y_train.shape)
print("Validation:", X_val.shape, y_val.shape)
print("Testing:", X_test.shape, y_test.shape)
```

```
# Notebook cell 77
```

```
assert set(X_train.index).isdisjoint(
    X_val.index
)
```

```
assert set(X_train.index).isdisjoint(
    X_test.index
)

assert set(X_val.index).isdisjoint(
    X_test.index
)

print("No overlapping rows between splits.")
```

```
# Notebook cell 80
```

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler,
    MinMaxScaler,
    OneHotEncoder
)
import joblib
from pathlib import Path
```

```
# Notebook cell 82
```

```
numeric_features = X_train.select_dtypes(include=["number"]).columns.tolist()
```

```
# Notebook cell 84
```

```
categorical_features = [
 col for col in X_train.columns
 if col not in numeric_features
]
```

```
# Notebook cell 86
```

```
minmax_features = [
    "Age",
    "Tenure in Months",
    "Monthly Charge",
    "Total Charges",
    "Total Revenue",
    "AverageMonthlySpend"
]
```

```
# Notebook cell 88
```

```
standard_features = [
 col for col in numeric_features
 if col not in minmax_features
]
```

```
# Notebook cell 90
```

```
standard_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")),
("scaler", StandardScaler())])
```

```
# Notebook cell 92
```

```
minmax_pipeline = Pipeline([("imputer", SimpleImputer(strategy="median")),
("scaler", MinMaxScaler())])
```

```
# Notebook cell 94
```

```
categorical_pipeline = Pipeline([("imputer",
SimpleImputer(strategy="most_frequent")), ("encoder",
OneHotEncoder(handle_unknown="ignore"))])
```

```
# Notebook cell 96
```

```
preprocessor = ColumnTransformer([
    (
        "standard",
        standard_pipeline,
        standard_features
    ),
    (
        "minmax",
        minmax_pipeline,
        minmax_features
    ),
    (
        "categorical",
```

```
        categorical_pipeline,
        categorical_features
    )
])
```

```
# Notebook cell 98
X_train_ready = preprocessor.fit_transform(X_train)
```

```
# Notebook cell 100
X_val_ready = preprocessor.transform(X_val)
```

```
# Notebook cell 101
print("\nStandardScaler columns:", standard_features)
print("\nMinMaxScaler columns:", minmax_features)
print("\nCategorical columns:", categorical_features)
print("\nPrepared training shape:", X_train_ready.shape)
print("\nPrepared validation shape:", X_val_ready.shape)
```

Stage 3: Exploratory Data Analysis (EDA)

```
# Notebook cell 107
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Notebook cell 109
sns.set_theme(style="whitegrid")
```

```
# Notebook cell 112
eda_df = X_train.copy()
```

```
# Notebook cell 114
eda_df["Churn"] = y_train.to_numpy()
```

```
# Notebook cell 115
print("Training dataset shape:", eda_df.shape)
```

```
# Notebook cell 116
print("\nFirst five rows:")
print(eda_df.head())
```

```
# Notebook cell 117
print("\nColumn names:")
print(eda_df.columns.tolist())
```

```
# Notebook cell 119
churn_counts = (eda_df["Churn"].value_counts().reindex([0, 1], fill_value=0))
churn_percentages = (churn_counts / churn_counts.sum() * 100)
print("Customer counts:")
print(churn_counts.rename(index={
 0: "Stayed",
 1: "Churned"
}))
print("\nCustomer percentages:")
print(churn_percentages.round(2))
plt.figure(figsize=(7, 5))
ax = sns.countplot(
 data=eda_df,
 x="Churn",
 order=[0, 1]
)
ax.set_xticks([0, 1])
ax.set_xticklabels(["Stayed (0)", "Churned (1)"])
plt.title("Telecom Customer Churn Distribution")
plt.xlabel("Customer Status")
plt.ylabel("Number of Customers")
for container in ax.containers:
 ax.bar_label(container)
plt.tight_layout()
plt.show()
```

```
# Notebook cell 123
plt.figure(figsize=(8, 5))
plt.hist(eda_df["Age"],bins=30, edgecolor="black")
plt.title("Customer Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
```

```
plt.show()
```

```
# Notebook cell 126
```

```
churn_counts = (
 eda_df["Churn"]
    .map({
        0: "Stayed",
        1: "Churned"
    })
    .value_counts()
)

plt.figure(figsize=(7, 7))
plt.pie(
    churn_counts.values,
    labels=churn_counts.index,
    autopct="%.1f%%",
    startangle=90
)
plt.title("Customer Churn Percentage")
plt.show()
```

```
# Notebook cell 129
```

```
tenure_churn = (eda_df.groupby("Tenure in Months")["Churn"].mean().reset_index())
tenure_churn["Churn Rate"] = (tenure_churn["Churn"] * 100)
plt.figure(figsize=(10, 5))
plt.plot(
    tenure_churn["Tenure in Months"],
    tenure_churn["Churn Rate"],
    marker="o"
)
plt.title("Churn Rate by Tenure")
plt.xlabel("Tenure in Months")
plt.ylabel("Churn Rate (%)")
plt.show()
```

```
# Notebook cell 132
```

```
tenure_counts = (eda_df.groupby(["Tenure in Months",
"Churn"]).size().unstack(fill_value=0).sort_index())
x = tenure_counts.index
stayed_counts = (
    tenure_counts[0]
    if 0 in tenure_counts.columns
    else np.zeros(len(x))
)
churned_counts = (
    tenure_counts[1]
    if 1 in tenure_counts.columns
    else np.zeros(len(x))
)
plt.figure(figsize=(10, 6))
plt.stackplot(
    x,
    stayed_counts,
    churned_counts,
    labels=[
        "Stayed",
        "Churned"
    ],
    alpha=0.7
)
plt.title("Customer Status by Tenure")
plt.xlabel("Tenure in Months")
plt.ylabel("Customer Count")
plt.legend()
plt.show()
```

```
# Notebook cell 135
```

```
plt.figure(figsize=(8, 5))
sns.kdeplot(
```

```
    data=eda_df,
    x="Monthly Charge",
    hue="Churn",
    fill=True,
    common_norm=False,
    alpha=0.4
)
plt.title("Monthly Charge Distribution by Churn")
plt.show()
```

```
# Notebook cell 138
plt.figure(figsize=(8, 5))
sns.ecdfplot(
    data=eda_df,
    x="Tenure in Months",
    hue="Churn"
)
plt.title("ECDF of Tenure by Churn")
plt.xlabel("Tenure in Months")
plt.ylabel("Cumulative Proportion")
plt.show()
```

```
# Notebook cell 141
swarm_df = eda_df.sample(min(500, len(eda_df)), random_state=42)
plt.figure(figsize=(8, 5))
sns.swarmplot(
    data=swarm_df,
    x="Churn",
    y="Monthly Charge",
    size=3
)
plt.title("Swarm Plot of Monthly Charges")
plt.show()
```

```
# Notebook cell 144
plt.figure(figsize=(8, 5))
sns.boxenplot(data=eda_df, x="Churn", y="Total Charges")
plt.title("Total Charges Distribution by Churn")
plt.show()
```

```
# Notebook cell 147
sns.lmplot(
    data=eda_df,
    x="Tenure in Months",
    y="Total Charges",
    hue="Churn",
    height=6,
    aspect=1.3,
    scatter_kws={
        "alpha": 0.3
    }
)
plt.title("Tenure vs Total Charges by Churn")
plt.show()
```

```
# Notebook cell 150
sns.jointplot(
    data=eda_df,
    x="Tenure in Months",
    y="Monthly Charge",
    hue="Churn",
    kind="scatter"
)
plt.show()
```

```
# Notebook cell 153
pair_cols = [
    "Age",
    "Tenure in Months",
    "Monthly Charge",
    "Total Charges",
    "Number of Referrals",
```

```
    "Churn"
]
sns.pairplot(
    eda_df[pair_cols],
    hue="Churn",
    diag_kind="hist"
)
plt.show()
```

```
# Notebook cell 156
```

```
from sklearn.preprocessing import MinMaxScaler

parallel_cols = [
    "Age",
    "Tenure in Months",
    "Monthly Charge",
    "Total Charges",
    "Number of Referrals"
]

parallel_df = eda_df[parallel_cols + ["Churn"]].copy()

scaler = MinMaxScaler()

parallel_df[parallel_cols] = scaler.fit_transform(parallel_df[parallel_cols])
parallel_df["Churn"] = parallel_df["Churn"].astype("category")

parallel_sample = parallel_df.sample(
    min(500, len(parallel_df)),
    random_state=42
)

pd.plotting.parallel_coordinates(parallel_sample, "Churn", alpha=0.3)
plt.title("Parallel Coordinates")
plt.show()
```

```
# Notebook cell 159
```

```
numeric_eda = eda_df.select_dtypes(include=["int64", "float64"])
corr = numeric_eda.corr()
plt.figure(figsize=(14, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)
plt.title("Correlation Heatmap")
plt.show()
```

```
# Notebook cell 162
```

```
sns.clustermap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    figsize=(12, 12)
)
plt.show()
```

```
# Notebook cell 165
```

```
contract_churn =
(eda_df.groupby("Contract")["Churn"].mean().sort_values(ascending=False)* 100)
plt.figure(figsize=(9, 5))
sns.barplot(x=contract_churn.index, y=contract_churn.values)
plt.title("Churn Rate by Contract")
plt.xlabel("Contract Type")
plt.ylabel("Churn Rate (%)")
```

```
plt.xticks(rotation=20)
plt.show()
```

```
# Notebook cell 168
```

```
charge_bins = [
    0,
    30,
    50,
    70,
    90,
    120,
    np.inf
]

charge_labels = [
    "0-30",
    "31-50",
    "51-70",
    "71-90",
    "91-120",
    "120+"
]

temp = eda_df.copy()
temp["Charge Bucket"] = pd.cut(temp["Monthly Charge"], bins=charge_bins,
labels=charge_labels, include_lowest=True)
charge_churn = (
    temp
    .groupby(
        "Charge Bucket",
        observed=False
    )["Churn"]
    .mean()
    * 100
)
plt.figure(figsize=(10, 5))
sns.barplot(x=charge_churn.index, y=charge_churn.values)
plt.title("Churn Rate by Monthly Charge")
plt.xlabel("Monthly Charge Range")
plt.ylabel("Churn Rate (%)")
plt.show()
```

```
# Notebook cell 171
```

```
feature_list = [
    "Age",
    "Tenure in Months",
    "Monthly Charge",
    "Total Charges",
    "Total Revenue"
]

for feature in feature_list:

    plt.figure(figsize=(8, 5))
    sns.boxplot(
        data=eda_df,
        x="Churn",
        y=feature
    )
    plt.title(f"{feature} Distribution by Churn")
    plt.xlabel("Churn")
    plt.ylabel(feature)
    plt.show()
```

```
# Notebook cell 173
```

```
from sklearn.utils.class_weight import compute_class_weight

classes = np.unique(y_train)
```

```
weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = dict(
    zip(classes, weights)
)



print(class_weights)
```

Stage 4: CTGAN, TVAE and WGAN-GP synthetic data generation

```
# Notebook cell 178
catboost_numeric_medians = (
    X_train[numeric_features]
    .median(numeric_only=True)
    .to_dict()
)

catboost_categorical_fill = {}

for col in categorical_features:

    mode = (
        X_train[col]
        .dropna()
        .astype(str)
        .mode()
    )

    catboost_categorical_fill[col] = (
        str(mode.iloc[0])
        if len(mode)
        else "Unknown"
    )
```

```
# Notebook cell 179
def prepare_native_catboost(frame):

    out = frame.reindex(
        columns=X_train.columns
    ).copy()

    for col in numeric_features:

        out[col] = (
            pd.to_numeric(
                out[col],
                errors="coerce"
            )
            .fillna(catboost_numeric_medians[col])
        )

    for col in categorical_features:

        out[col] = out[col].map(
            lambda value: str(value).strip()
            if pd.notna(value)
            else catboost_categorical_fill[col]
        )
```

```
    return out
```

```
# Notebook cell 180
```

```
X_train_cb = prepare_native_catboost(
    X_train
)

X_val_cb = prepare_native_catboost(
    X_val
)
print(X_train_cb)
print(X_val_cb)
```

```
# Notebook cell 182
```

```
gan_train = prepare_native_catboost(X_train).copy()
```

```
# Notebook cell 183
```

```
gan_train["Churn"] = (y_train.to_numpy(dtype=int))
```

```
# Notebook cell 184
```

```
gan_discrete_columns = (categorical_features + ["Churn"])
```

```
# Notebook cell 185
```

```
print("GAN training shape:", gan_train.shape)
display(gan_train.head())
```

```
# Notebook cell 187
```

```
assert gan_train.isna().sum().sum() == 0

assert "Churn" not in X_train.columns

assert set(
    gan_train["Churn"].unique()
) == {0, 1}

print("GAN training data is ready.")
```

```
# Notebook cell 189
```

```
from ctgan import CTGAN
```

```
# Notebook cell 190
```

```
ctgan_model = CTGAN(epochs=50, batch_size=500, verbose=True, enable_gpu=False)
```

```
# Notebook cell 191
```

```
ctgan_model.fit(gan_train, discrete_columns=gan_discrete_columns)
```

```
# Notebook cell 192
```

```
synthetic_ctgan = ctgan_model.sample(len(gan_train))
```

```
# Notebook cell 193
```

```
print(synthetic_ctgan.shape)
display(synthetic_ctgan.head())
```

```
# Notebook cell 194
```

```
from pathlib import Path
import joblib

GAN_DIR = Path("gan_models")
GAN_DIR.mkdir(exist_ok=True)

synthetic_ctgan.to_csv(
    GAN_DIR / "synthetic_ctgan.csv",
    index=False
)

joblib.dump(
    ctgan_model,
    GAN_DIR / "ctgan_model.pkl"
)
```

```
# Notebook cell 197
```

```
from ctgan import TVAE
```

```
# Notebook cell 199
tvae_model = TVAE(epochs=50, batch_size=500, verbose=True, enable_gpu=False)
# Notebook cell 201
tvae_model.fit(gan_train, discrete_columns=gan_discrete_columns)
# Notebook cell 202
synthetic_tvae = tvae_model.sample(len(gan_train))
# Notebook cell 203
print(synthetic_tvae.shape)
display(synthetic_tvae.head())
# Notebook cell 204
synthetic_tvae.to_csv(
    GAN_DIR / "synthetic_tvae.csv",
    index=False
)

joblib.dump(
    tvae_model,
    GAN_DIR / "tvae_model.pkl"
)
# Notebook cell 210
import tensorflow as tf
from tensorflow import keras # type: ignore

tf.random.set_seed(42)

wgan_source = gan_train.copy()

wgan_categories = gan_discrete_columns

wgan_numeric = [
    col for col in wgan_source.columns
    if col not in wgan_categories
]
# Notebook cell 212
wgan_scaler = MinMaxScaler(
    feature_range=(-1, 1)
)

numeric_array = wgan_scaler.fit_transform(
    wgan_source[wgan_numeric]
)

wgan_encoder = OneHotEncoder(
    handle_unknown="ignore",
    sparse_output=False
)

categorical_array = wgan_encoder.fit_transform(
    wgan_source[wgan_categories].astype(str)
)

categorical_array = categorical_array * 2 - 1

wgan_real = np.concatenate(
    [numeric_array, categorical_array],
    axis=1
).astype("float32")

print("WGAN Training Shape:", wgan_real.shape)
# Notebook cell 214
noise_dim = 32
data_dim = wgan_real.shape[1]
```

```
generator = keras.Sequential([
    keras.layers.Input(shape=(noise_dim,)),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(
        data_dim,
        activation="tanh"
    )
])
```

```
# Notebook cell 216
```

```
critic = keras.Sequential([
    keras.layers.Input(shape=(data_dim,)),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(128),
    keras.layers.LeakyReLU(
        negative_slope=0.2
    ),

    keras.layers.Dense(1)
])
```

```
# Notebook cell 218
```

```
g_optimizer = keras.optimizers.Adam(
    learning_rate=0.0001,
    beta_1=0.0,
    beta_2=0.9
)

d_optimizer = keras.optimizers.Adam(
    learning_rate=0.0001,
    beta_1=0.0,
    beta_2=0.9
)

LAMBDA_GP = 10.0
CRITIC_STEPS = 3
```

```
# Notebook cell 220
```

```
@tf.function
def wgan_step(real_batch):

    batch_size = tf.shape(real_batch)[0]

    # Train Critic
    for _ in range(CRITIC_STEPS):

        with tf.GradientTape() as disc_tape:

            noise = tf.random.normal(
                (batch_size, noise_dim)
```

```
            )

            fake_batch = tf.stop_gradient(
                generator(noise, training=True)
            )

            real_score = critic(
                real_batch,
                training=True
            )

            fake_score = critic(
                fake_batch,
                training=True
            )

            # Interpolate real and fake
            alpha = tf.random.uniform(
                (batch_size, 1)
            )

            interpolated = (
                real_batch
                + alpha * (fake_batch - real_batch)
            )

            # Gradient penalty
            with tf.GradientTape() as gp_tape:
                gp_tape.watch(interpolated)

                interpolated_score = critic(
                    interpolated,
                    training=True
                )

            gradients = gp_tape.gradient(
                interpolated_score,
                interpolated
            )

            gradient_norm = tf.sqrt(
                tf.reduce_sum(
                    tf.square(gradients),
                    axis=1
                ) + 1e-12
            )

            gp = tf.reduce_mean(
                tf.square(gradient_norm - 1.0)
            )

            critic_loss = (
                tf.reduce_mean(fake_score)
                - tf.reduce_mean(real_score)
                + LAMBDA_GP * gp
            )

        critic_gradients = disc_tape.gradient(
            critic_loss,
            critic.trainable_variables
        )

        d_optimizer.apply_gradients(
```

```
            zip(
                critic_gradients,
                critic.trainable_variables
            )
        )

    # Train Generator
    with tf.GradientTape() as gen_tape:

        noise = tf.random.normal(
            (batch_size, noise_dim)
        )

        generated = generator(
            noise,
            training=True
        )

        generator_loss = -tf.reduce_mean(
            critic(generated, training=True)
        )

    generator_gradients = gen_tape.gradient(
        generator_loss,
        generator.trainable_variables
    )

    g_optimizer.apply_gradients(
        zip(
            generator_gradients,
            generator.trainable_variables
        )
    )

    return critic_loss, generator_loss
```

```
# Notebook cell 222
dataset = (
    tf.data.Dataset
    .from_tensor_slices(wgan_real)
    .shuffle(len(wgan_real), seed=42)
    .batch(64)
)

WGAN_EPOCHS = 10

for epoch in range(WGAN_EPOCHS):

    for batch in dataset:
        critic_loss, generator_loss = wgan_step(
            batch
        )

    print(
        f"Epoch {epoch + 1}/{WGAN_EPOCHS}",
        f"Critic Loss: {float(critic_loss):.4f}",
        f"Generator Loss: {float(generator_loss):.4f}"
    )
```

```
# Notebook cell 224
generated_array = generator(
    tf.random.normal(
        (len(gan_train), noise_dim)
    ),
    training=False
).numpy()
```

```
n_numeric = len(wgan_numeric)

decoded_numeric = (
    wgan_scaler.inverse_transform(
        generated_array[:, :n_numeric]
    )
)

synthetic_wgan = pd.DataFrame(
    decoded_numeric,
    columns=wgan_numeric
)
```

```
# Notebook cell 225
offset = n_numeric

for i, col in enumerate(wgan_categories):

    categories = wgan_encoder.categories_[i]

    width = len(categories)

    block = generated_array[
        :,
        offset:offset + width
    ]

    synthetic_wgan[col] = categories[
        np.argmax(block, axis=1)
    ]

    offset += width

synthetic_wgan["Churn"] = pd.to_numeric(
    synthetic_wgan["Churn"],
    errors="coerce"
)

print(synthetic_wgan.shape)

display(synthetic_wgan.head())
```

```
# Notebook cell 227
synthetic_wgan.to_csv(
    GAN_DIR / "synthetic_wgan.csv",
    index=False
)

generator.save(
    GAN_DIR / "wgan_generator.keras"
)

joblib.dump(
    wgan_scaler,
    GAN_DIR / "wgan_numeric_scaler.pkl"
)

joblib.dump(
    wgan_encoder,
    GAN_DIR / "wgan_categorical_encoder.pkl"
)

joblib.dump(
```

```
    {
        "numeric_cols": wgan_numeric,
        "categorical_cols": wgan_categories,
        "noise_dim": noise_dim
    },
    GAN_DIR / "wgan_metadata.pkl"
)
# Notebook cell 229
display(synthetic_ctgan)
# Notebook cell 230
display(synthetic_tvae)
# Notebook cell 231
display(synthetic_wgan)
# Notebook cell 233
synthetic_datasets = {
    "CTGAN": synthetic_ctgan,
    "TVAE": synthetic_tvae,
    "WGAN-GP": synthetic_wgan
}

for name, data in synthetic_datasets.items():

    print("\nModel:", name)

    print("Shape:", data.shape)

    print(
        "Missing Values:",
        data.isnull().sum().sum()
    )

    print(
        "Churn Distribution:",
        data["Churn"].value_counts(
            normalize=True
        )
    )
# Notebook cell 235
plt.figure(figsize=(11, 6))

sns.kdeplot(
    data=gan_train,
    x="Monthly Charge",
    label="Real Training Data"
)

for name, data in synthetic_datasets.items():

    sns.kdeplot(
        data=data,
        x="Monthly Charge",
        label=name
    )

plt.title(
    "Monthly Charge: Real vs Synthetic"
)

plt.legend()
plt.show()
# Notebook cell 237
def repair_engineered_features(data):
```

```
    data = data.copy()

    data["AverageMonthlySpend"] = (
        data["Total Charges"]
        /
        data["Tenure in Months"].replace(
            0, np.nan
        )
    )

    data["TenureGroup"] = pd.cut(
        data["Tenure in Months"],
        bins=[-np.inf, 6, 12, 24, 48, np.inf],
        labels=[
            "0-6 months",
            "7-12 months",
            "13-24 months",
            "25-48 months",
            "49+ months"
        ]
    ).astype(str)

    return data
```

```
# Notebook cell 239
selected_synthetic = synthetic_datasets["CTGAN"]

n_synthetic = min(
    len(selected_synthetic),
    len(X_train) // 2
)

selected_synthetic = selected_synthetic.sample(
    n=n_synthetic,
    random_state=42
)

X_synth = selected_synthetic.drop(
    columns="Churn"
).reindex(columns=X_train.columns)

y_synth = (
    selected_synthetic["Churn"]
    .astype(int)
)

X_train_augmented = pd.concat(
    [
        X_train.reset_index(drop=True),
        X_synth.reset_index(drop=True)
    ],
    ignore_index=True
)

y_train_augmented = pd.concat(
    [
        y_train.reset_index(drop=True),
        y_synth.reset_index(drop=True)
    ],
    ignore_index=True
)

print(
    "Augmented Training Size:",
```

```
    X_train_augmented.shape
)
```

```
# Notebook cell 241
from catboost import CatBoostClassifier

X_train_aug_cb = prepare_native_catboost(
    X_train_augmented
)

catboost_gan = CatBoostClassifier(
    iterations=500,
    depth=5,
    learning_rate=0.03,
    loss_function="Logloss",
    eval_metric="AUC",
    random_seed=42,
    verbose=False
)

catboost_gan.fit(
    X_train_aug_cb,
    y_train_augmented,
    cat_features=categorical_features,
    eval_set=(X_val_cb, y_val),
    early_stopping_rounds=40
)
```

```
# Notebook cell 242
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    roc_auc_score
)

val_proba = catboost_gan.predict_proba(
    X_val_cb
)[:, 1]

val_pred = (
    val_proba >= 0.5
).astype(int)

print(
    "Validation Accuracy:",
    accuracy_score(y_val, val_pred)
)

print(
    "Validation F1:",
    f1_score(y_val, val_pred)
)

print(
    "Validation ROC-AUC:",
    roc_auc_score(y_val, val_proba)
)
```

Stage 5 — Machine Learning Models Training #### Continue your existing project using the X_train, y_train, X_val, y_val, and make_preprocessor() variables from your previous Pipeline steps.

```
# Notebook cell 244
X_train_fit = X_train.copy()
y_train_fit = y_train.copy()
```

```
X_train_fit_ready = np.asarray(
    preprocessor.transform(X_train_fit),
    dtype=np.float32
)

X_train_fit_cb = prepare_native_catboost(
    X_train_fit
)

X_val_ready = np.asarray(
    preprocessor.transform(X_val),
    dtype=np.float32
)

X_val_cb = prepare_native_catboost(X_val)
# Notebook cell 246
from sklearn.linear_model import LogisticRegression
# Notebook cell 247
lr_model = LogisticRegression(max_iter=2500, C=1.0, random_state=42)
# Notebook cell 248
lr_model.fit(X_train_fit_ready, y_train_fit)
# Notebook cell 249
lr_probabilities = lr_model.predict_proba(X_val_ready)[:, 1]
# Notebook cell 250
print(lr_probabilities)
# Notebook cell 253
from catboost import CatBoostClassifier
# Notebook cell 254
cat_model = CatBoostClassifier(
    iterations=500,
    depth=5,
    learning_rate=0.03,
    loss_function="Logloss",
    eval_metric="AUC",
    random_seed=42,
    verbose=False,
    allow_writing_files=False
)
# Notebook cell 255
cat_model.fit(
    X_train_fit_cb,
    y_train_fit,
    cat_features=categorical_features,
    eval_set=(X_val_cb, y_val),
    early_stopping_rounds=40
)
# Notebook cell 256
cat_probabilities = cat_model.predict_proba(X_val_cb)[:, 1]
# Notebook cell 257
print(cat_probabilities)
# Notebook cell 259
from xgboost import XGBClassifier
# Notebook cell 260
xgb_model = XGBClassifier(
    n_estimators=400,
    max_depth=4,
    learning_rate=0.03,
    subsample=0.85,
    colsample_bytree=0.85,
    objective="binary:logistic",
    eval_metric="logloss",
    tree_method="hist",
    random_state=42
)
```

```
# Notebook cell 261
xgb_model.fit(X_train_fit_ready, y_train_fit)
# Notebook cell 262
xgb_probabilities = xgb_model.predict_proba(X_val_ready)[:, 1]
# Notebook cell 263
print(xgb_probabilities)
# Notebook cell 266
import tensorflow as tf
from tensorflow import keras # type: ignore

dnn_model = keras.Sequential([
    keras.layers.Input(
        shape=(X_train_fit_ready.shape[1],)
    ),

    keras.layers.Dense(
        128, activation="relu"
    ),

    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.35),

    keras.layers.Dense(
        64, activation="relu"
    ),

    keras.layers.Dropout(0.30),

    keras.layers.Dense(
        1, activation="sigmoid"
    )
])

dnn_model.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        keras.metrics.AUC(name="auc")
    ]
)
dnn_model.summary()
# Notebook cell 267
early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True
)

dnn_history = dnn_model.fit(
    X_train_fit_ready,
    y_train_fit.to_numpy(),

    validation_data=(
        X_val_ready,
        y_val.to_numpy()
    ),

    epochs=70,
    batch_size=32,

    callbacks=[early_stopping]
)
```

```
# Notebook cell 270
import torch
from pytorch_tabnet.tab_model import TabNetClassifier

tabnet_model = TabNetClassifier(
    n_d=16,
    n_a=16,
    n_steps=3,
    gamma=1.3,

    optimizer_fn=torch.optim.Adam,
    optimizer_params={"lr": 0.02},

    seed=42,
    verbose=1
)

tabnet_model.fit(
    X_train=X_train_fit_ready.astype(
        np.float32
    ),

    y_train=y_train_fit.to_numpy(
        dtype=np.int64
    ),

    eval_set=[(
        X_val_ready.astype(np.float32),
        y_val.to_numpy(dtype=np.int64)
    )],

    eval_name=["validation"],
    eval_metric=["auc"],

    max_epochs=80,
    patience=12,
    batch_size=256,
    virtual_batch_size=64
)
# Notebook cell 271
tabnet_probabilities = (
    tabnet_model.predict_proba(
        X_val_ready.astype(np.float32)
    )[:, 1]
)
# Notebook cell 272
print(tabnet_probabilities)
# Notebook cell 275
import copy
from pathlib import Path
import numpy as np
import pandas as pd
import joblib
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
f1_score,
                             roc_auc_score, classification_report,
confusion_matrix,
                             ConfusionMatrixDisplay, RocCurveDisplay)

FT_SEED = 42
```

```
np.random.seed(FT_SEED)
torch.manual_seed(FT_SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(FT_SEED)
ft_device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("FT device:", ft_device)
for name in ("X_train", "X_val", "X_test", "y_train", "y_val", "y_test",
             "preprocessor", "standard_features", "minmax_features"):
    assert name in globals(), f"Missing {name}: preprocessing cells"
```

```
# Notebook cell 276
```

```
ft_std_cols = list(standard_features)
ft_mm_cols = list(minmax_features)
ft_num_cols = ft_std_cols + ft_mm_cols
ft_cat_cols = [c for c in X_train.columns if c not in ft_num_cols]

assert set(ft_num_cols).issubset(X_train.columns)
assert len(ft_num_cols) == len(set(ft_num_cols)), "Numeric column duplicate!"
assert set(ft_num_cols).isdisjoint(ft_cat_cols)
assert set(ft_num_cols + ft_cat_cols) == set(X_train.columns)

print("Numerical columns:", len(ft_num_cols), ft_num_cols)
print("Categorical columns:", len(ft_cat_cols), ft_cat_cols)
```

```
# Notebook cell 277
```

```
def ft_category_series(df, col):
    return df[col].astype("string").fillna("__MISSING__").str.strip().replace("",
"__MISSING__")

ft_vocab = {}
for col in ft_cat_cols:
    unique_values = pd.unique(ft_category_series(X_train, col))
    ft_vocab[col] = {str(value): i + 1 for i, value in enumerate(unique_values)}

ft_cardinalities = [len(ft_vocab[col]) + 1 for col in ft_cat_cols]
print("Embedding sizes:", dict(zip(ft_cat_cols, ft_cardinalities)))
```

```
# Notebook cell 278
```

```
def ft_transform_raw(frame, fitted_preprocessor, std_cols, mm_cols, cat_cols,
vocab):
    frame = frame.copy()
    required = std_cols + mm_cols + cat_cols
    missing = [c for c in required if c not in frame.columns]
    if missing:
        raise ValueError(f"Missing FT input columns: {missing}")

    number_parts = []
    if std_cols:
        number_parts.append(np.asarray(

fitted_preprocessor.named_transformers_["standard"].transform(frame[std_cols]),
            dtype=np.float32))
    if mm_cols:
        number_parts.append(np.asarray(

fitted_preprocessor.named_transformers_["minmax"].transform(frame[mm_cols]),
            dtype=np.float32))
    x_num = (np.concatenate(number_parts, axis=1) if number_parts
             else np.zeros((len(frame), 0), dtype=np.float32))

    x_cat = np.zeros((len(frame), len(cat_cols)), dtype=np.int64)
    for i, col in enumerate(cat_cols):
        x_cat[:, i] = (ft_category_series(frame, col)
                       .map(vocab[col]).fillna(0).to_numpy(dtype=np.int64))
    if not np.isfinite(x_num).all():
        raise ValueError("Numeric features contain NaN/inf; check fitted imputer
and features")
    return x_num, x_cat
```

```
ft_transform_kwargs = (preprocessor, ft_std_cols, ft_mm_cols, ft_cat_cols,
ft_vocab)
ft_Xtr_num, ft_Xtr_cat = ft_transform_raw(X_train, *ft_transform_kwargs)
ft_Xva_num, ft_Xva_cat = ft_transform_raw(X_val, *ft_transform_kwargs)
ft_ytr = np.asarray(y_train, dtype=np.float32).reshape(-1)
ft_yva = np.asarray(y_val, dtype=np.float32).reshape(-1)

assert ft_Xtr_num.shape[0] == len(ft_ytr)
assert ft_Xtr_cat.shape == (len(ft_ytr), len(ft_cat_cols))
assert all(ft_Xtr_cat[:, i].max() < size for i, size in
enumerate(ft_cardinalities))
print("Train numeric:", ft_Xtr_num.shape, "categorical:", ft_Xtr_cat.shape)
print("Val numeric:", ft_Xva_num.shape, "categorical:", ft_Xva_cat.shape)
```

```
# Notebook cell 279
FT_BATCH_SIZE = 128

def ft_loader(x_num, x_cat, y=None, shuffle=False):
    tensors = [torch.from_numpy(x_num), torch.from_numpy(x_cat)]
    if y is not None:
        tensors.append(torch.from_numpy(np.asarray(y, dtype=np.float32)))
    return DataLoader(TensorDataset(*tensors), batch_size=FT_BATCH_SIZE,
                      shuffle=shuffle, num_workers=0)

ft_train_loader = ft_loader(ft_Xtr_num, ft_Xtr_cat, ft_ytr, shuffle=True)
ft_val_loader = ft_loader(ft_Xva_num, ft_Xva_cat, ft_yva, shuffle=False)
print("Training batches:", len(ft_train_loader), "Validation batches:",
len(ft_val_loader))
```

```
# Notebook cell 280
class TelecomFTTransformer(nn.Module):
    def __init__(self, n_numeric, cardinalities, d_token=32,
                 n_heads=4, n_layers=2, dropout=0.15):
        super().__init__()
        if d_token % n_heads:
            raise ValueError("d_token must be divisible by n_heads")
        self.num_weight = nn.Parameter(torch.randn(n_numeric, d_token) * 0.02)
        self.num_bias = nn.Parameter(torch.zeros(n_numeric, d_token))
        self.cat_embeddings = nn.ModuleList(
            [nn.Embedding(n_categories, d_token) for n_categories in
cardinalities])
        self.cat_bias = nn.Parameter(torch.zeros(len(cardinalities), d_token))
        self.cls = nn.Parameter(torch.zeros(1, 1, d_token))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token, nhead=n_heads, dim_feedforward=4*d_token,
            dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers, enable_nested_tensor=False)
        self.head = nn.Sequential(nn.LayerNorm(d_token), nn.Linear(d_token, 1))

    def forward(self, x_num, x_cat):
        numeric_tokens = x_num.unsqueeze(-1)*self.num_weight + self.num_bias
        tokens = [numeric_tokens]
        for i, embedding in enumerate(self.cat_embeddings):
            tokens.append(embedding(x_cat[:, i]).unsqueeze(1) + self.cat_bias[i])
        features = torch.cat(tokens, dim=1)
        cls_token = self.cls.expand(features.size(0), -1, -1)
        encoded = self.encoder(torch.cat([cls_token, features], dim=1))
        return self.head(encoded[:, 0]).squeeze(-1)  # logit; no sigmoid here

FT_CONFIG = dict(d_token=32, n_heads=4, n_layers=2, dropout=0.15)
ft_model = TelecomFTTransformer(
    n_numeric=len(ft_num_cols), cardinalities=ft_cardinalities, **FT_CONFIG
).to(ft_device)
print(ft_model)
```

```
# Notebook cell 281
ft_n0 = int((ft_ytr == 0).sum())
ft_n1 = int((ft_ytr == 1).sum())
if ft_n1 == 0 or ft_n0 == 0:
    raise ValueError("Both target classes must be present in training data")
ft_pos_weight = ft_n0 / ft_n1
ft_loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(ft_pos_weight,
device=ft_device))
ft_optimizer = torch.optim.AdamW(ft_model.parameters(), lr=8e-4, weight_decay=1e-4)
FT_MAX_EPOCHS, FT_PATIENCE = 35, 7
print("No churn:", ft_n0, "Churn:", ft_n1, "pos_weight:", round(ft_pos_weight, 3))
```

```
# Notebook cell 282
def ft_predict_proba(model, loader, device):
    model.eval()
    output = []
    with torch.inference_mode():
        for batch in loader:
            x_num, x_cat = batch[:2]
            logits = model(x_num.to(device), x_cat.to(device))
            output.append(torch.sigmoid(logits).cpu().numpy())
    return np.concatenate(output) if output else np.array([], dtype=np.float32)
```

```
# Notebook cell 284
ft_checkpoint = Path("models/telecom_churn_ft_transformer.pt")
ft_checkpoint.parent.mkdir(parents=True, exist_ok=True)
ft_history = []
ft_best_auc = -np.inf
ft_no_improve = 0
ft_best_epoch = 0

for epoch in range(1, FT_MAX_EPOCHS + 1):
    ft_model.train()
    train_loss_sum = 0.0
    for x_num, x_cat, y_batch in ft_train_loader:
        x_num = x_num.to(ft_device)
        x_cat = x_cat.to(ft_device)
        y_batch = y_batch.to(ft_device)
        ft_optimizer.zero_grad(set_to_none=True)
        logits = ft_model(x_num, x_cat)
        loss = ft_loss_fn(logits, y_batch)
        loss.backward()
        nn.utils.clip_grad_norm_(ft_model.parameters(), max_norm=1.0)
        ft_optimizer.step()
        train_loss_sum += loss.item() * len(y_batch)
    train_loss = train_loss_sum / len(ft_ytr)

    ft_model.eval()
    val_loss_sum = 0.0
    with torch.inference_mode():
        for x_num, x_cat, y_batch in ft_val_loader:
            logits = ft_model(x_num.to(ft_device), x_cat.to(ft_device))
            val_loss_sum += (ft_loss_fn(logits, y_batch.to(ft_device)).item()
                             * len(y_batch))
    val_loss = val_loss_sum / len(ft_yva)
    val_probs = ft_predict_proba(ft_model, ft_val_loader, ft_device)
    val_auc = roc_auc_score(ft_yva.astype(int), val_probs)
    ft_history.append({"epoch": epoch, "train_loss": train_loss,
                       "val_loss": val_loss, "val_auc": val_auc})
    print(f"Epoch {epoch:02d} | train_loss={train_loss:.4f} | "
          f"val_loss={val_loss:.4f} | val_auc={val_auc:.4f}")

    if val_auc > ft_best_auc + 1e-4:
        ft_best_auc, ft_best_epoch, ft_no_improve = val_auc, epoch, 0
        torch.save(ft_model.state_dict(), ft_checkpoint)
    else:
        ft_no_improve += 1
        if ft_no_improve >= FT_PATIENCE:
```

```
            print("Early stopping at epoch", epoch)
            break

ft_model.load_state_dict(torch.load(ft_checkpoint, map_location=ft_device,
weights_only=True))
ft_model.eval()
print("Best epoch:", ft_best_epoch, "Best val AUC:", round(ft_best_auc, 4))
```

```
# Notebook cell 286
ft_history_df = pd.DataFrame(ft_history)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(ft_history_df["epoch"], ft_history_df["train_loss"], label="Train")
axes[0].plot(ft_history_df["epoch"], ft_history_df["val_loss"], label="Validation")
axes[0].set(title="FT-Transformer Loss", xlabel="Epoch", ylabel="Loss")
axes[0].legend()
axes[1].plot(ft_history_df["epoch"], ft_history_df["val_auc"])
axes[1].set(title="Validation ROC-AUC", xlabel="Epoch", ylabel="ROC-AUC")
plt.tight_layout()
plt.show()
```

```
# Notebook cell 288
ft_val_probs = ft_predict_proba(ft_model, ft_val_loader, ft_device)

def ft_evaluate(y_true, probabilities, threshold, split_name):
    truth = np.asarray(y_true, dtype=int).reshape(-1)
    pred = (probabilities >= threshold).astype(int)
    return {"Model": "FT-Transformer", "Split": split_name,
            "Threshold": float(threshold), "Accuracy": accuracy_score(truth, pred),
            "Precision": precision_score(truth, pred, zero_division=0),
            "Recall": recall_score(truth, pred, zero_division=0),
            "F1-score": f1_score(truth, pred, zero_division=0),
            "ROC-AUC": roc_auc_score(truth, probabilities)}

ft_val_default = ft_evaluate(ft_yva, ft_val_probs, 0.5, "Validation")
ft_tuning_rows = [ft_evaluate(ft_yva, ft_val_probs, threshold, "Validation")
                  for threshold in np.arange(0.20, 0.801, 0.01)]
ft_tuning_table = pd.DataFrame(ft_tuning_rows).sort_values(
    ["F1-score", "Recall", "Accuracy"], ascending=False).reset_index(drop=True)
ft_best_threshold = round(float(ft_tuning_table.loc[0, "Threshold"]), 2)
ft_val_tuned = ft_evaluate(ft_yva, ft_val_probs, ft_best_threshold, "Validation")
print("Default threshold:")
display(pd.DataFrame([ft_val_default]).round(4))
print("F1-tuned threshold:", ft_best_threshold)
display(pd.DataFrame([ft_val_tuned]).round(4))
```

```
# Notebook cell 290
ft_Xte_num, ft_Xte_cat = ft_transform_raw(X_test, *ft_transform_kwargs)
ft_yte = np.asarray(y_test, dtype=np.float32).reshape(-1)
ft_test_loader = ft_loader(ft_Xte_num, ft_Xte_cat, y=None, shuffle=False)
ft_test_probs = ft_predict_proba(ft_model, ft_test_loader, ft_device)
ft_test_pred = (ft_test_probs >= ft_best_threshold).astype(int)
ft_test_metrics = ft_evaluate(ft_yte, ft_test_probs, ft_best_threshold, "Test")
display(pd.DataFrame([ft_test_metrics]).round(4))
print(classification_report(ft_yte.astype(int), ft_test_pred,
      target_names=["Stayed", "Churned"], zero_division=0))
fig, ax = plt.subplots(figsize=(5, 4))
ConfusionMatrixDisplay.from_predictions(ft_yte.astype(int), ft_test_pred,
   display_labels=["Stayed", "Churned"], ax=ax, colorbar=False)
plt.title("FT-Transformer — Test Confusion Matrix")
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(5, 4))
RocCurveDisplay.from_predictions(ft_yte.astype(int), ft_test_probs, ax=ax,
                                 name="FT-Transformer")
plt.title("FT-Transformer — Test ROC Curve")
plt.tight_layout()
plt.show()
```

```
# Notebook cell 292
ft_metadata = {"input_columns": list(X_train.columns),
               "standard_features": ft_std_cols, "minmax_features": ft_mm_cols,
               "categorical_features": ft_cat_cols, "vocab": ft_vocab,
               "cardinalities": ft_cardinalities, "model_config": FT_CONFIG,
               "n_numeric": len(ft_num_cols), "threshold": ft_best_threshold,
               "best_epoch": ft_best_epoch, "best_validation_auc":
float(ft_best_auc),
               "training_variant": "Real training data only"}
joblib.dump(ft_metadata, "models/ft_transformer_metadata.pkl")
joblib.dump(preprocessor, "models/telecom_preprocessor.pkl")
pd.DataFrame([ft_val_default, ft_val_tuned, ft_test_metrics]).to_csv(
    "models/ft_transformer_metrics.csv", index=False)
ft_history_df.to_csv("models/ft_transformer_training_history.csv", index=False)
print("Saved:", ft_checkpoint)
print("Saved: models/ft_transformer_metadata.pkl")
print("Saved: models/telecom_preprocessor.pkl")
```

```
# Notebook cell 294
ft_loaded_meta = joblib.load("models/ft_transformer_metadata.pkl")
ft_loaded_preprocessor = joblib.load("models/telecom_preprocessor.pkl")
ft_loaded_model = TelecomFTTransformer(
    n_numeric=ft_loaded_meta["n_numeric"],
    cardinalities=ft_loaded_meta["cardinalities"],
    **ft_loaded_meta["model_config"]
).to(ft_device)
ft_loaded_model.load_state_dict(torch.load(
    "models/telecom_churn_ft_transformer.pt", map_location=ft_device,
weights_only=True))
ft_loaded_model.eval()

num, cat = ft_transform_raw(
    X_val.head(5), ft_loaded_preprocessor,
    ft_loaded_meta["standard_features"], ft_loaded_meta["minmax_features"],
    ft_loaded_meta["categorical_features"], ft_loaded_meta["vocab"])
ft_loaded_probs = ft_predict_proba(
    ft_loaded_model, ft_loader(num, cat, y=None), ft_device)
np.testing.assert_allclose(ft_loaded_probs, ft_val_probs[:5], rtol=1e-5, atol=1e-6)
print("Loaded model predictions match original model")
print(pd.DataFrame({"Probability of Churn": ft_loaded_probs,
                    "Predicted Churn": (ft_loaded_probs >=
ft_loaded_meta["threshold"]).astype(int)}))
```

```
# Notebook cell 296
plt.figure(figsize=(8, 5))

plt.plot(
    dnn_history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    dnn_history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title(
    "Advanced DNN Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()

plt.show()
```

```
# Notebook cell 297
### DNN loss graph
# Notebook cell 298
plt.figure(figsize=(8, 5))

plt.plot(
    dnn_history.history["loss"],
    label="Training Loss"
)

plt.plot(
    dnn_history.history["val_loss"],
    label="Validation Loss"
)

plt.title(
    "Advanced DNN Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()

plt.show()
```

Stage 06: Final Model Evaluation & Comparison.

```
# Notebook cell 301
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    precision_recall_curve,
    average_precision_score
)

from sklearn.inspection import permutation_importance

sns.set_theme(style="whitegrid")
os.makedirs("evaluation_results", exist_ok=True)
# Notebook cell 303
# Use the already fitted preprocessor

def prepare_features(X):
    data = preprocessor.transform(X)

    if hasattr(data, "toarray"):
        data = data.toarray()

    return np.asarray(data, dtype=np.float32)
```

```
X_val_final = prepare_features(X_val)
X_test_final = prepare_features(X_test)

y_val_final = np.asarray(y_val).ravel()
y_test_final = np.asarray(y_test).ravel()

print("Validation:", X_val_final.shape)
print("Test:", X_test_final.shape)
```

```
# Notebook cell 305
models = {
    "Logistic Regression": lr_model,
    "XGBoost": xgb_model,
    "CatBoost": cat_model,
    "DNN": dnn_model,
    "TabNet": tabnet_model,
    "FT-Transformer": ft_model
}

if "catboost_gan" in globals():
    models["CatBoost + CTGAN"] = catboost_gan
```

```
# Notebook cell 306
def get_probability(name, X):

    model = models[name]

    if name in ["CatBoost", "CatBoost + CTGAN"]:

        X_input = prepare_native_catboost(X)
        probability = model.predict_proba(X_input)[:, 1]

    elif name == "DNN":

        X_input = prepare_features(X)
        probability = model.predict(
            X_input, verbose=0
        ).ravel()

    elif name == "FT-Transformer":

        X_num, X_cat = ft_transform_raw(
            X, *ft_transform_kwargs
        )

        loader = ft_loader(
            X_num, X_cat,
            y=None, shuffle=False
        )

        probability = ft_predict_proba(
            model, loader, ft_device
        )

    else:

        X_input = prepare_features(X)
        probability = model.predict_proba(X_input)[:, 1]

    return np.asarray(probability).ravel()
```

```
# Notebook cell 308
best_thresholds = {}
```

```
for name in models:

    probabilities = get_probability(name, X_val)

    best_f1 = -1
    best_threshold = 0.5

    for threshold in np.arange(0.20, 0.81, 0.05):

        predictions = (
            probabilities >= threshold
        ).astype(int)

        score = f1_score(
            y_val_final,
            predictions,
            zero_division=0
        )

        if score > best_f1:
            best_f1 = score
            best_threshold = float(threshold)

    best_thresholds[name] = best_threshold

    print(
        name,
        "| Threshold:", round(best_threshold, 2),
        "| Validation F1:", round(best_f1, 4)
    )
```

```
# Notebook cell 310
results = []
test_probabilities = {}

for name in models:

    probabilities = get_probability(name, X_test)

    test_probabilities[name] = probabilities

    predictions = (
        probabilities >= best_thresholds[name]
    ).astype(int)

    results.append({
        "Model": name,

        "Accuracy": accuracy_score(
            y_test_final, predictions
        ),

        "Precision": precision_score(
            y_test_final, predictions,
            zero_division=0
        ),

        "Recall": recall_score(
            y_test_final, predictions,
            zero_division=0
        ),

        "F1 Score": f1_score(
```

```
            y_test_final, predictions,
            zero_division=0
        ),

        "ROC-AUC": roc_auc_score(
            y_test_final, probabilities
        )
    })

results_df = pd.DataFrame(results)

display(results_df.round(4))
```

```
# Notebook cell 312
comparison = results_df.sort_values(
    by="F1 Score",
    ascending=False
).reset_index(drop=True)

display(comparison.round(4))
```

```
# Notebook cell 313
comparison.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(
    kind="bar",
    figsize=(12, 6)
)

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
# Notebook cell 315
plt.figure(figsize=(10, 7))

for name, probability in test_probabilities.items():

    fpr, tpr, _ = roc_curve(
        y_test_final, probability
    )

    auc = roc_auc_score(
        y_test_final, probability
    )

    plt.plot(
        fpr, tpr,
        label=f"{name}: {auc:.3f}"
    )

plt.plot([0, 1], [0, 1], "k--")

plt.title("ROC Curve — All Models")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
# Notebook cell 317
import os

os.makedirs("evaluation_results", exist_ok=True)
```

```
comparison.to_csv(
    "evaluation_results/final_model_comparison.csv",
    index=False
)

print("Final evaluation results saved!")
```

Save Best Model for FastAPI Deployment

```
# Notebook cell 320
print("Best Model:", best_thresholds)

best_model_name = comparison.loc[0, "Model"]
best_threshold = best_thresholds[best_model_name]

print("Best Model:", best_model_name)
print("Best Threshold:", round(best_threshold, 3))

best_model = models[best_model_name]
```

```
# Notebook cell 322
import os
import joblib
import torch

os.makedirs("models", exist_ok=True)

if best_model_name == "Logistic Regression":
    model_path = "models/best_model.pkl"
    joblib.dump(best_model, model_path)

elif best_model_name == "XGBoost":
    model_path = "models/best_model.json"
    best_model.save_model(model_path)

elif best_model_name in ["CatBoost", "CatBoost + CTGAN"]:
    model_path = "models/best_model.cbm"
    best_model.save_model(model_path)

elif best_model_name == "DNN":
    model_path = "models/best_model.keras"
    best_model.save(model_path)

elif best_model_name == "TabNet":
    model_path = best_model.save_model("models/best_tabnet")

elif best_model_name == "FT-Transformer":
    model_path = "models/best_ft_transformer.pt"
    torch.save(best_model.state_dict(), model_path)

else:
    raise ValueError("Unsupported model")

print("Best model saved:", model_path)
```

```
# Notebook cell 324
sample_customer = X_test.iloc[[0]]

probability = float(
    get_probability(
        best_model_name,
        sample_customer
    )[0]
```

```
)

prediction = int(
    probability >= best_thresholds[best_model_name]
)

print(f"Churn Probability: {probability:.2%}")

print(
    "Prediction:",
    "Churn" if prediction == 1 else "No Churn"
)
```

Explainable AI (SHAP)

```
# Notebook cell 326
import shap
from catboost import Pool
from IPython.display import display

os.makedirs("evaluation_results/shap", exist_ok=True)

print("SHAP libraries imported!")
```

```
# Notebook cell 328
X_shap = prepare_native_catboost(
    X_val.sample(
        n=min(100, len(X_val)),
        random_state=42
    )
)

shap_pool = Pool(
    X_shap,
    cat_features=categorical_features
)

print("SHAP dataset:", X_shap.shape)

display(X_shap.head())
```

```
# Notebook cell 330
shap_output = cat_model.get_feature_importance(
    data=shap_pool,
    type="ShapValues"
)

shap_values = shap_output[:, :-1]
base_values = shap_output[:, -1]

print("SHAP values:", shap_values.shape)
print("Number of features:", X_shap.shape[1])
```

```
# Notebook cell 332
importance_df = pd.DataFrame({
    "Feature": X_shap.columns,
    "Importance": np.abs(shap_values).mean(axis=0)
})

top20 = importance_df.sort_values(
    by="Importance",
    ascending=False
).head(20)

display(top20)
```

```
# Notebook cell 334
plt.figure(figsize=(10, 8))

plt.barh(
    top20["Feature"][::-1],
    top20["Importance"][::-1]
)

plt.title("Top 20 SHAP Feature Importance")
plt.xlabel("Mean Absolute SHAP Value")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/top20_features.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
# Notebook cell 336
shap.summary_plot(
    shap_values,
    X_shap,
    feature_names=X_shap.columns.tolist(),
    max_display=20,
    show=False
)

plt.title("SHAP Summary — Telecom Churn")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/shap_summary.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
# Notebook cell 338
shap.summary_plot(
    shap_values,
    X_shap,
    plot_type="bar",
    max_display=20,
    show=False
)

plt.title("Global SHAP Feature Importance")
plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/shap_bar.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
# Notebook cell 340
customer_index = 0

customer_shap = shap.Explanation(
    values=shap_values[customer_index],
    base_values=float(base_values[customer_index]),
```

```
    data=X_shap.iloc[customer_index].to_numpy(),
    feature_names=X_shap.columns.tolist()
)

shap.plots.waterfall(
    customer_shap,
    max_display=20,
    show=False
)

plt.tight_layout()

plt.savefig(
    "evaluation_results/shap/customer_waterfall.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
# Notebook cell 342
customer = X_shap.iloc[[0]]

probability = cat_model.predict_proba(
    customer
)[0, 1]

threshold = best_thresholds.get(
    "CatBoost", 0.5
)

prediction = int(probability >= threshold)

print(f"Churn Probability: {probability:.2%}")

print(
    "Prediction:",
    "Churn" if prediction == 1 else "No Churn"
)
# Notebook cell 344
importance_df.to_csv(
    "evaluation_results/shap/feature_importance.csv",
    index=False
)

shap_results = pd.DataFrame(
    shap_values,
    columns=X_shap.columns,
    index=X_shap.index
)

shap_results.to_csv(
    "evaluation_results/shap/customer_shap_values.csv"
)

print("All SHAP results saved successfully!")
```

A.2 FastAPI module: app/model_def.py

```
"""FT-Transformer definition and feature transformation (copied from the
notebook)."""
import numpy as np
import pandas as pd
```

```
import torch
from torch import nn



class TelecomFTTransformer(nn.Module):
    def __init__(self, n_numeric, cardinalities, d_token=32,
                 n_heads=4, n_layers=2, dropout=0.15):
        super().__init__()
        if d_token % n_heads:
            raise ValueError("d_token must be divisible by n_heads")
        self.num_weight = nn.Parameter(torch.randn(n_numeric, d_token) * 0.02)
        self.num_bias = nn.Parameter(torch.zeros(n_numeric, d_token))
        self.cat_embeddings = nn.ModuleList(
            [nn.Embedding(n_categories, d_token) for n_categories in
cardinalities])
        self.cat_bias = nn.Parameter(torch.zeros(len(cardinalities), d_token))
        self.cls = nn.Parameter(torch.zeros(1, 1, d_token))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_token, nhead=n_heads, dim_feedforward=4 * d_token,
            dropout=dropout, activation="gelu", batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=n_layers, enable_nested_tensor=False)
        self.head = nn.Sequential(nn.LayerNorm(d_token), nn.Linear(d_token, 1))

    def forward(self, x_num, x_cat):
        numeric_tokens = x_num.unsqueeze(-1) * self.num_weight + self.num_bias
        tokens = [numeric_tokens]
        for i, embedding in enumerate(self.cat_embeddings):
            tokens.append(embedding(x_cat[:, i]).unsqueeze(1) + self.cat_bias[i])
        features = torch.cat(tokens, dim=1)
        cls_token = self.cls.expand(features.size(0), -1, -1)
        encoded = self.encoder(torch.cat([cls_token, features], dim=1))
        return self.head(encoded[:, 0]).squeeze(-1)  # logit



def category_series(df, col):
    return df[col].astype("string").fillna("__MISSING__").str.strip().replace("",
"__MISSING__")



def transform_raw(frame, fitted_preprocessor, std_cols, mm_cols, cat_cols, vocab):
    """Same logic as ft_transform_raw in the notebook."""
    frame = frame.copy()
    missing = [c for c in std_cols + mm_cols + cat_cols if c not in frame.columns]
    if missing:
        raise ValueError(f"Missing FT input columns: {missing}")
    parts = [

np.asarray(fitted_preprocessor.named_transformers_["standard"].transform(frame[std_
cols]), dtype=np.float32),

np.asarray(fitted_preprocessor.named_transformers_["minmax"].transform(frame[mm_col
s]), dtype=np.float32),
    ]
    x_num = np.concatenate(parts, axis=1)
    x_cat = np.zeros((len(frame), len(cat_cols)), dtype=np.int64)
    for i, col in enumerate(cat_cols):
        x_cat[:, i] = category_series(frame,
col).map(vocab[col]).fillna(0).to_numpy(dtype=np.int64)
    if not np.isfinite(x_num).all():
        raise ValueError("Numeric features contain NaN/inf")
    return x_num, x_cat



def add_engineered_features(df):
```

```
    """Recreate the two engineered features exactly as in the notebook."""
    df = df.copy()
    df["AverageMonthlySpend"] = df["Total Charges"] / df["Tenure in
Months"].replace(0, np.nan)
    df["TenureGroup"] = pd.cut(
        df["Tenure in Months"],
        bins=[-np.inf, 6, 12, 24, 48, np.inf],
        labels=["0-6 months", "7-12 months", "13-24 months", "25-48 months", "49+
months"],
    ).astype(object)
    return df
```

The other API files (`main.py`, `schemas.py`, `predictor.py`, `tests/test_api.py`, `Dockerfile`) are listed in Chapter 14.

Appendix B: requirements.txt

```
# Full-project requirements (notebook + API). Versions are NOT pinned because they
were not
# recorded in the notebook; run 'pip freeze' in the training environment and pin
them.
numpy
pandas
scikit-learn
matplotlib
seaborn
joblib
ctgan
tensorflow
torch
pytorch-tabnet
xgboost
catboost
shap
fastapi
uvicorn[standard]
pydantic>=2
httpx
pytest
```

Package versions were not recorded in the notebook and are therefore not stated here. Run `pip freeze > requirements-frozen.txt` in the environment used for training and replace the unpinned list with those versions before deployment.

Appendix C: Project Folder Structure

```
Telecom_Churn_Project/
  Telecom.ipynb                     # notebook (source of all results)
  telecom_customer_churn.csv        # dataset (Maven Analytics)
  gan_models/                       # created by the notebook
    synthetic_ctgan.csv  ctgan_model.pkl
    synthetic_tvae.csv   tvae_model.pkl
    synthetic_wgan.csv   wgan_generator.keras
    wgan_numeric_scaler.pkl  wgan_categorical_encoder.pkl  wgan_metadata.pkl
  models/                           # created by the notebook
    telecom_churn_ft_transformer.pt
    best_ft_transformer.pt
    ft_transformer_metadata.pkl
    telecom_preprocessor.pkl
    ft_transformer_metrics.csv
    ft_transformer_training_history.csv
  evaluation_results/
    final_model_comparison.csv
    shap/  top20_features.png  shap_summary.png  shap_bar.png
           customer_waterfall.png  feature_importance.csv  customer_shap_values.csv
  fastapi_app/                      # proposed service (Chapter 14)
    app/  tests/  models/  requirements.txt  Dockerfile
```

Appendix D: API Request and Response Examples

Status: illustrative. These examples show the format of requests and responses defined by the design in Chapter 14. They have not been produced by a running service, and numeric outputs are placeholders.

D.1 Single prediction

```
POST /predict
Content-Type: application/json

{ "Gender": "Female", "Age": 37, "Married": "Yes", ... "Total Revenue": 974.81 }

200 OK
{ "churn_probability": 0.123456, "prediction": "No Churn", "threshold": 0.57,
"model_name": "FT-Transformer" }
```

D.2 Validation error (unknown category)

```
422 Unprocessable Entity
{ "detail": ["Contract: unknown value(s) ['Weekly']; allowed ['Month-to-Month',
'One Year', 'Two Year']"] }
```

D.3 Missing field

```
422 Unprocessable Entity
{ "detail": [ { "type": "missing", "loc": ["body", "Contract"], "msg": "Field
required" } ] }
```

D.4 curl example

```
curl -X POST http://127.0.0.1:8000/predict \
     -H "Content-Type: application/json" \
     -H "X-API-Key: change-me" \
     -d @customer.json
```

Appendix E: Additional Evaluation Charts

Additional exploratory plots produced in the notebook (training split).

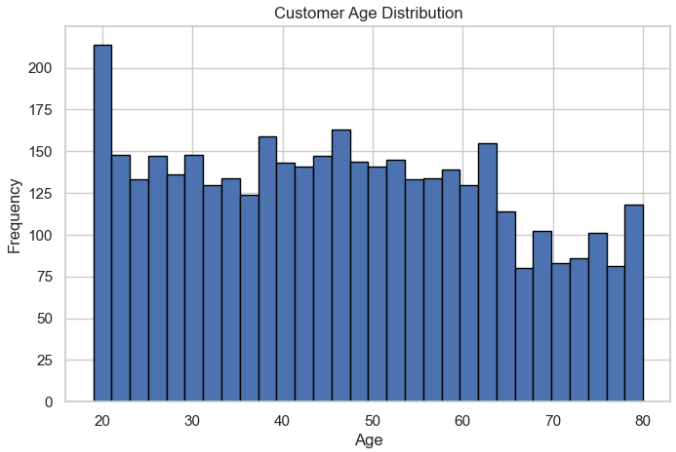


Figure E.1: Customer age distribution

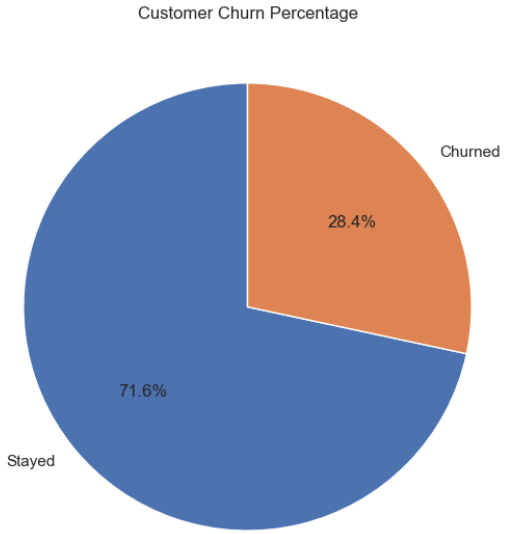


Figure E.2: Customer churn percentage (pie chart)

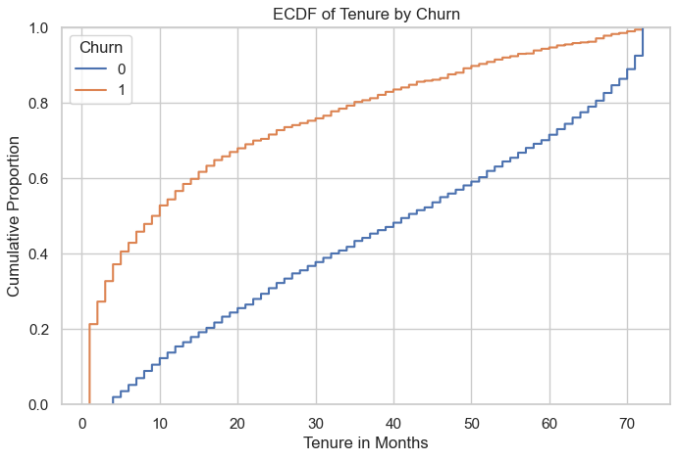


Figure E.3: ECDF of tenure by churn status

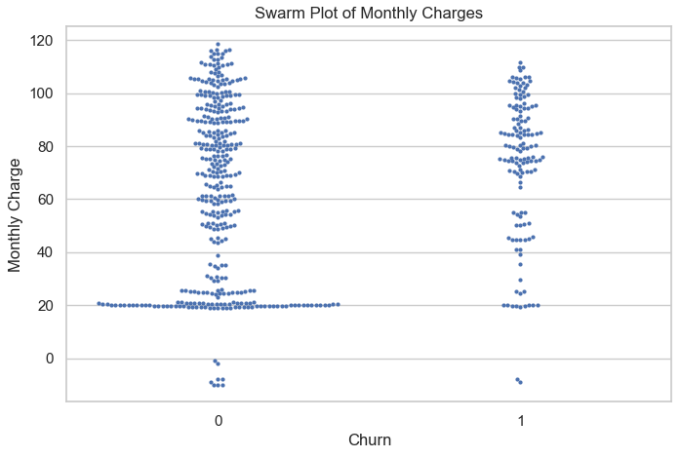


Figure E.4: Swarm plot of monthly charges by churn status (500-customer sample)

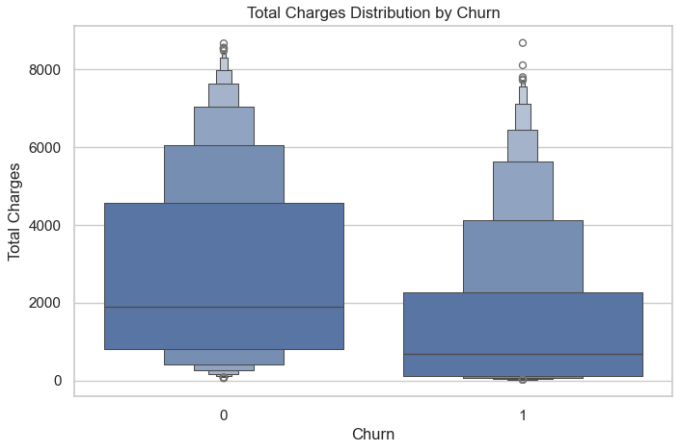


Figure E.5: Total charges distribution by churn status



Figure E.6: Tenure against total charges by churn status

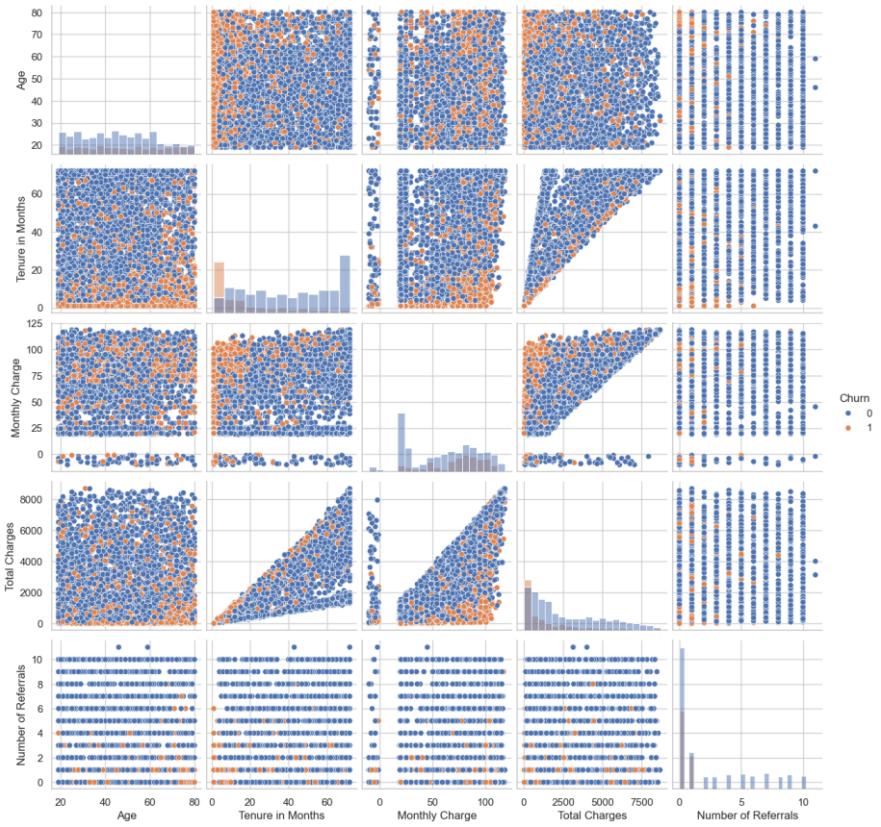


Figure E.7: Pair plot of selected numeric features

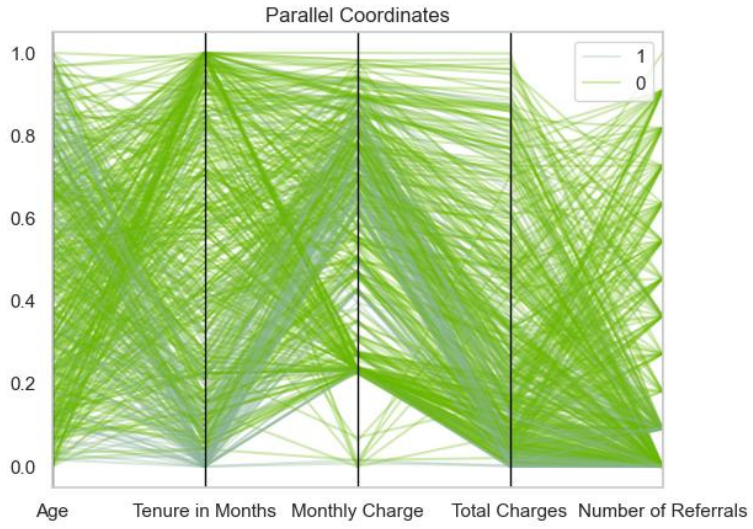


Figure E.8: Parallel coordinates of selected numeric features

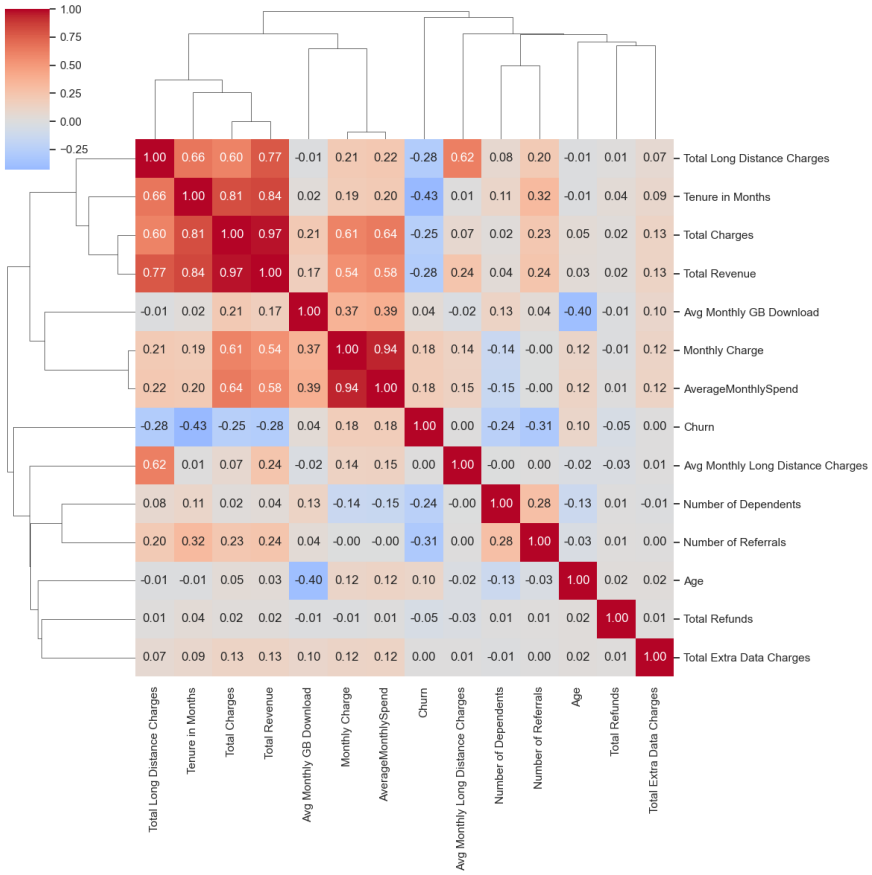


Figure E.9: Clustered correlation heatmap

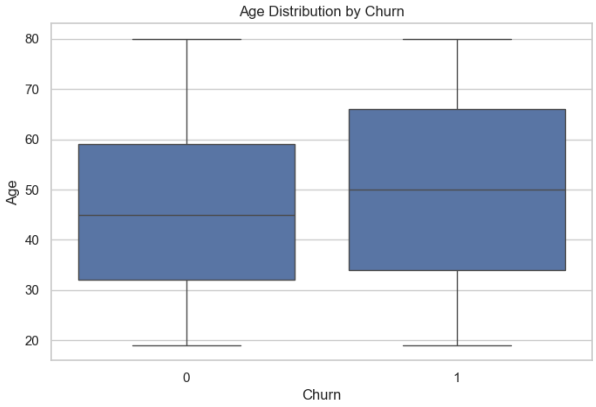


Figure E.10: Age distribution by churn status

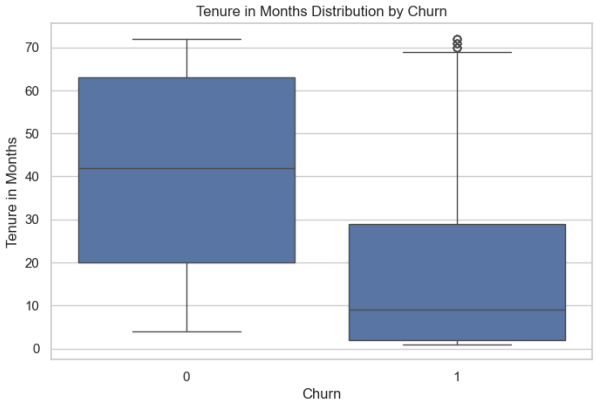


Figure E.11: Tenure distribution by churn status

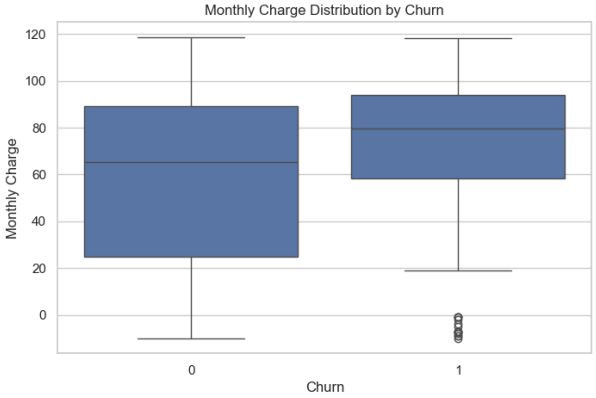


Figure E.12: Monthly charge distribution by churn status

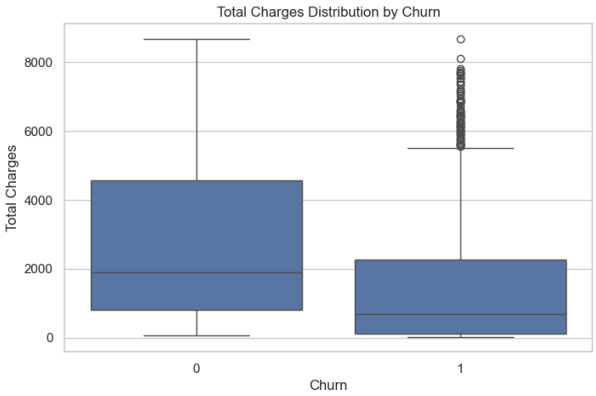


Figure E.13: Total charges distribution by churn status

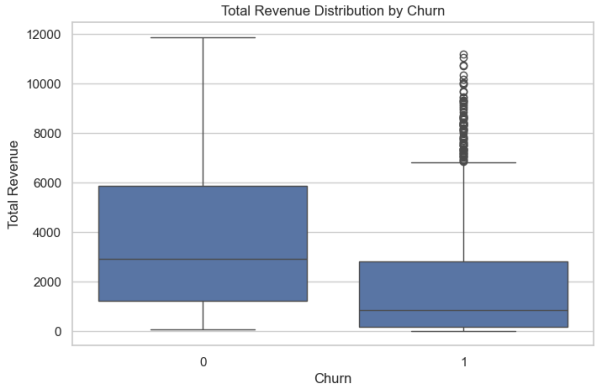


Figure E.14: Total revenue distribution by churn status

Appendix F: Test Cases and Verification Results

Executed notebook verification (Tests T1 to T5 of Table 15.2). API tests T7 to T13 are pending.

Recorded verification output (from the executed notebook):

```
No overlapping rows between splits.
GAN training data is ready.
Loaded model predictions match original model
   Probability of Churn  Predicted Churn
0              0.853664                1
1              0.044485                0
2              0.418618                0
3              0.281435                0
4              0.901074                1

Churn Probability: 21.69%
Prediction: No Churn
```

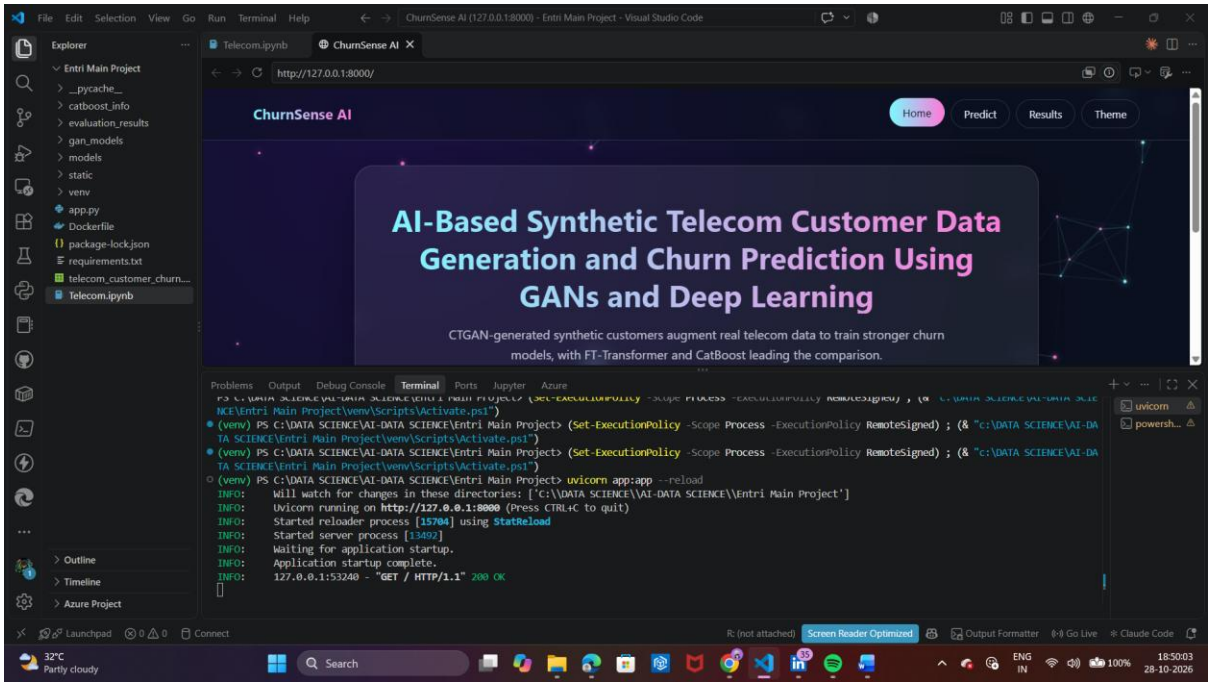
Table E.1: Verification checklist for the final deliverables

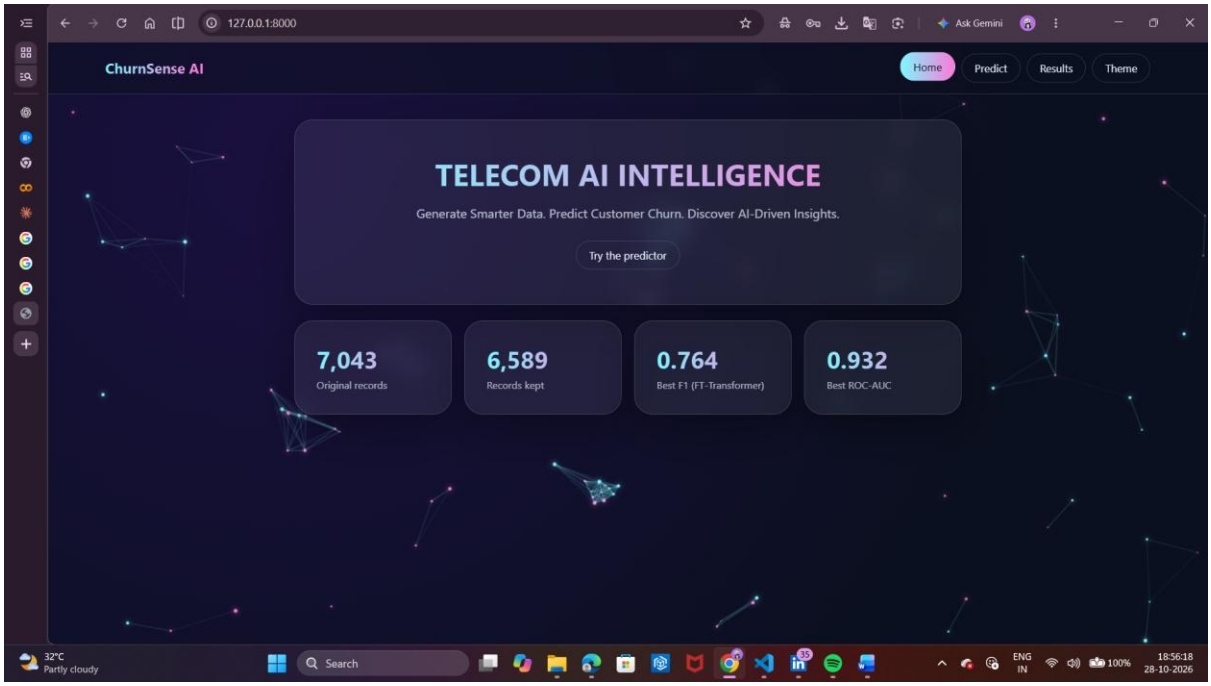
Item	Status
Notebook results reproduced in this report	Copied from recorded outputs; not re-executed
Random Forest results	Not performed
Cross-validation	Not performed
SHAP for FT-Transformer	Pending
API execution and tests	Pending
Placeholders for personal and institutional details	To be completed by the student

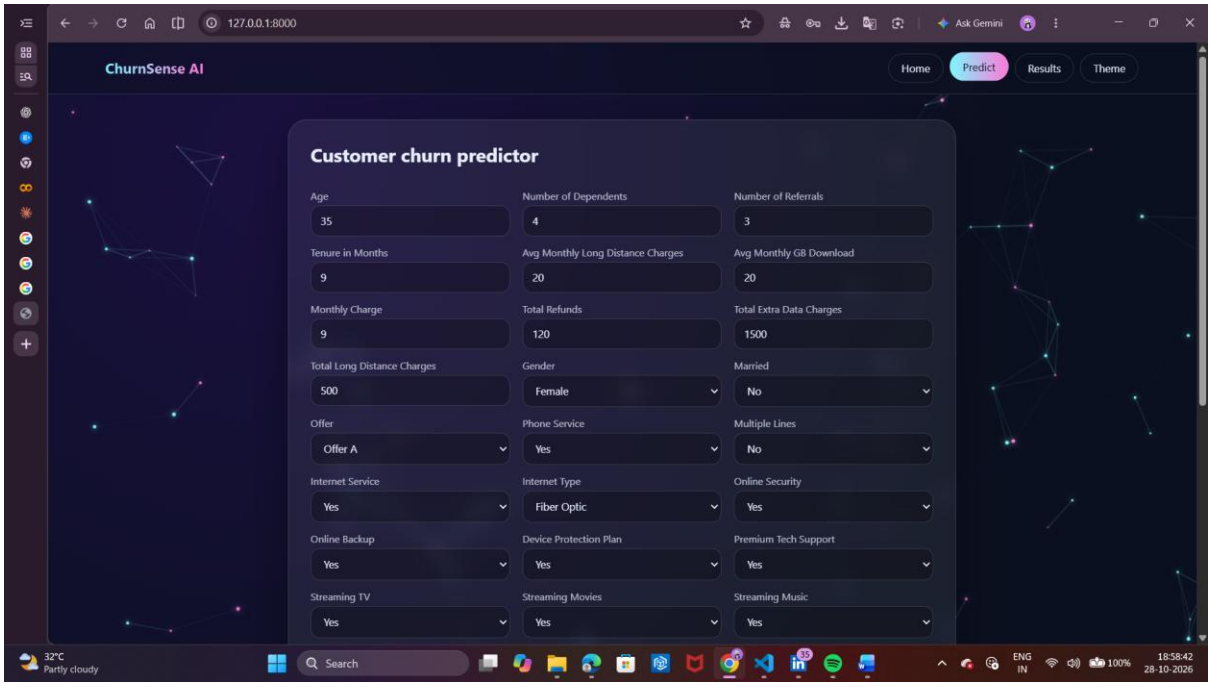
Appendix G: Instructions for Running the Project

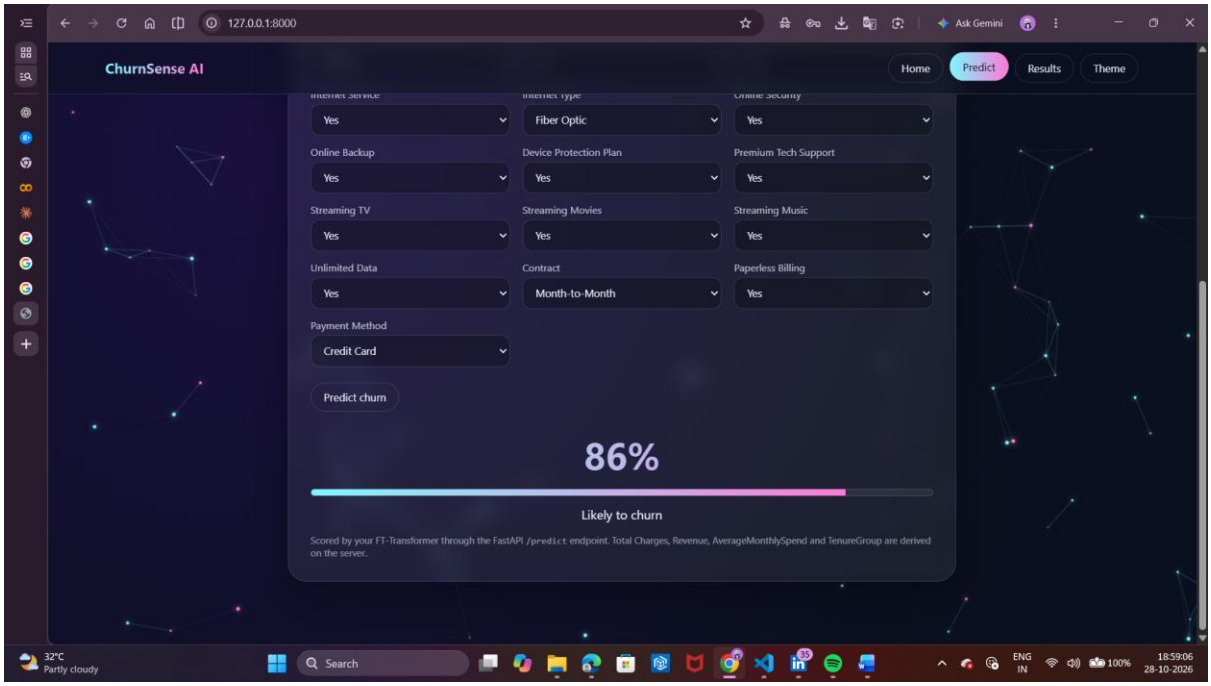
1. Install Python 3.11 and create a virtual environment.
2. Install the packages in Appendix B (`pip install -r requirements.txt`). TensorFlow, PyTorch, CatBoost and CTGAN are large; allow time and disk space.
3. Download `telecom_customer_churn.csv` from the Maven Analytics Data Playground [14] and place it next to the notebook.
4. Open `Telecom.ipynb` in Jupyter and run all cells in order. Training on CPU takes roughly one to several minutes per model (CTGAN: about 53 seconds, as logged).
5. The notebook creates `gan_models/`, `models/` and `evaluation_results/`. Confirm that `models/` contains the three FT-Transformer artifacts.
6. Copy the three artifacts into `fastapi_app/models/` and start the service with `uvicorn app.main:app --reload` from the `fastapi_app` folder.
7. Open `http://127.0.0.1:8000/docs` and run `pytest -q` to execute the API tests.

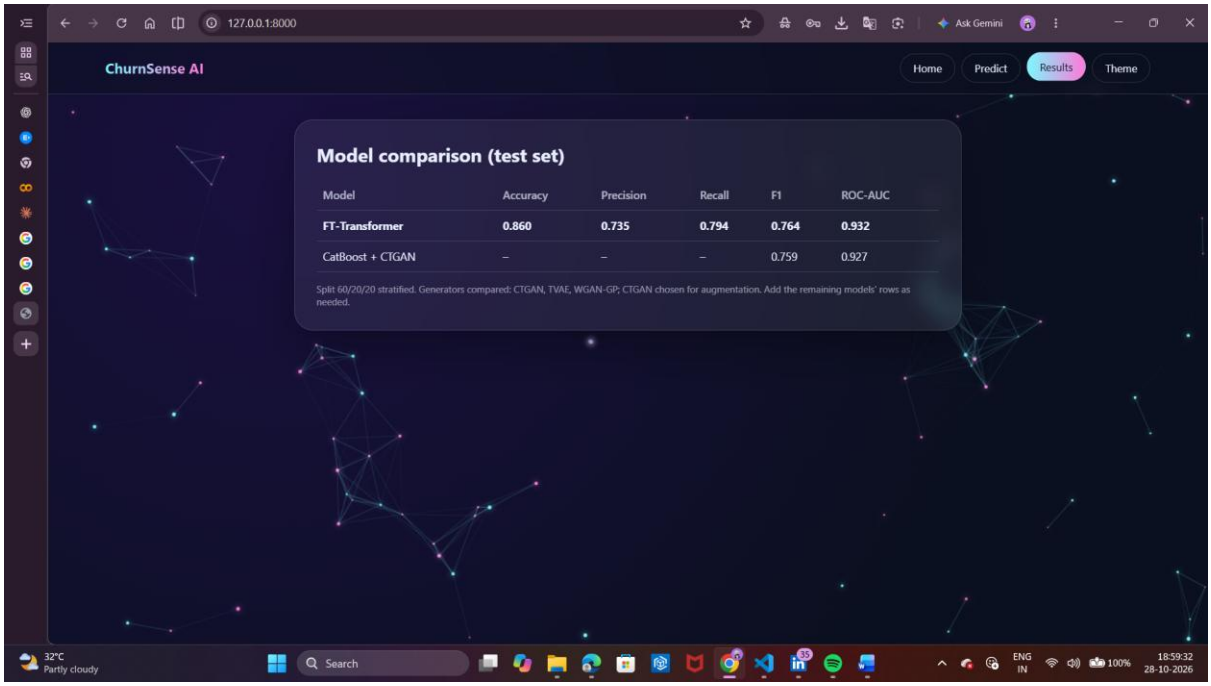
Random seeds are fixed (42) for splits, CatBoost, XGBoost, TabNet and PyTorch; exact numbers may still differ slightly across library versions and hardware.

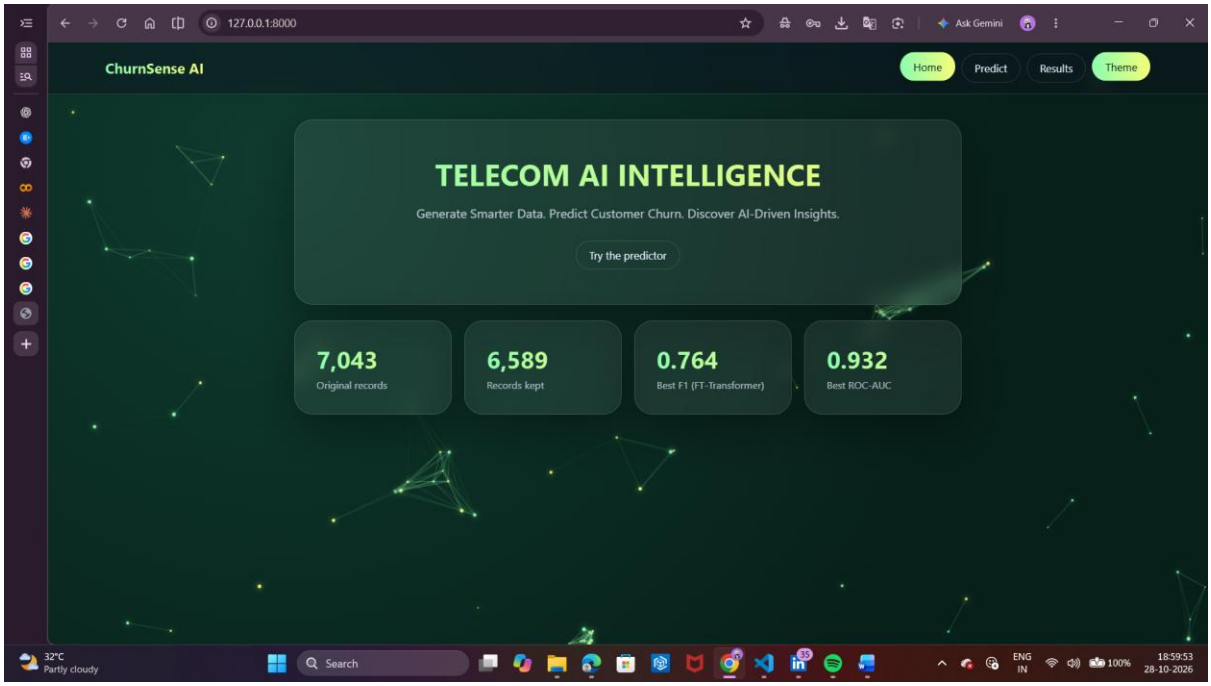


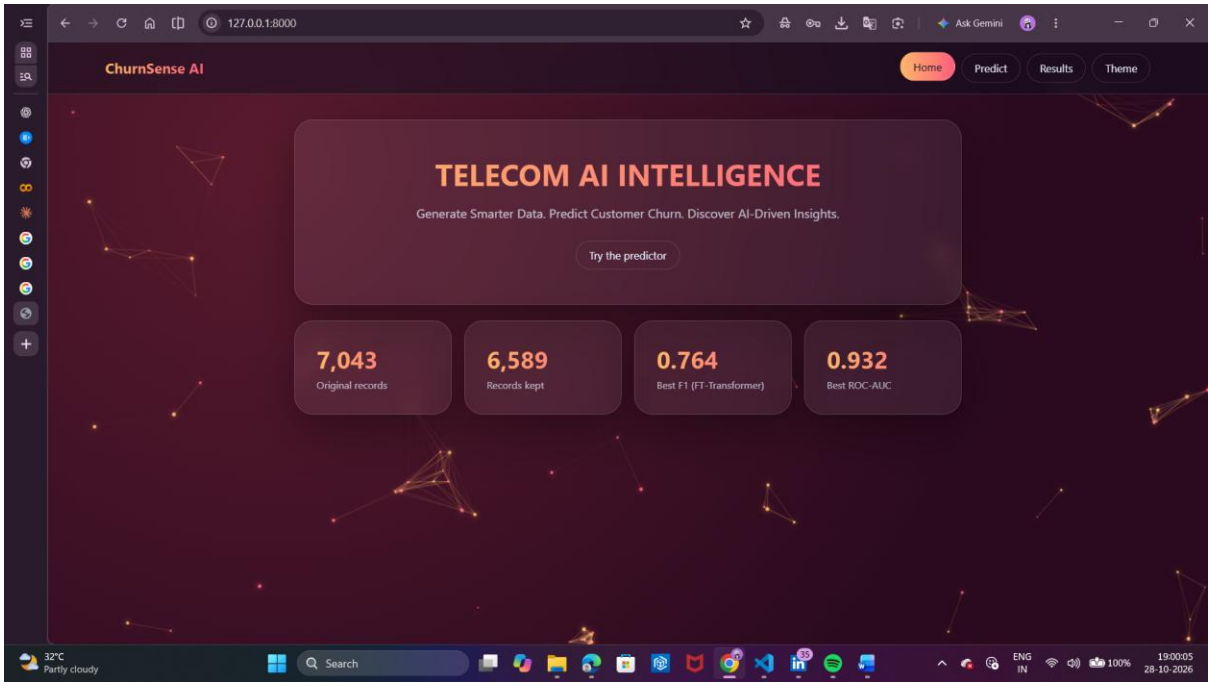

127.0.0.1:8000
ChurnSense AI
Home
Predict
Results
Theme
TELECOM AI INTELLIGENCE
Generate Smarter Data. Predict Customer Churn. Discover AI-Driven Insights.
Try the predictor
7,043
Original records
6,589
Records kept
0.764
Best F1 (FT-Transformer)
0.932
Best ROC-AUC


127.0.0.1:8000
ChurnSense AI
Home
Predict
Results
Theme
Customer churn predictor
Age
35
Number of Dependents
4
Number of Referrals
3
Tenure in Months
9
Avg Monthly Long Distance Charges
20
Avg Monthly GB Download
20
Monthly Charge
9
Total Refunds
120
Total Extra Data Charges
1500
Total Long Distance Charges
500
Gender
Female
Married
No
Offer
Offer A
Phone Service
Yes
Multiple Lines
No
Internet Service
Yes
Internet Type
Fiber Optic
Online Security
Yes
Online Backup
Yes
Device Protection Plan
Yes
Premium Tech Support
Yes
Streaming TV
Yes
Streaming Movies
Yes
Streaming Music
Yes


127.0.0.1:8000
ChurnSense AI
Home
Predict
Results
Theme
Yes
Fiber Optic
Yes
Online Backup
Yes
Device Protection Plan
Yes
Premium Tech Support
Yes
Streaming TV
Yes
Streaming Movies
Yes
Streaming Music
Yes
Unlimited Data
Yes
Contract
Month-to-Month
Paperless Billing
Yes
Payment Method
Credit Card
Predict churn
86%
Likely to churn
Scored by your FT-Transformer through the FastAPI /predict endpoint. Total Charges, Revenue, AverageMonthlySpend and TenureGroup are derived on the server.
32°C
Partly cloudy
Search
ENG
IN
100%
18:59:06
28-10-2026


127.0.0.1:8000
ChurnSense AI
Home
Predict
Results
Theme
Model comparison (test set)
Model
Accuracy
Precision
Recall
F1
ROC-AUC
FT-Transformer
0.860
0.735
0.794
0.764
0.932
CatBoost + CTGAN
–
–
–
0.759
0.927
Split 60/20/20 stratified. Generators compared: CTGAN, TVAE, WGAN-GP; CTGAN chosen for augmentation. Add the remaining models' rows as needed.
32°C
Partly cloudy
Search
ENG
IN
100%
18:59:32
28-10-2026


127.0.0.1:8000
Ask Gemini
ChurnSense AI
Home
Predict
Results
Theme
TELECOM AI INTELLIGENCE
Generate Smarter Data. Predict Customer Churn. Discover AI-Driven Insights.
Try the predictor
7,043
Original records
6,589
Records kept
0.764
Best F1 (FT-Transformer)
0.932
Best ROC-AUC
32°C
Partly cloudy
Search
ENG
IN
100%
18:59:53
28-10-2026


127.0.0.1:8000
Ask Gemini
ChurnSense AI
Home
Predict
Results
Theme
TELECOM AI INTELLIGENCE
Generate Smarter Data. Predict Customer Churn. Discover AI-Driven Insights.
Try the predictor
7,043
Original records
6,589
Records kept
0.764
Best F1 (FT-Transformer)
0.932
Best ROC-AUC
32°C
Partly cloudy
Search
ENG
IN
100%
19:00:05
28-10-2026